راهنمای کاربر

ویرایشگر و تولیدکننده‌ی کد لیست پارامتر

Parameter List Editor & Code Generator

نسخه‌ی ابزار ۰۱.۰۴.۰۱

آبان ۱۴۰۵

راهنمای گام‌به‌گام ساختن پروژه، نگه‌داری لیست پارامتر و تولید کد C

فهرست مطالب

۱. معرفی ۶

۱.۱ این ابزار چه کار می‌کند؟ ۶

۱.۲ روند کار در یک نگاه ۶

۱.۳ پیش‌نیازها ۶

۱.۴ درباره‌ی این راهنما ۷

۲. مفاهیم پایه ۸

۲.۱ پروژه و ساب‌لیست ۸

۲.۲ پارامتر و مقدار ۸

۲.۳ آرایه ۹

۲.۴ استراکچر و تایپ استراکچر ۹

۲.۵ تگ‌ها ۹

۲.۶ شناسه‌ی لاگ ۹

۲.۷ ایندکس ۹

۲.۸ دو قالب خروجی ۱۰

۲.۹ اینام ۱۰

۳. نصب و اجرا ۱۱

۳.۱ نصب ۱۱

۳.۲ اجرای برنامه ۱۱

۳.۳ بررسی رایانه ۱۲

۳.۴ اجرای مستقیم یک کار ۱۳

۳.۵ بستن برنامه ۱۳

۳.۶ اگر ویرایشگر باز نشد ۱۴

۴. شروع سریع: از صفر تا کد C ۱۵

۵. پروژه‌ها ۱۸

۵.۱ ساختار پوشه‌ی پروژه ۱۸

۵.۲ ساختن پروژه‌ی جدید ۱۸

۵.۳ باز کردن پروژه ۱۹

۵.۴ پروژه‌های اخیر و بستن پروژه ۲۰

۵.۵ جابه‌جا کردن، کپی و اشتراک ۲۱

۶. آشنایی با محیط ویرایشگر ۲۲

۶.۱ نمای کلی ۲۲

۶.۲ جدول پارامترها ۲۳

۶.۳ فرم ویرایش ۲۶

۶.۴ پنل Problems و نوار وضعیت ۲۶

۶.۵ تم تیره و روشن ۲۶

۷. کار با پارامترها ۲۷

۷.۱ افزودن پارامتر ۲۷

۷.۲ فیلدهای فرم ۲۸

۷.۳ آرایه‌ها و عنصرها ۲۹

۷.۴ نام نمایشی و توضیح هر ردیف ۳۰

۷.۵ شناسه‌ی لاگ روی یک عنصر ۳۱

۷.۶ تکثیر، جابه‌جایی و حذف ۳۲

۷.۷ غیرفعال کردن پارامتر ۳۲

۷.۸ واگرد، ذخیره و بارگذاری دوباره ۳۲

۸. استراکچرها ۳۴

۸.۱ تایپ استراکچر چیست؟ ۳۴

۸.۲ ساختن استراکچر ۳۴

۸.۳ ویرایش تایپ‌ها ۳۵

۸.۴ سهم تایپ و سهم هر استراکچر ۳۶

۸.۵ آرایه‌ای از استراکچر ۳۷

۸.۶ قواعد ۳۸

۹. اینام‌ها ۳۹

۹.۱ اینام چیست؟ ۳۹

۹.۲ تعریف و ویرایش اینام ۳۹

۹.۳ پارامتری از نوع اینام ۴۱

۹.۴ تغییر اینامی که استفاده شده است ۴۲

۹.۵ اینام در کد C ۴۲

۹.۶ قواعد ۴۳

۱۰. ساب‌لیست‌ها ۴۴

۱۰.۱ چند ساب‌لیست در یک پروژه ۴۴

۱۰.۲ ساختن ساب‌لیست ۴۵
۱۰.۳ افزودن ساب‌لیست موجود ۴۶
۱۰.۴ برداشتن یا حذف ساب‌لیست ۴۶
۱۰.۵ رفتن بین ساب‌لیست‌ها و نمای ادغام‌شده ۴۶

۱۱. تگ‌ها، شناسه‌ی لاگ و لیست‌های کشویی ۴۸

۱۱.۱ تگ‌ها ۴۸
۱۱.۲ مقدار تگ‌ها ۴۹
۱۱.۳ شناسه‌ی لاگ ۴۹
۱۱.۴ آفست حافظه‌ی تگ‌ها ۴۹
۱۱.۵ لیست‌های کشویی ۵۰
۱۱.۶ تگ‌های آزاد و توضیح آن‌ها ۵۲

۱۲. تولید کد ۵۴

۱۲.۱ دکمه‌ی Generate code ۵۴
۱۲.۲ فایل‌های تولیدی ۵۴
۱۲.۳ نام‌ها در کد ۵۵
۱۲.۴ هشدار «چیدمان حافظه عوض شد» ۵۵
۱۲.۵ تولید کد از خط فرمان ۵۷

۱۳. استفاده از کد تولیدی در فرم‌ور ۵۸

۱۳.۱ افزودن فایل‌ها به پروژه‌ی فرم‌ور ۵۸
۱۳.۲ راه‌اندازی ۵۸
۱۳.۳ خواندن و نوشتن یک مقدار ۵۹
۱۳.۴ فیلد استراکچر و عنصر آرایه ۶۰
۱۳.۵ نوشتن همراه با اعلام تغییر ۶۰
۱۳.۶ گشتن در همه‌ی پارامترها ۶۰
۱۳.۷ تابع‌ها و فیلدها ۶۱
۱۳.۸ نمونه‌ی کامل ۶۱

۱۴. قالب جدول پارامتر (param_table) ۶۲

۱۴.۱ این قالب برای چیست؟ ۶۲
۱۴.۲ هر ساب‌لیست، یک زیرسیستم ۶۳
۱۴.۳ محدودیت‌ها ۶۳
۱۴.۴ آدرس ۶۳
۱۴.۵ فایل nsrParamTables.h ۶۴

۱۴.۶ ایندکس در این قالب ۶۵
۱۴.۷ فایل nsrParamOffsets.h ۶۵
۱۴.۸ فایل nsrSystemNames.h ۶۵
۱۴.۹ هشدار تغییر شماره ۶۶

۱۵. فایل تنظیمات پروژه (pl_project.json) ۶۷

۱۵.۱ نگاه کلی ۶۷
۱۵.۲ بخش‌های project و inputs ۶۸
۱۵.۳ بخش output ۶۸
۱۵.۴ بخش style ۶۹
۱۵.۵ بخش codegen ۶۹
۱۵.۶ بخش validation ۷۰
۱۵.۷ بخش tag_descriptions ۷۰
۱۵.۸ پنجره‌ی Project settings ۷۰

۱۶. خطاها و هشدارها ۷۳

۱۶.۱ پنل Problems ۷۳
۱۶.۲ پیام‌های رایج ۷۳

۱۷. تغییر شماره‌ی نسخه‌ی ابزار ۷۶

۱۷.۱ قالب شماره‌ی نسخه ۷۶
۱۷.۲ تغییر نسخه با set_version.bat ۷۶
۱۷.۳ شماره‌ی نسخه کجاها دیده می‌شود؟ ۷۷

۱۸. میان‌برها و پرسش‌های رایج ۷۸

۱۸.۱ میان‌برهای صفحه‌کلید ۷۸
۱۸.۲ پرسش‌های رایج ۷۸

پیوست الف. نوع‌های داده ۸۰

پیوست ب. واژه‌نامه ۸۱

۱. معرفی

۱.۱ این ابزار چه کار می‌کند؟

هر فرم‌ور فهرستی از پارامترها دارد: مقدارهایی مثل سرعت هوایی، بیشینه‌ی ارتفاع یا ولتاژ باتری. هر پارامتر نام، نوع داده، بازه‌ی مجاز، مقدار پیش‌فرض، واحد، شناسه‌ی لاگ و چند برچسب دسته‌بندی دارد. نوشتن و نگه‌داری دستی این فهرست در کد C کاری کند و پرخطاست.

این ابزار فهرست پارامترها را در یک ویرایشگر جدول‌مانند از شما می‌گیرد، هر اشتباهی را همان لحظه نشان می‌دهد و با یک کلیک کد C آماده‌ی کامپایل می‌سازد. شما فقط فهرست را نگه‌داری می‌کنید و کد هر بار از روی همان فهرست ساخته می‌شود.

ابزار دو بخش دارد: **ویرایشگر**، که مانند یک برنامه‌ی معمولی در پنجره‌ی خودش باز می‌شود و همه‌ی کارها در آن انجام می‌شود؛ و **تولیدکننده‌ی کد**، که همان کار دکمه‌ی **Generate code** را از خط فرمان هم انجام می‌دهد.

۱.۲ روند کار در یک نگاه

گام	کار شما	نتیجه
۱	یک پروژه می‌سازید (فصل ۵).	پوشه‌ای که فایل `pl_project.json` در آن است.
۲	یک یا چند ساب‌لیست به پروژه اضافه می‌کنید (فصل ۱۰).	فایل‌های XML در پوشه‌ی `parameter_lists`.
۳	پارامترها را در ویرایشگر وارد می‌کنید (فصل‌های ۷ و ۸).	فهرستی که ویرایشگر هم‌زمان بررسی‌اش می‌کند.
۴	دکمه‌ی **Generate code** را می‌زنید (فصل ۱۲).	فایل‌های C در پوشه‌ی خروجی پروژه.
۵	فایل‌های تولیدی را به پروژه‌ی فرم‌ور اضافه می‌کنید (فصل ۱۳).	فرم‌ور از پارامترها استفاده می‌کند.

۱.۳ پیش‌نیازها

- ویندوز یا لینوکس.
- پایتون ۳.۱۱ یا جدیدتر. نسخه‌های قدیمی‌تر پذیرفته نمی‌شوند و ابزار همین را با پیامی قرمز اعلام می‌کند.
- یک مرورگر بر پایه‌ی Chromium، مانند Microsoft Edge یا Google Chrome. ویرایشگر در پنجره‌ای مستقل از همین مرورگر باز می‌شود. اگر چنین مرورگری نباشد، مرورگر پیش‌فرض رایانه به کار می‌رود.

به اینترنت نیازی نیست و هیچ بسته‌ی اضافه‌ای نصب نمی‌شود؛ ابزار فقط از کتابخانه‌ی استاندارد پایتون استفاده می‌کند.

> **نکته:** همه‌چیز روی رایانه‌ی خودتان اجرا می‌شود. ویرایشگر فقط از درون همین رایانه (نشانی `127.0.0.1`) در دسترس است و هیچ داده‌ای از رایانه بیرون نمی‌رود.

۱.۴ درباره‌ی این راهنما

فصل‌ها به همان ترتیبی چیده شده‌اند که معمولاً با ابزار کار می‌کنید. اگر عجله دارید، فصل ۴ (شروع سریع) را بخوانید و بقیه را هر وقت لازم شد.

بخوانید	اگر می‌خواهید…
فصل ۳	ابزار را نصب و اجرا کنید
فصل ۴	در چند دقیقه از صفر به کد C برسید
فصل ۵	پروژه بسازید یا باز کنید
فصل ۶	بخش‌های صفحه‌ی ویرایشگر را بشناسید
فصل‌های ۷ و ۸	پارامتر، آرایه و استراکچر وارد کنید
فصل ۹	پارامتری بسازید که فقط چند مقدار نام‌دار بگیرد
فصل ۱۰	پروژه را به چند ساب‌لیست تقسیم کنید
فصل‌های ۱۲ و ۱۳	کد C بسازید و در فرمور به کار ببرید
فصل ۱۴	با قالب قدیمی جدول پارامتر کار کنید
فصل ۱۵	تنظیمات پروژه را تغییر دهید
فصل ۱۶	معنی یک پیام خطا را بفهمید
فصل ۱۷	شماره‌ی نسخه‌ی ابزار را تغییر دهید

قراردادهای نوشتاری

معنی	نمونه
نام دکمه، منو یا کادری که روی صفحه می‌بینید. برچسب‌های ویرایشگر انگلیسی‌اند و در این راهنما هم همان‌طور آمده‌اند.	**Generate code**
نام فایل، پوشه یا دستور، یا متنی که باید عیناً تایپ شود.	`pl_project.json`
اول منو را باز کنید، بعد گزینه‌ی آن را بزنید.	**Project ▾ ← Open project…**

۲. مفاهیم پایه

چند اصطلاح در همه‌ی بخش‌های ابزار تکرار می‌شوند. پیش از شروع، یک بار مرورشان کنید.

۲.۱ پروژه و ساب‌لیست

پروژه یک پوشه است که فایل پروژه در آن قرار دارد؛ فایلی به نام `pl_project.json`، یا هر نام دیگری که به آن بدهید. این فایل می‌گوید پروژه از کدام فهرست‌ها ساخته شده، چه نوع کدی تولید می‌کند و فایل‌ها را کجا می‌نویسد. ویرایشگر مانند یک ویرایشگر متن کار می‌کند: هر بار یک پروژه را باز می‌کند.

ساب‌لیست (Parameter sublist) یک فایل XML است که بخشی از پارامترهای پروژه را نگه می‌دارد. یک پروژه می‌تواند چند ساب‌لیست داشته باشد؛ مثلاً یکی برای هر زیرسیستم. همه‌ی ساب‌لیست‌های یک پروژه، به ترتیبی که تعیین می‌کنید، با هم **لیست پارامتر** کامل پروژه را می‌سازند.

اپ خود ابزار است، یعنی پوشه‌ی `paramgen`. اپ یک بار روی رایانه قرار می‌گیرد و پروژه‌ها در جای دیگری نگه داشته می‌شوند.

۲.۲ پارامتر و مقدار

پارامتر یک مقدار نام‌دار در فرمور است که می‌توان آن را خواند، تنظیم کرد یا در لاگ ثبت کرد. مهم‌ترین مشخصات هر پارامتر این‌هاست:

مشخصه	معنی
Name	نام پارامتر در کد C. باید یک نام معتبر C باشد.
User Name	نامی که کاربر می‌خواند؛ در ایستگاه زمینی و در لاگ.
Type	نقش پارامتر: `MONITORING` (پایش)، `SETTING` (تنظیم) یا `COMMAND` (فرمان).
Data type	نوع C مقدار، مثل `float32_t` یا `uint16_t` (پیوست الف)؛ یا یک اینام (بخش ۲.۹).
Size	تعداد عنصرها. `1` یعنی مقدار تکی و عدد بزرگ‌تر یعنی آرایه.
Unit	واحد، مثل `m/s` یا `V`.
Default value	مقدار اولیه‌ی پارامتر.
Minimum و **Maximum**	بازه‌ی مجاز مقدار.
Resolution	کوچک‌ترین گام تغییر مقدار.
تگ‌ها و **LogID**	برچسب‌های دسته‌بندی و شماره‌ی لاگ (فصل ۱۱).
Description	توضیح آزاد.

مقدار کوچک‌ترین جزء جدول تولیدی است: هر چیزی که در کد C یک ردیف از جدول را می‌گیرد. پارامتر ساده یک مقدار دارد، آرایه‌ی چهارتایی چهار مقدار، و استراکچر به تعداد فیلدهایش.

۲.۳ آرایه

پارامتری که **Size** آن بیشتر از `1` باشد، آرایه است: یک اعلان با چند مقدار. مثلاً `CellVoltage` با اندازه‌ی `4` چهار مقدار دارد، از `CellVoltage[0]` تا `CellVoltage[3]`، و هر عنصر ایندکس و شناسه‌ی لاگ خودش را دارد.

۲.۴ استراکچر و تایپ استراکچر

استراکچر گروهی از فیلدها زیر یک نام است؛ مثل `Position` که فیلدهای `Lat`، `Lon` و `Alt` را دارد. شکل استراکچر را **تایپ استراکچر** (Structure type) تعیین می‌کند؛ همان چیزی که در C با `struct` تعریف می‌شود. چند استراکچر می‌توانند از یک تایپ ساخته شوند و هر تغییری در تایپ به همه‌ی آن‌ها می‌رسد.

استراکچر هم می‌تواند آرایه باشد: `Motor` با اندازه‌ی `4` یعنی چهار موتور که هرکدام فیلدهای `Rpm` و `Temp` را دارند.

۲.۵ تگ‌ها

تگ‌ها برچسب‌هایی‌اند که هر پارامتر را دسته‌بندی می‌کنند: به کدام زیرسیستم تعلق دارد، آیا پس از ریست حفظ می‌شود، آیا لاگ می‌شود و چه سطح دسترسی‌ای دارد. ابزار مقدار تگ‌ها را بررسی می‌کند و در کد تولیدی می‌گذارد؛ اینکه هر تگ در عمل چه اثری داشته باشد، تصمیم خود فرمور است (فصل ۱۱). چهار تگ آزاد `Tag1` تا `Tag4` هم هست که معنی هرکدام را خود پروژه تعیین می‌کند (بخش ۱۱.۶).

۲.۶ شناسه‌ی لاگ

هر مقداری که در لاگ ثبت می‌شود، یک شماره‌ی یکتا لازم دارد که به آن شناسه‌ی لاگ (LogID) می‌گوییم. شماره‌های یک پارامتر پشت سر هم‌اند: آرایه‌ی چهارتایی که از `200` شروع شود، شماره‌های `200` تا `203` را می‌گیرد.

۲.۷ ایندکس

هر مقدار در جدول تولیدی یک شماره‌ی ردیف دارد که در کد به شکل ماکرو می‌آید؛ مثلاً `#define AIRSPEED (0U)`. شمارش از اولین ساب‌لیست شروع می‌شود و در هر ساب‌لیست از بالا به پایین پیش می‌رود. برای همین ترتیب پارامترها و ترتیب ساب‌لیست‌ها اهمیت دارد (بخش ۱۲.۴). در قالب جدول پارامتر، ایندکس در هر ساب‌لیست دوباره از صفر شروع می‌شود (بخش ۱۴.۶).

۲.۸ دو قالب خروجی

هر پروژه هنگام ساخته شدن یکی از دو قالب خروجی را انتخاب می‌کند:

Parameter table	Smart data (پیش‌فرض)	
فایل `nsrParamTables.h`، فایل توضیحی `nsrParamOffsets.h` و فایل نام سیستم‌ها `nsrSystemNames.h`	هفت فایل `.c` و `.h` با پیشوند `cg_`	چه می‌سازد
فرم‌ور قدیمی‌تر که جدولی از خطوط `ADD_PARAM` دارد	فرم‌وری که کتابخانه‌ی smart data را دارد	برای چه فرم‌وری
پشتیبانی نمی‌شود؛ هر ردیف یک مقدار است	پشتیبانی می‌شود	استراکچر و آرایه
لازم نیست	لازم است	کتابخانه‌ی C
فصل ۱۴	فصل‌های ۱۲ و ۱۳	توضیح کامل

فایل‌های ساب‌لیست در هر دو قالب یکسان‌اند؛ تفاوت فقط در کدی است که از رویشان ساخته می‌شود.

۲.۹ اینام

اینام (Enumeration) نوع داده‌ای است که فقط چند مقدار نام‌دار را می‌پذیرد؛ مثلاً `eFlightMode` با مقدارهای `FLIGHT_MODE_MANUAL`، `FLIGHT_MODE_HOLD`، `FLIGHT_MODE_LAND` و… . هر مقدار یک نام و یک عدد دارد. در C، اینام یک `typedef` از یک نوع صحیح است و هر مقدارش یک `#define`؛ پس پارامتری از نوع اینام با همان تابع‌های smart data نوع صحیحش خوانده و نوشته می‌شود، ولی هر نوشتنی که عددش جزو مقدارهای اینام نباشد پذیرفته نمی‌شود. اینام فقط در قالب Smart data هست (فصل ۹).

۳. نصب و اجرا

۳.۱ نصب

ابزار نصب‌کننده ندارد. پوشه‌ی `paramgen` را در هر جای دلخواه رایانه کپی کنید؛ کار نصب تمام است. اگر از قالب Smart data استفاده می‌کنید، دو پوشه‌ی `smart_data` و `parameter_descriptor` را هم که کنار `paramgen` قرار دارند نگه دارید؛ این دو، کتابخانه‌ی C هستند که کد تولیدی به آن‌ها تکیه می‌کند (فصل ۱۳).

محتوای پوشه‌ی paramgen	کاربرد
`run.sh` و `run.bat`	تنها فایلی که اجرا می‌کنید؛ اولی در ویندوز و دومی در لینوکس.
`src` و `bin`	نقطه‌های شروع و کد خود ابزار.
`templates`	الگوهایی که پروژه‌ی تازه از آن‌ها ساخته می‌شود: فایل پروژه، قالب فایل‌های C (`c.json`) و لیست‌های کشویی پیش‌فرض (`choices.json`).
`docs`	همین راهنما، راهنمای توسعه‌دهنده و یک نمونه‌ی کد C.
`tests`	آزمون‌های خود ابزار.
`var`	چیزهایی که خود برنامه می‌نویسد: فهرست پروژه‌های اخیر (`settings.json`) و لیست‌های کشویی این رایانه (`choices.json`).

اگر پایتون روی رایانه نیست، نسخه‌ی ۳.۱۱ یا جدیدتر را از python.org نصب کنید و در نصب‌کننده گزینه‌ی **Add python.exe to PATH** را تیک بزنید. `run.bat` هر پایتونی را که روی رایانه پیدا کند امتحان می‌کند و اولین را که واقعاً اجرا شود و ۳.۱۱ یا جدیدتر باشد به کار می‌برد؛ پس اگر یک میان‌بر خراب `python` هم روی رایانه باشد، مشکلی پیش نمی‌آید.

۳.۲ اجرای برنامه

در ویندوز روی `run.bat` دوبار کلیک کنید. در لینوکس، در یک ترمینال به پوشه‌ی `paramgen` بروید و دستور `./run.sh` را اجرا کنید. منوی زیر نمایش داده می‌شود:

```
================================================================================
  Parameter list code generator 01.04.01
================================================================================
  project: Flight Computer  (D:\Projects\Flight_Computer)

   1  Editor          open a project and edit its parameter lists
   2  Generate code   write the C files of the project opened last
   3  Tests           run every self test of the tool
   4  Check this PC   is the Python on this machine complete enough?

   0  Quit

  Select an option [1]:
```

شماره‌ی کار را بنویسید و Enter بزنید. اگر فقط Enter بزنید، ویرایشگر باز می‌شود. سطر `project` پروژه‌ای را نشان می‌دهد که آخرین بار در ویرایشگر باز بوده است؛ گزینه‌ی 2 روی همین پروژه کار می‌کند.

کار	گزینه
ویرایشگر را باز می‌کند (فصل ۶).	`1 Editor`
بدون باز کردن ویرایشگر، کد پروژه‌ای را می‌سازد که آخرین بار باز بوده است.	`2 Generate code`
آزمون‌های خود ابزار را اجرا می‌کند. معمولاً فقط پس از تغییر دادن خود ابزار لازم است (فصل ۱۷).	`3 Tests`
بررسی می‌کند که پایتون این رایانه برای اجرای ابزار کافی است (بخش ۳.۳).	`4 Check this PC`
از منو خارج می‌شود.	`0 Quit`

پس از پایان هر کار، پیامی می‌گوید کار چطور تمام شد و با زدن Enter به منو برمی‌گردید.

۳.۳ بررسی رایانه

بار اول، پیش از هر کار دیگر، گزینه‌ی 4 (Check this PC) را اجرا کنید. این بررسی سه چیز را می‌سنجد:

- نسخه‌ی پایتون، که باید ۳.۱۱ یا جدیدتر باشد. اگر قدیمی‌تر باشد، پیامی قرمز همین را می‌گوید و بررسی همان‌جا متوقف می‌شود.
- ماژول‌هایی از کتابخانه‌ی استاندارد پایتون که ابزار به آن‌ها نیاز دارد.
- کامل بودن پوشه‌ی ابزار.

اگر همه‌چیز درست باشد، پیام **Everything is ready** نمایش داده می‌شود.

> **نکته:** `run.bat` و `run.sh` هم پیش از نمایش منو نسخه‌ی پایتون را بررسی می‌کنند. اگر پایتون پیدا نشود یا قدیمی‌تر از ۳.۱۱ باشد، همان ابتدا پیام می‌دهند.

۳.۴ اجرای مستقیم یک کار

می‌توانید منو را کنار بگذارید و نام کار را مستقیم بعد از `run.bat` بنویسید (در لینوکس `./run.sh`). هر چیزی که بعد از نام کار بیاید، به همان کار داده می‌شود:

کار	دستور
ویرایشگر را با آخرین پروژه باز می‌کند.	`run.bat editor`
ویرایشگر را با پروژه‌ی این پوشه باز می‌کند.	`run.bat editor D:\Projects\Flight_Computer`
ویرایشگر را روی درگاه (پورت) دیگری اجرا می‌کند. درگاه پیش‌فرض `8765` است.	`run.bat editor -p 8800`
ویرایشگر را به‌جای پنجره‌ی مستقل، در یک زبانه‌ی مرورگر پیش‌فرض باز می‌کند.	`run.bat editor --tab`
پنجره‌ای باز نمی‌کند و فقط نشانی ویرایشگر را چاپ می‌کند.	`run.bat editor --no-browser`
کد پروژه را می‌سازد (بخش ۱۲.۵).	`run.bat generate`
آزمون‌های ابزار را اجرا می‌کند.	`run.bat tests`
رایانه را بررسی می‌کند (همان گزینه‌ی `4`).	`run.bat setup`

۳.۵ بستن برنامه

هنگام کار با ویرایشگر دو پنجره باز است: خود ویرایشگر، و یک پنجره‌ی کنسول (پنجره‌ی متنی سیاه) که سرور کوچک ابزار در آن اجرا می‌شود. تا وقتی کار می‌کنید، پنجره‌ی کنسول را نبندید.

برای بستن برنامه، ابتدا تغییرهای ذخیره‌نشده را ذخیره کنید، پنجره‌ی ویرایشگر را ببندید و سپس در پنجره‌ی کنسول `Ctrl+C` را بزنید. اگر ویرایشگر را از منو باز کرده باشید، به منو برمی‌گردید.

> **هشدار:** بستن پنجره‌ی ویرایشگر درباره‌ی تغییرهای ذخیره‌نشده چیزی نمی‌پرسد و آن تغییرها از دست می‌روند. علامت `*` کنار نام فایل ساب‌لیست همیشه نشان می‌دهد که تغییری ذخیره نشده است.

۳.۶ اگر ویرایشگر باز نشد

مشکل	راه حل
پیام **Python was not found**	هیچ پایتون سالمی پیدا نشد. پایتون ۳.۱۱ یا جدیدتر را نصب کنید و گزینه‌ی **Add python.exe to PATH** را بزنید.
پیام **The editor could not start**	احتمالاً برنامه‌ی دیگری درگاه `8765` را گرفته است. با درگاه دیگری اجرا کنید: `run.bat editor -p 8800`
ویرایشگر تازه پروژه‌ی قبلی را نشان می‌دهد	ویرایشگر قبلی هنوز روشن است؛ بستن پنجره‌ی ویرایشگر، پنجره‌ی سیاه (کنسول) آن را نمی‌بندد. آن کنسول را ببندید و دوباره اجرا کنید.
پنجره‌ی ویرایشگر باز نشد	اگر ابزار Edge یا Chrome را پیدا نکند، مرورگر پیش‌فرض را امتحان می‌کند. اگر آن هم باز نشد، نشانی چاپ‌شده در کنسول (مثلاً `http://127.0.0.1:8765/`) را در هر مرورگری باز کنید.
مرورگر در مسیر غیرمعمولی نصب است	پیش از اجرا، مسیر آن را تعیین کنید: `set PL_CODEGEN_BROWSER=D:\Portable\msedge.exe`
پنجره‌های Open و Browse ویندوز باز نمی‌شوند	ویرایشگر برای این پنجره‌ها از بخش tkinter پایتون استفاده می‌کند. اگر این بخش نصب نباشد، ویرایشگر به‌جای آن پنجره‌ی انتخاب داخلی خودش را نشان می‌دهد و کار ادامه پیدا می‌کند.

۴. شروع سریع: از صفر تا کد C

این فصل یک پروژه‌ی کوچک را از صفر تا کد C می‌سازد تا روند کار را ببینید. جزئیات هر گام در فصل‌های بعد آمده است. همه‌ی مثال‌های این راهنما از همین پروژه‌ی نمونه، Flight Computer، گرفته شده‌اند.

گام ۱ — پروژه را بسازید

۱. ویرایشگر را باز کنید: `run.bat` را اجرا کنید و گزینه‌ی `1` را بزنید.

۲. در صفحه‌ی شروع، **New project...** را بزنید.

۳. در **Name** نام پروژه را بنویسید؛ مثلاً `Flight Computer`.

۴. در **What it generates** گزینه‌ی **Smart data** را نگه دارید.

۵. در **Location** پوشه‌ای را بنویسید که پروژه داخل آن ساخته شود، یا آن را با **Browse...** انتخاب کنید.

۶. **OK** را بزنید. پوشه‌ی پروژه با نام آن ساخته و پروژه باز می‌شود.

گام ۲ — یک ساب‌لیست بسازید

پارامترها داخل ساب‌لیست نگه داشته می‌شوند و پروژه‌ی تازه هنوز ساب‌لیستی ندارد. دکمه‌ی ☰ را در نوار بالا بزنید و سپس **+ Add parameter sublist** را. نامی مثل `core` بنویسید و **OK** را بزنید. ساب‌لیست ساخته و باز می‌شود.

گام ۳ — یک پارامتر اضافه کنید

▾ **+ Add variable** ← **Primitive** را بزنید (یا `Ctrl+N`). یک پارامتر تازه به انتهای فهرست اضافه و انتخاب می‌شود. فرم سمت راست را مانند جدول زیر پر کنید:

فیلد	مقدار
Name	`Airspeed`
User Name	`Airspeed`
Type	`MONITORING`
Data type	`float32_t`
Unit	`m/s`
Default value	`0`
Minimum	`0`
Maximum	`120`
Resolution	`0.01`
Tag System Name	`SYS_NAV`

فیلد	مقدار
Tag Loggable	`TRUE`
LogID	دکمه‌ی **Next free** را بزنید (مثلاً `100`).
Description	`Indicated airspeed.`

Tag ParamMode در پروژه‌ی Smart data همیشه `PARAM_MODE_NONE` است و تغییر نمی‌کند (بخش ۱۱.۱).

هر تغییر همان لحظه اعمال و بررسی می‌شود و دکمه‌ی تأییدی وجود ندارد. اگر چیزی نادرست باشد، در پنل **Problems** پایین صفحه نمایش داده می‌شود.

گام ۴ — یک آرایه و یک استراکچر بسازید

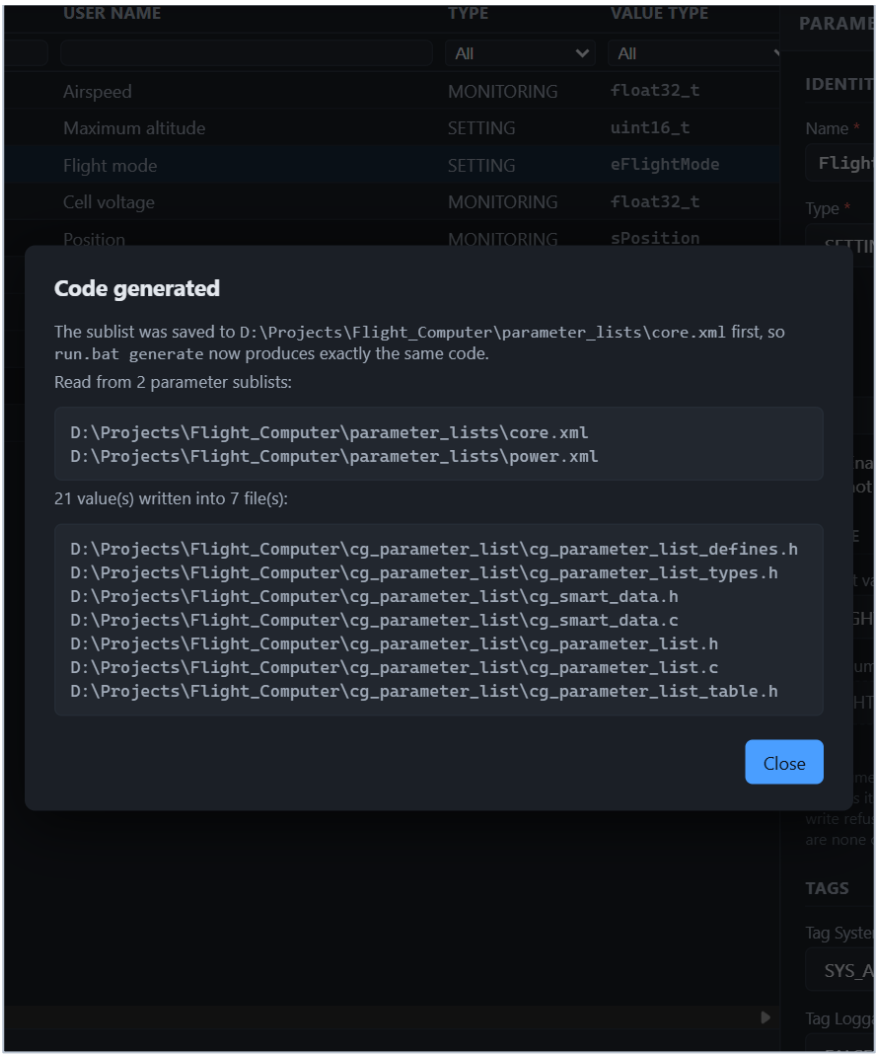
- **آرایه:** پارامتر دیگری بسازید و **Size** آن را `4` بگذارید. این پارامتر در جدول به چهار عنصر تقسیم می‌شود: `CellVoltage[0]` تا `CellVoltage[3]`.
- **استراکچر:** **+ Add variable** ▾ ← **User defined…** را بزنید، نام `Position` را بنویسید و یک تایپ تازه به نام `sPosition` با فیلدهای `Lat`، `Lon` و `Alt` تعریف کنید.

گام ۵ — ذخیره و بررسی کنید

با `Ctrl+S` ساب‌لیست را ذخیره کنید. سپس پنل **Problems** را نگاه کنید: تا وقتی خطای قرمزی در آن هست، کدی ساخته نمی‌شود. با کلیک روی هر خطا به پارامتر مربوط به آن می‌روید.

گام ۶ — کد را بسازید

Generate code را بزنید (یا `Ctrl+G`). فایل‌ها در پوشه‌ی `cg_parameter_list` داخل پروژه نوشته می‌شوند و پنجره‌ای فهرست آن‌ها را نشان می‌دهد.



شکل ۱ — پنجره‌ی نتیجه‌ی تولید کد: ساب‌لیست‌هایی که خوانده شدند و فایل‌هایی که نوشته شدند

گام ۷ — کد را در فرمور به کار ببرید

فایل‌های تولیدی به یک کتابخانه‌ی کوچک C تکیه می‌کنند: پوشه‌های `smart_data` و `parameter_descriptor` که کنار `paramgen` قرار دارند. این دو پوشه و فایل‌های تولیدی را به پروژه‌ی فرمور اضافه کنید و در شروع برنامه تابع `fParameterList_Init()` را صدا بزنید. جزئیات در فصل ۱۳ آمده است.

> **مهم:** پارامترهای تازه را همیشه به **انتهای** فهرست اضافه کنید، نه وسط آن. ترتیب پارامترها ایندکس و جای ذخیره‌ی هر مقدار را تعیین می‌کند و جابه‌جا شدن آن‌ها روی بردهایی که از قبل داده ذخیره کرده‌اند اثر می‌گذارد (بخش ۱۲.۴).

۵. پروژه‌ها

۵.۱ ساختار پوشه‌ی پروژه

پروژه‌ی Flight Computer پس از اولین تولید کد این ساختار را دارد:

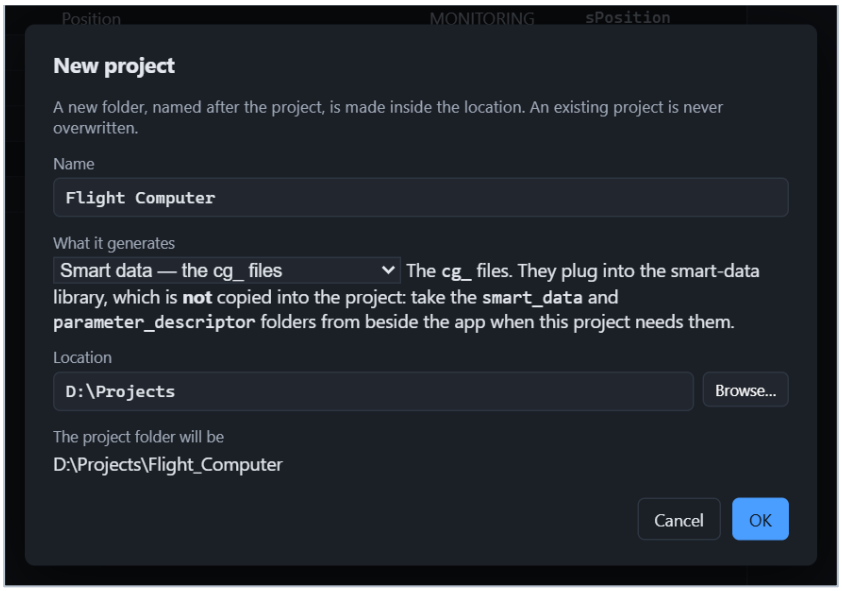
```
Flight_Computer/
    pl_project.json
    parameter_lists/
        core.xml
        core_types.xml
        power.xml
    cg_parameter_list/
        cg_parameter_list.c
        cg_parameter_list.h
        ...
        parameter_list.xml
        parameter_list_types.xml
```

مسیر	چیست
`pl_project.json`	فایل پروژه. همه‌ی تنظیمات پروژه در آن است (فصل ۱۵).
`parameter_lists`	ساب‌لیست‌ها. کنار هر ساب‌لیستی که استراکچر دارد، فایلی با پسوند `_types.xml` هست که تایپ‌های استراکچر همان ساب‌لیست را نگه می‌دارد. این دو فایل همیشه با هم جابه‌جا می‌شوند.
`cg_parameter_list`	کد C تولیدی. جای این پوشه را می‌توان در فایل پروژه تغییر داد.
`parameter_list.xml`	نسخه‌ی ادغام‌شده‌ی آخرین تولید موفق. تولید بعدی خودش را با آن مقایسه می‌کند تا اگر چیدمان حافظه تغییر کرده باشد هشدار بدهد (بخش ۱۲.۴). این فایل را دستی ویرایش نکنید.

هنگام ساختن پروژه فقط پوشه‌ی پروژه و فایل `pl_project.json` ساخته می‌شوند. بقیه بعداً، با ساختن اولین ساب‌لیست و اولین تولید کد، پیدا می‌شوند.

۵.۲ ساختن پروژه‌ی جدید

در صفحه‌ی شروع **New project…** را بزنید، یا همین گزینه را از منوی **Project** ▾ انتخاب کنید.



شکل ۲ — پنجره‌ی ساختن پروژه‌ی جدید

بخش	کار
Name	نام پروژه. در نوار بالای ویرایشگر و در فهرست پروژه‌های اخیر نمایش داده می‌شود.
What it generates	قالب خروجی: **Smart data** یا **Parameter table** (بخش ۲.۸). متن کنار کادر می‌گوید هرکدام چه می‌سازد. این انتخاب را بعداً فقط از راه فایل پروژه می‌توان تغییر داد.
Location	پوشه‌ای که پوشه‌ی پروژه داخل آن ساخته می‌شود. به‌طور پیش‌فرض همان پوشه‌ای است که `paramgen` در آن قرار دارد. می‌توانید مسیر را بنویسید یا با **Browse...** پنجره‌ی انتخاب پوشه‌ی ویندوز را باز کنید. پوشه‌ای که هنوز وجود ندارد هم پذیرفته می‌شود و همراه پروژه ساخته می‌شود.
The project folder will be	مسیر نهایی پوشه‌ی پروژه. فاصله‌های نام به `_` تبدیل می‌شوند؛ مثلاً `Flight Computer` به `Flight_Computer`.

> **نکته:** ساختن پروژه هرگز پروژه‌ی موجود را بازنویسی نمی‌کند. اگر در آن پوشه از قبل فایل پروژه‌ای باشد (با هر نامی)، پروژه ساخته نمی‌شود و پیامی همین را می‌گوید. اگر ساب‌لیست باز تغییر ذخیره‌نشده داشته باشد، پیش از ساختن پروژه‌ی تازه از شما پرسیده می‌شود.

۵.۳ باز کردن پروژه

از منوی **Project** ▾ گزینه‌ی **Open project...** را بزنید، یا دکمه‌ی همین نام را در صفحه‌ی شروع. پنجره‌ی Open ویندوز باز می‌شود و همه‌ی فایل‌های JSON را نشان می‌دهد. فایل پروژه‌ی موردنظر (`pl_project.json`، یا هر نامی که به آن داده‌اید) را انتخاب کنید و **Open** را بزنید. فایل JSONای که پروژه نیست باز نمی‌شود و پیامی همین را می‌گوید.

نام فایل پروژه مهم نیست: می‌توانید `pl_project.json` را تغییر نام بدهید و پروژه همچنان باز می‌شود. اگر به‌جای فایل، پوشه‌ی پروژه را بدهید (مثلاً به `run.bat`)، فایل پروژه‌ی درون آن پیدا می‌شود؛ اگر پوشه دو فایل پروژه داشته باشد، پیام نام هر دو را می‌گوید تا یکی را انتخاب کنید.

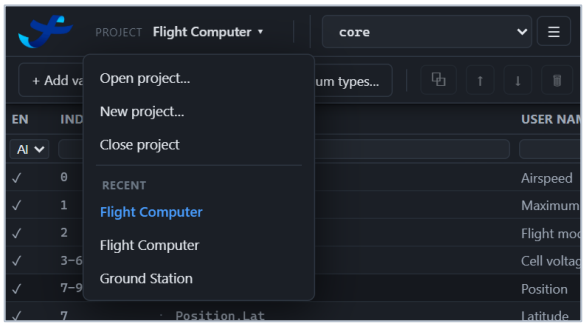
این پنجره از پوشه‌ی پروژه‌ی باز شروع می‌کند. اگر پروژه‌ای باز نباشد، از پوشه‌ی آخرین پروژه، و اگر آن هم نباشد، از پوشه‌ای که `paramgen` در آن است.

تا وقتی پنجره‌ی ویندوز باز است، ویرایشگر پیامی نشان می‌دهد و کلیکی نمی‌پذیرد. اگر پنجره‌ی ویندوز پشت ویرایشگر مانده باشد، آن را از نوار وظیفه‌ی ویندوز پیدا کنید.

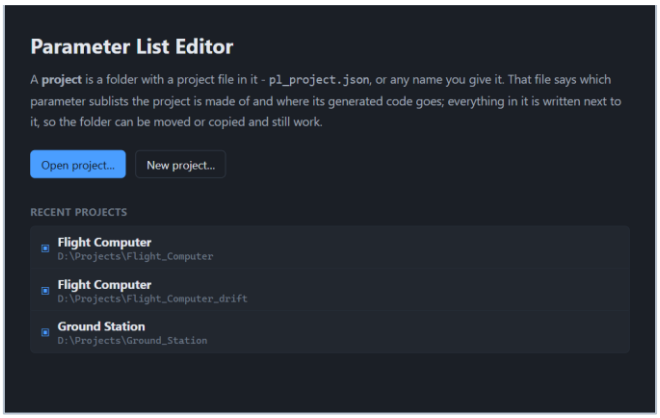
پروژه‌هایی که با نسخه‌های قدیمی‌تر ساخته شده‌اند و فایلشان هنوز `project.json` نام دارد هم بدون هیچ تغییری باز می‌شوند. اگر فایل انتخاب‌شده فایل پروژه نباشد، باز نمی‌شود و دلیلش گفته می‌شود.

۵.۴ پروژه‌های اخیر و بستن پروژه

پروژه‌هایی که اخیراً باز کرده‌اید در پایین منوی **Project** ▾ و در صفحه‌ی شروع فهرست شده‌اند و با یک کلیک باز می‌شوند. **Close project** پروژه را می‌بندد و به صفحه‌ی شروع برمی‌گردد.



شکل ۳ — منوی Project و فهرست پروژه‌های اخیر



شکل ۴ — صفحه‌ی شروع، وقتی هیچ پروژه‌ای باز نیست

فهرست پروژه‌های اخیر در `paramgen/var/settings.json` نگه داشته می‌شود و پاک کردن آن خطری ندارد.

۵.۵ جابه‌جا کردن، کپی و اشتراک

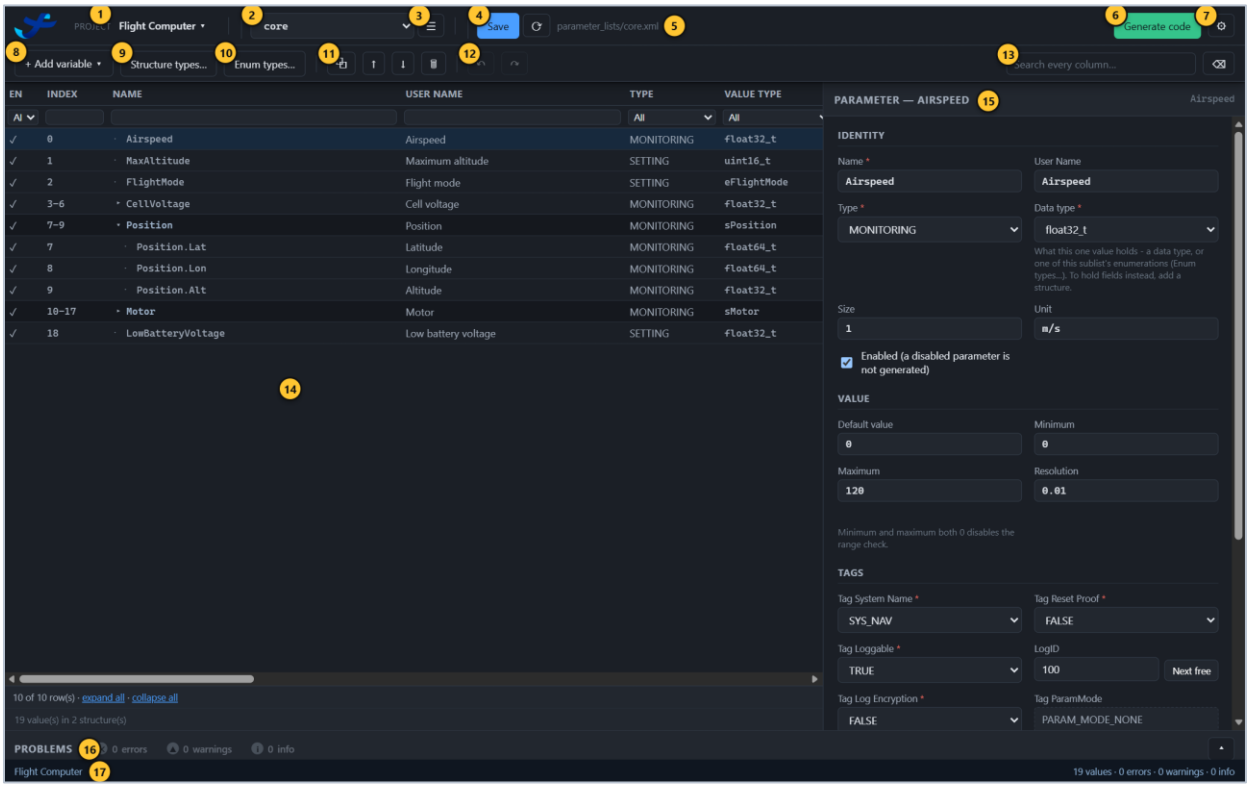
همه‌ی مسیرهای داخل فایل پروژه نسبت به خود همان فایل نوشته می‌شوند. بنابراین می‌توانید پوشه‌ی پروژه را جابه‌جا کنید، روی رایانه‌ی دیگری کپی کنید یا برای همکارتان بفرستید و همچنان کار می‌کند. سرآیند فایل‌های تولیدی هم مسیر ساب‌لیست‌ها را نسبت به پوشه‌ی پروژه می‌نویسد (مثلاً `parameter_lists/core.xml`)، نه مسیر کامل یک رایانه؛ پس تولید کد روی رایانه‌ی دیگر فایل‌ها را بی‌دلیل تغییر نمی‌دهد.

> **پیشنهاد:** برای کار گروهی، پوشه‌ی پروژه را در گیت نگه دارید. دو نفر نباید هم‌زمان یک پروژه را در دو ویرایشگر ویرایش کنند؛ هر ویرایشگر نسخه‌ی خودش را در حافظه دارد.

۶. آشنایی با محیط ویرایشگر

۶.۱ نمای کلی

صفحه‌ی اصلی ویرایشگر چهار بخش دارد: نوار ابزار در بالا، جدول پارامترها در سمت چپ، فرم ویرایش در سمت راست و پنل **Problems** در پایین. شماره‌های شکل زیر در جدول بعد از آن توضیح داده شده‌اند.



شکل ۵ — بخش‌های پنجره‌ی اصلی ویرایشگر

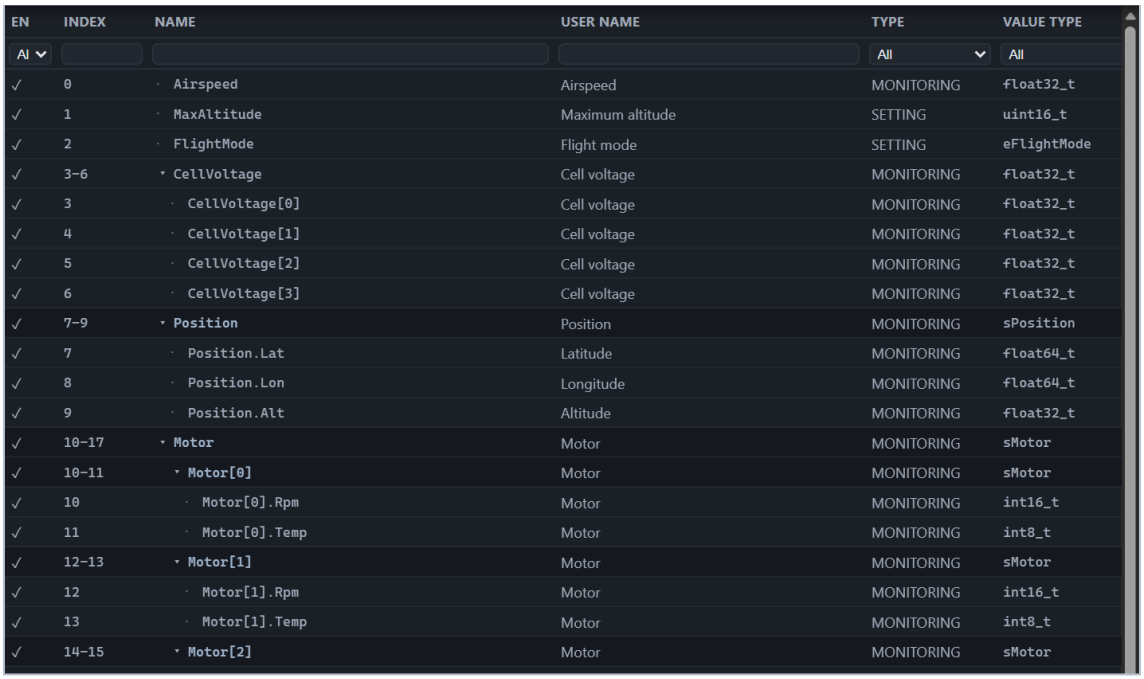
گوشه‌ی بالای ویرایشگر لوگوی شرکت است. عنوان پنجره (در نوار عنوان و نوار وظیفه‌ی ویندوز) مسیر کامل فایل پروژه‌ی باز را نشان می‌دهد، مثل `D:\Projects\Flight_Computer\pl_project.json — Parameter List Editor`؛ علامت * در ابتدای آن یعنی ساب‌لیست باز تغییر ذخیره‌نشده دارد.

#	بخش	کار
۱	**Project** ▾	باز کردن، ساختن و بستن پروژه، و فهرست پروژه‌های اخیر. نام پروژه‌ی باز روی همین دکمه نوشته شده است.
۲	انتخاب ساب‌لیست	ساب‌لیستی که ویرایش می‌کنید. گزینه‌ی **every sublist, merged** — همه‌ی ساب‌لیست‌ها را با هم نشان می‌دهد. این کادر فقط وقتی دیده می‌شود که پروژه دست‌کم دو ساب‌لیست داشته باشد.
۳	☰	پنجره‌ی ساب‌لیست‌ها: کدام ساب‌لیست‌ها در ساخت کد شرکت کنند، به چه ترتیبی، و ساختن یا افزودن ساب‌لیست (فصل ۱۰).

#	بخش	کار
۴	**Save** و ⟳	ذخیره‌ی ساب‌لیست باز؛ و خواندن دوباره‌ی ساب‌لیست و فایل پروژه از دیسک.
۵	نام فایل	مسیر فایل ساب‌لیست باز، نسبت به پوشه‌ی پروژه. علامت * پیش از آن یعنی تغییری ذخیره نشده است.
۶	**Generate code**	تولید کد C (فصل ۱۲).
۷	⚙	تنظیمات پروژه (بخش ۱۵.۸)، لیست‌های کشویی (بخش ۱۱.۵) و تغییر تم تیره و روشن.
۸	**+ Add variable** ▾	افزودن پارامتر ساده (**Primitive**) یا استراکچر (**User defined…**).
۹	**Structure types…**	تعریف، ویرایش و حذف تایپ‌های استراکچر (فصل ۸).
۱۰	**Enum types…**	تعریف، ویرایش و حذف اینام‌های ساب‌لیست (فصل ۹).
۱۱	⧉ ↑ ↓ 🗑	تکثیر، جابه‌جایی به بالا و پایین، و حذف پارامتر انتخاب‌شده.
۱۲	↶ و ↷	واگرد (Undo) و ازنو (Redo).
۱۳	جست‌وجو	کادر **Search every column…** در همه‌ی ستون‌ها جست‌وجو می‌کند و دکمه‌ی ⌫ همه‌ی فیلترها را پاک می‌کند.
۱۴	جدول پارامترها	یک ردیف برای هر مقدار (بخش ۶.۲).
۱۵	فرم ویرایش	همه‌ی فیلدهای ردیف انتخاب‌شده (بخش ۶.۳).
۱۶	**Problems**	خطاها و هشدارها (فصل ۱۶).
۱۷	نوار وضعیت	آخرین کار انجام‌شده و شمار مقدارها، خطاها و هشدارها.

۶.۲ جدول پارامترها

جدول همه‌ی پارامترهای ساب‌لیست باز را نشان می‌دهد؛ برای هر مقدار یک ردیف، به همان ترتیبی که در کد C می‌آیند. جدول برای دیدن و پیدا کردن است و ویرایش در فرم سمت راست انجام می‌شود. روی هر ردیف کلیک کنید تا فرم آن باز شود.



EN	INDEX	NAME	USER NAME	TYPE	VALUE TYPE
All				All	All
✓	0	Airspeed	Airspeed	MONITORING	float32_t
✓	1	MaxAltitude	Maximum altitude	SETTING	uint16_t
✓	2	FlightMode	Flight mode	SETTING	eFlightMode
✓	3-6	▾ CellVoltage	Cell voltage	MONITORING	float32_t
✓	3	CellVoltage[0]	Cell voltage	MONITORING	float32_t
✓	4	CellVoltage[1]	Cell voltage	MONITORING	float32_t
✓	5	CellVoltage[2]	Cell voltage	MONITORING	float32_t
✓	6	CellVoltage[3]	Cell voltage	MONITORING	float32_t
✓	7-9	▾ Position	Position	MONITORING	sPosition
✓	7	Position.Lat	Latitude	MONITORING	float64_t
✓	8	Position.Lon	Longitude	MONITORING	float64_t
✓	9	Position.Alt	Altitude	MONITORING	float32_t
✓	10-17	▾ Motor	Motor	MONITORING	sMotor
✓	10-11	▾ Motor[0]	Motor	MONITORING	sMotor
✓	10	Motor[0].Rpm	Motor	MONITORING	int16_t
✓	11	Motor[0].Temp	Motor	MONITORING	int8_t
✓	12-13	▾ Motor[1]	Motor	MONITORING	sMotor
✓	12	Motor[1].Rpm	Motor	MONITORING	int16_t
✓	13	Motor[1].Temp	Motor	MONITORING	int8_t
✓	14-15	▾ Motor[2]	Motor	MONITORING	sMotor

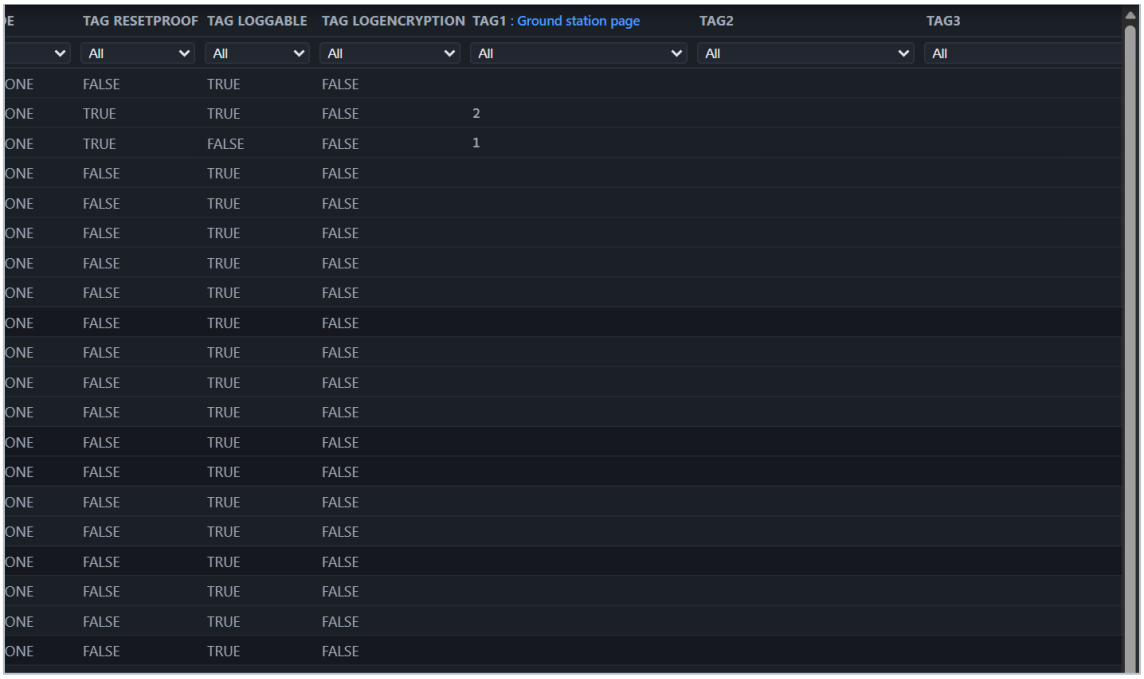
شکل ۶ — جدول پارامترها با همه‌ی سطح‌های باز

ستون	معنی
En	آیا این مقدار در کد ساخته می‌شود: ✓ بله، ✗ خاموش است، و ○ یعنی خودش روشن است ولی استراکچری که در آن قرار دارد خاموش است.
Index	ایندکس مقدار در جدول تولیدی. ردیفی که چند مقدار را در بر دارد، بازه نشان می‌دهد؛ مثل `3-6`. پارامتر خاموش ایندکس ندارد.
Name	نام. با ▸ و ▾ کنار آن، سطح زیرین (عنصرهای آرایه یا فیلدهای استراکچر) باز و بسته می‌شود.
Sublist	فقط در نمای ادغام‌شده: هر ردیف از کدام ساب‌لیست آمده است.
User Name و **Type**	نام نمایشی و نقش پارامتر.
Value type	نوع داده؛ برای استراکچر نام تایپ استراکچر، و برای اینام نام اینام.
Size و **Unit**	تعداد عنصرهای آرایه، و واحد.
Default، **Min**، **Max**، **Res**	مقدار پیش‌فرض، کمینه، بیشینه و رزولوشن. برای اینام، **Default** نام مقدار پیش‌فرض است؛ مثل `FLIGHT_MODE_MANUAL`.
LogID	شناسه‌ی لاگ؛ برای چند مقدار به شکل بازه، مثل `200-203`.
ستون‌های **Tag …**	پنج تگ اصلی: SystemName، ParamMode، ResetProof، Loggable و LogEncryption.
Tag1 تا **Tag4**	تگ‌های آزاد. اگر پروژه برای تگی توضیح نوشته باشد، عنوان ستونش آن را هم دارد؛ مثل `TAG1 : Ground station page` (بخش ۱۱.۶).
Address	فقط در پروژه‌ی جدول پارامتر (بخش ۱۴.۴).
Description	توضیح هر ردیف؛ برای هر مقدار همان متنی است که در کد تولیدی می‌آید.



VALUE TYPE	SIZE	UNIT	DEFAULT	MIN	MAX	RES	LOGID	TAG SYSTEMNAME	TAG PAI
All	All	All						All	All
float32_t		m/s	0	0	120	0.01	100	SYS_NAV	PARAM_
uint16_t		m	3000	0	6000	1	101	SYS_AC_AUTOPILOT	PARAM_
eFlightMode			FLIGHT_MODE_MANUAL	0	5	1		SYS_AC_AUTOPILOT	PARAM_
float32_t	4	V	0	0	5	0.01	200-203	SYS_AC_POWER	PARAM_
float32_t		V	0	0	5	0.01	200	SYS_AC_POWER	PARAM_
float32_t		V	0	0	5	0.01	201	SYS_AC_POWER	PARAM_
float32_t		V	0	0	5	0.01	202	SYS_AC_POWER	PARAM_
float32_t		V	0	0	5	0.01	203	SYS_AC_POWER	PARAM_
sPosition							300-302	SYS_NAV	PARAM_
float64_t		deg	0	-90	90	1e-7	300	SYS_NAV	PARAM_
float64_t		deg	0	-180	180	1e-7	301	SYS_NAV	PARAM_
float32_t		m	0	-500	9000	0.1	302	SYS_NAV	PARAM_
sMotor	4						400-407	SYS_AC_POWER	PARAM_
sMotor							400-401	SYS_AC_POWER	PARAM_
int16_t		rpm	0	0	12000	1	400	SYS_AC_POWER	PARAM_
int8_t		C	0	-40	125	1	401	SYS_AC_POWER	PARAM_
sMotor							402-403	SYS_AC_POWER	PARAM_
int16_t		rpm	0	0	12000	1	402	SYS_AC_POWER	PARAM_
int8_t		C	0	-40	125	1	403	SYS_AC_POWER	PARAM_
sMotor							404-405	SYS_AC_POWER	PARAM_

شکل ۷ — همان جدول، اسکرول‌شده به راست: بازه، شناسه‌ی لاگ و تگ‌ها



E	TAG RESETPROOF	TAG LOGGABLE	TAG LOGENCRYPTION	TAG1 : Ground station page	TAG2	TAG3
	All	All	All	All	All	All
ONE	FALSE	TRUE	FALSE			
ONE	TRUE	TRUE	FALSE	2		
ONE	TRUE	FALSE	FALSE	1		
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			
ONE	FALSE	TRUE	FALSE			

شکل ۸ — انتهای راست جدول: تگ‌های آزاد، با توضیحی که پروژه برای Tag1 نوشته است

باز و بسته کردن ردیف‌ها

استراکچرها در ابتدا بازند و آرایه‌ها بسته، تا آرایه‌های بزرگ بقیه‌ی ردیف‌ها را پنهان نکنند. پیوندهای **expand all** و **collapse all** زیر جدول همه‌ی سطح‌ها را یک‌جا باز یا بسته می‌کنند. وقتی همه‌چیز باز باشد، ایندکس‌ها از بالا به پایین پشت سر هم می‌آیند و جدول دقیقاً همان چیزی است که در کد C ساخته می‌شود.

فیلتر و جست‌وجو

ردیف زیر عنوان ستون‌ها برای فیلتر است، مانند Excel. ستون‌هایی که مقدارهای محدودی دارند (مثل **Type**، **Value type**، **Unit** و تگ‌ها) فهرست کشویی دارند و بقیه کادر متنی. کادر **Search every column…** در نوار ابزار در همه‌ی ستون‌ها با هم جست‌وجو می‌کند و دکمه‌ی ⌫ همه‌ی فیلترها را پاک می‌کند. تا وقتی فیلتری فعال است، همه‌ی سطح‌ها جست‌وجو می‌شوند، حتی آن‌هایی که بسته‌اند.

رنگ‌ها

ردیف خاموش کم‌رنگ است و ردیفی که خطا دارد با نواری قرمز در کنارش مشخص می‌شود. ردیف استراکچرها رنگ زمینه‌ی متفاوتی دارد، چون خود استراکچر ردیفی در جدول C ندارد. زیر جدول تعداد ردیف‌های نمایش‌داده‌شده و تعداد مقدارها و استراکچرهای ساب‌لیست نوشته می‌شود.

۶.۳ فرم ویرایش

با کلیک روی هر ردیف، فرم سمت راست پر می‌شود. فرم یک پارامتر ساده چهار بخش دارد: **Identity**، **Value**، **Tags** (شناسه‌ی لاگ هم در همین بخش است) و **Description**. ستاره‌ی قرمز کنار نام فیلد یعنی پر کردن آن الزامی است.

هر تغییر همان لحظه روی ساب‌لیست اعمال و کل پروژه دوباره بررسی می‌شود؛ دکمه‌ی تأییدی وجود ندارد. کادرهای متنی وقتی از آن‌ها خارج شوید یا Enter بزنید ثبت می‌شوند، و فهرست‌های کشویی و تیک‌ها بلافاصله.

فرم با نوع ردیف تغییر می‌کند. برای `bool_t` به‌جای بازه فقط مقدار پیش‌فرض `false` یا `true` پرسیده می‌شود؛ فرم استراکچر بخش **Value** ندارد؛ و فرم فیلدِ یک استراکچر فقط چیزهایی را قابل ویرایش نشان می‌دهد که واقعاً مال همان فیلد است (بخش ۸.۴). عنوان بالای فرم می‌گوید چه چیزی را ویرایش می‌کنید: **Parameter**، **Structure**، **Array** یا **Element**. برای پارامتری از نوع اینام، **Default value** فهرستی کشویی از مقدارهای همان اینام است و بازه فقط‌خواندنی (فصل ۹).

۶.۴ پنل Problems و نوار وضعیت

پنل **Problems** پایین صفحه، هر بار که چیزی تغییر کند، نتیجه‌ی بررسی کل پروژه را نشان می‌دهد. کلیک روی هر خط، پارامتر مربوط را در جدول انتخاب می‌کند. اگر مشکلی نباشد، پنل خودبه‌خود جمع می‌شود. شرح کامل آن در فصل ۱۶ آمده است.

نوار وضعیت پایین‌ترین خط صفحه است: سمت چپ آن آخرین کار انجام‌شده را نشان می‌دهد (مثل Saved یا Edited) و سمت راست آن شمار مقدارها، خطاها، هشدارها و پیام‌های اطلاعاتی را.

۶.۵ تم تیره و روشن

ویرایشگر با تم تیره باز می‌شود. برای رفتن به تم روشن، ⚙ ← **Switch to the light theme** را بزنید. تم انتخابی تا بستن ویرایشگر حفظ می‌شود و اجرای بعدی دوباره با تم تیره شروع می‌شود.

۷. کار با پارامترها

۷.۱ افزودن پارامتر

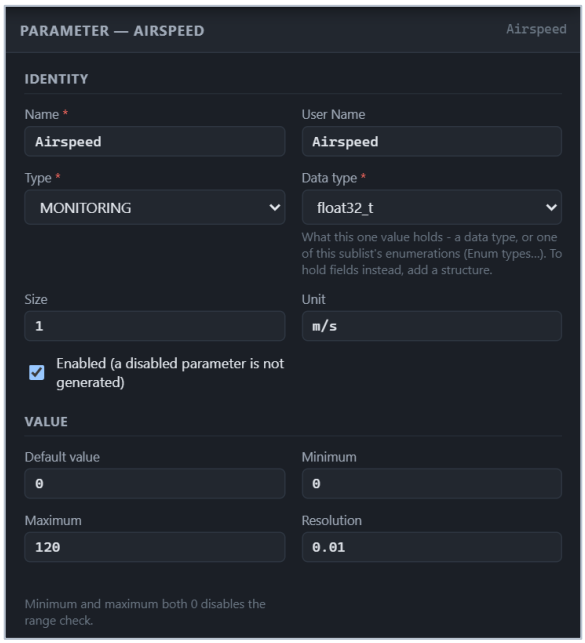
▾ Add variable + ← **Primitive** (یا `Ctrl+N`) یک پارامتر ساده به انتهای ساب‌لیست اضافه می‌کند، آن را انتخاب می‌کند و جدول را تا آن پایین می‌برد. پارامتر تازه با این مقدارها شروع می‌شود:

فیلد	مقدار اولیه
Name	`NewParameter`. اگر این نام گرفته شده باشد، با یک شماره یکتا می‌شود: `NewParameter2`، `NewParameter3` و…
Type	اولین مقدار فهرست **Parameter types**؛ معمولاً `MONITORING`.
Data type	`uint8_t`
Default value	`0`
Minimum و **Maximum**	هر دو `0`؛ یعنی بدون محدودیت بازه.
Resolution	پیش‌فرض همان نوع داده؛ برای `uint8_t` عدد `1`.
تگ‌ها	تگ‌های بله/خیر `FALSE`، و **Tag System Name** و **Tag ParamMode** اولین مقدار فهرستشان (`PARAM_MODE_NONE`). در پروژه‌ی جدول پارامتر، **Tag System Name** همان زیرسیستم ساب‌لیست است.

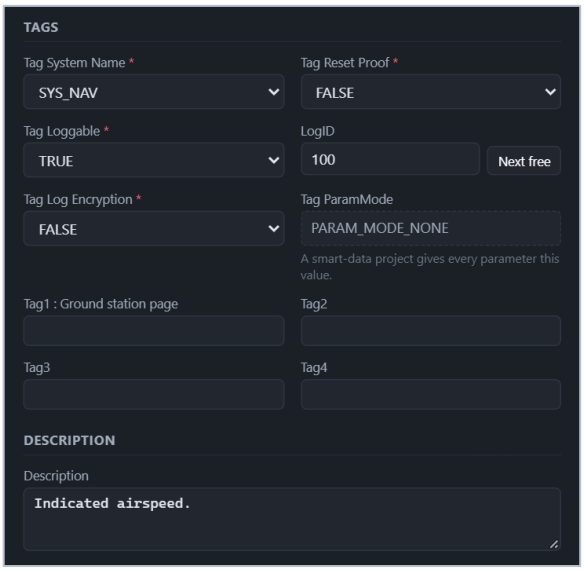
با این مقدارها، پارامتر تازه از همان ابتدا هیچ خطا یا هشداری ندارد و کافی است فقط آنچه را لازم دارید تغییر دهید.

> **نکته:** تا وقتی پروژه هیچ ساب‌لیستی ندارد، دکمه‌های **▾ Add variable +**، **Save**، **Structure types…** و **Enum types…** غیرفعال‌اند، چون پارامتر جایی برای نگه‌داری ندارد. اول یک ساب‌لیست بسازید: **☰** ← **Add parameter sublist +**.

۷.۲ فیلدهای فرم



بالای فرم: Identity و Value



پایین فرم: Tags، LogID و Description

شکل ۹ — فرم یک پارامتر ساده

فیلد	الزامی	توضیح و قاعده
Name	بله	نام پارامتر. باید یک نام معتبر C باشد: فقط حرف انگلیسی، رقم و _، و با رقم شروع نشود. کلیدواژه‌های C و C++ (مثل `int` یا `class`) پذیرفته نمی‌شوند. نام باید در کل پروژه یکتا باشد، حتی بین ساب‌لیست‌های مختلف.
User Name	—	نامی که کاربر می‌خواند. برای پارامتری که لاگ می‌شود الزامی است، چون لاگ مقدار را با همین نام ثبت می‌کند؛ برای بقیه اختیاری است (بخش ۷.۴).
Type	بله	نقش پارامتر؛ یکی از مقدارهای فهرست **Parameter types**.
Data type	بله	یکی از یازده نوع داده (پیوست الف)، یا یکی از اینام‌های ساب‌لیست (فصل ۹).
Size	—	`1` برای مقدار تکی و `n` برای آرایه‌ای با `n` عنصر؛ حداکثر `65535`.
Unit	—	واحد، مثل `mm` یا `m/s`. در کد تولیدی و توضیحات آن می‌آید.
Enabled	—	با برداشتن تیک، پارامتر در کد ساخته نمی‌شود ولی در فایل می‌ماند (بخش ۷.۷).
Default value	—	مقدار اولیه. باید داخل بازه‌ی مجاز باشد.
Minimum و **Maximum**	—	بازه‌ی مجاز. اگر هر دو `0` یا هر دو خالی باشند، کل بازه‌ی نوع داده مجاز است. اگر فقط یکی را پر کنید، هشدار می‌گیرید.
Resolution	—	کوچک‌ترین گام تغییر مقدار. اگر خالی بماند، پیش‌فرض نوع داده به کار می‌رود و هشدار می‌گیرید.
تگ‌ها و **LogID**	—	فصل ۱۱.

فیلد	الزامی	توضیح و قاعده
Description	—	توضیح آزاد. در کد تولیدی می‌آید و پر کردنش هیچ‌وقت الزامی نیست.

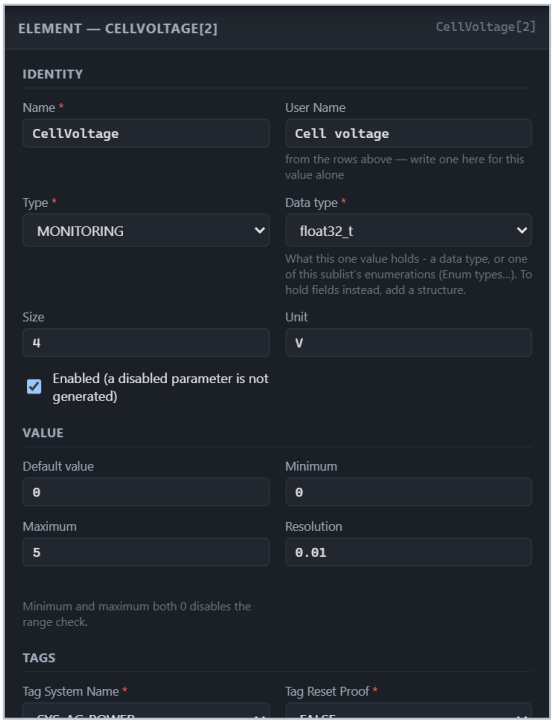
نوع داده‌ی صحیح فقط عدد صحیح می‌پذیرد. اگر برای `int16_t` رزولوشن `1.2` بنویسید، کد C آن را به `1` تبدیل می‌کند؛ برای همین این حالت خطا به شمار می‌آید. یا عدد صحیح بنویسید، یا نوع داده را `float32_t` کنید.

اگر نوع داده را تغییر دهید، مقدارهایی که در نوع تازه نمی‌گنجند خودبه‌خود اصلاح می‌شوند؛ مثلاً بازه‌ای که در نوع تازه جا نمی‌شود به «بدون محدودیت» برمی‌گردد. مقدارهایی که هنوز معتبرند دست نمی‌خورند. نوع `bool_t` بازه و رزولوشن ندارد و فرم برای آن فقط مقدار پیش‌فرض `false` یا `true` را می‌پرسد.

۷.۳ آرایه‌ها و عنصرها

وقتی **Size** بیشتر از `1` باشد، پارامتر در جدول یک ردیف اصلی (بدون کروشه) و برای هر عنصر یک ردیف جداگانه دارد. آرایه‌ها در ابتدا بسته‌اند؛ با ▸ بازشان کنید. هر عنصر ایندکس و شناسه‌ی لاگ خودش را دارد.

روی ردیف یک عنصر که کلیک کنید، فرم همان عنصر باز می‌شود و عنوان فرم مثلاً **Element — CellVoltage[2]** است.



شکل ۱۰ — فرم یک عنصر آرایه: خط راهنمای زیر User Name می‌گوید متن از کجا آمده است

چه اتفاقی می‌افتد	روی یک عنصر
مال همان عنصرند و فقط ردیف همان عنصر را در کد تولیدی تغییر می‌دهند (بخش ۷.۴).	**User Name** و **Description**
کل بلوک شماره‌های پارامتر را جابه‌جا می‌کند (بخش ۷.۵).	**LogID**
مال کل پارامترند. مثلاً واحد را روی `CellVoltage[3]` تغییر دهید، هر چهار عنصر تغییر می‌کنند؛ چون در C فقط یک اعلان وجود دارد.	بقیه‌ی فیلدها (نام، نوع داده، اندازه، واحد، بازه، تگ‌ها)
برای یک عنصر غیرفعال‌اند، چون این کارها مال کل پارامترند. ردیف اصلی (بدون کروشه) را انتخاب کنید.	تکثیر، جابه‌جایی و حذف

۷.۴ نام نمایشی و توضیح هر ردیف

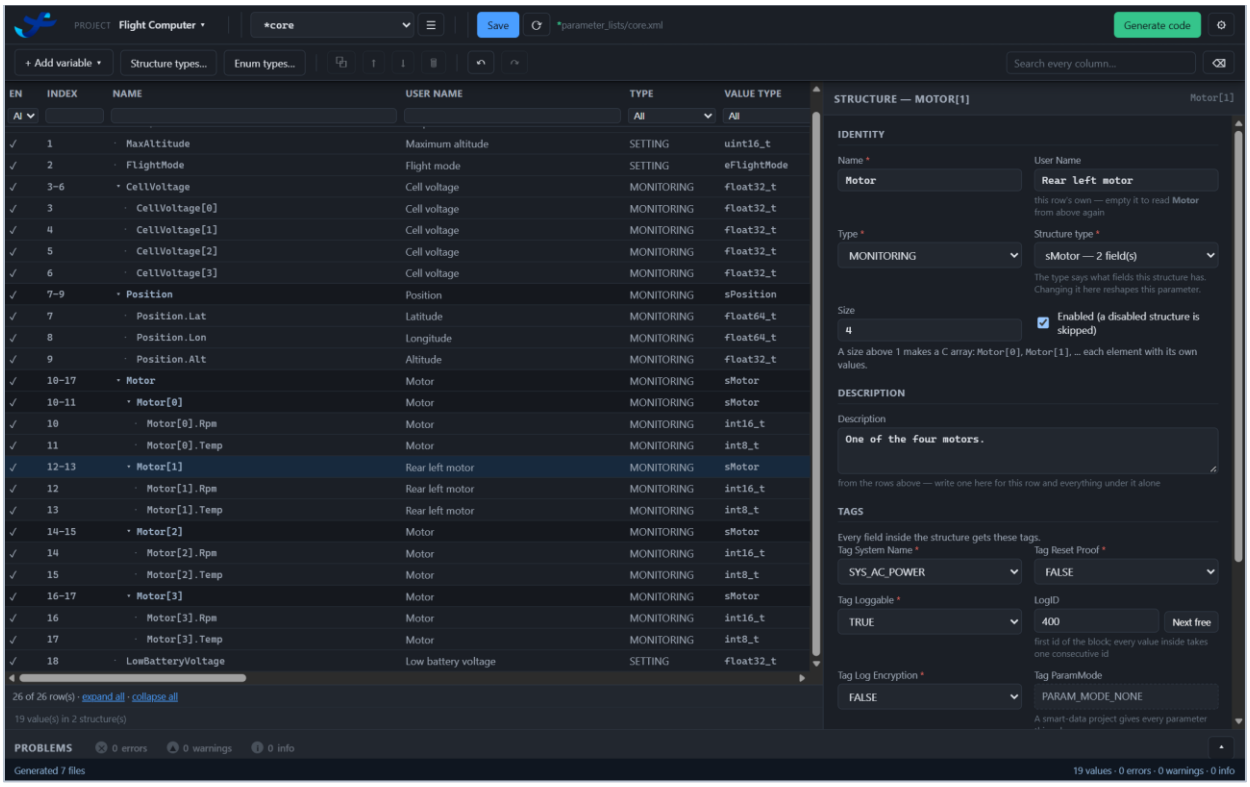
User Name و **Description** را می‌توان روی هر ردیف جدول جداگانه نوشت: روی خود پارامتر، روی یک عنصر آرایه، روی یک عنصر استراکچر، یا روی آرایه‌ای که داخل یک عنصر استراکچر است. قاعده ساده است:

- متنی که روی یک ردیف می‌نویسید به همه‌ی مقدارهای **زیر همان ردیف** می‌رسد، به‌جز مقدارهایی که متن خودشان را دارند.
- هر مقدار، متن **نزدیک‌ترین** ردیف بالای خودش را می‌گیرد که متنی دارد.
- متنی که روی یک ردیف می‌نویسید هرگز به ردیف‌های هم‌سطح آن نمی‌رسد.

مثال: فرض کنید استراکچر `Position` آرایه‌ای دوتایی است و هر عنصرش فیلد `Lat` و آرایه‌ی سه‌تایی `Samples` را دارد؛ یعنی روی هم هشت مقدار:

مقدارهایی که آن متن را می‌گیرند	ردیفی که متن را روی آن می‌نویسید
هر هشت مقدار	`Position`
`Position[0].Lat` و سه مقدار `Position[0].Samples`	`Position[0]`
فقط سه مقدار `Position[0].Samples`	`Position[0].Samples`
فقط همین یک مقدار	`Position[0].Samples[1]`

اگر بعداً متن `Position` را تغییر دهید، فقط مقدارهایی تغییر می‌کنند که متنی از ردیف‌های پایین‌تر نگرفته‌اند؛ متن‌هایی که روی ردیف‌های پایین‌تر نوشته‌اید سر جایشان می‌مانند. شکل زیر همین را نشان می‌دهد: نام `Rear left motor` روی ردیف `Motor[1]` نوشته شده و فقط به مقدارهای همان عنصر رسیده است.



شکل ۱۱ — نامی که روی Motor[1] نوشته شده، فقط به مقدارهای Motor[1] رسیده است

زیر کادرهای **User Name** و **Description** یک خط راهنما هست که می‌گوید متن داخل کادر مال خود همین ردیف است یا از ردیف‌های بالاتر آمده است. برای برگرداندن یک ردیف به حالت پیش‌فرض، کادرش را خالی کنید تا دوباره متن ردیف بالاتر را بگیرد. اگر دقیقاً همان متن ردیف بالاتر را بنویسید، چیزی ذخیره نمی‌شود.

User Name هر مقدار در کد تولیدی در فیلد `pUserName` و **Description** آن در `pDescription` نوشته می‌شود. فیلدی که توضیح ندارد، توضیح استراکچرش را می‌گیرد.

> **نکته:** در ساب‌لیست‌هایی که با نسخه‌های قبلی ابزار ساخته شده‌اند، ممکن است فیلدی از یک آرایه‌ی استراکچر یک نام یا توضیح مشترک برای همه‌ی عنصرها داشته باشد. ویرایشگر هنگام باز کردن چنین ساب‌لیستی آن متن را روی ردیف تک‌تک عنصرها می‌نویسد تا هرکدام جداگانه قابل ویرایش باشد. کد تولیدی هیچ تغییری نمی‌کند، ساب‌لیست تغییرکرده علامت نمی‌خورد و این تبدیل با ذخیره‌ی بعدی در فایل نوشته می‌شود.

۷.۵ شناسه‌ی لاگ روی یک عنصر

شناسه‌های لاگ یک پارامتر پشت سر هم‌اند: خود پارامتر شماره‌ی اول را دارد و هر مقدار بعدی شماره‌ی بعدی را. بنابراین نمی‌توان به عنصری در وسط آرایه شماره‌ی مستقلی داد. به‌جای آن، اگر روی یک عنصر شماره بنویسید، کل بلوک طوری جابه‌جا می‌شود که همان عنصر همان شماره را بگیرد:

پیش از تغییر	پس از نوشتن 205 روی CellVoltage[3]
آرایه‌ی چهارتایی که از 200 شروع می‌شود: 200 تا 203	بلوک به 202 تا 205 می‌رود؛ یعنی CellVoltage[0] شماره‌ی 202 و CellVoltage[3] شماره‌ی 205 را می‌گیرد.

اگر این جابه‌جایی بلوک را به زیر صفر یا بالای `65535` ببرد، ویرایشگر آن را نمی‌پذیرد و می‌گوید کمترین شماره‌ی ممکن چند است.

۷.۶ تکثیر، جابه‌جایی و حذف

- **تکثیر** (⧉ یا `Ctrl+D`): یک کپی از پارامتر، با همه‌ی زیرشاخه‌هایش، درست زیر خودش می‌سازد. نام کپی خودکار یکتا می‌شود و شناسه‌های لاگ آن خالی می‌شوند تا با پارامتر اصلی تداخل نداشته باشند. اگر پارامتر لاگ‌شونده باشد، کپی تا وقتی شناسه‌ی تازه نگیرد خطای `TAG010` دارد؛ دکمه‌ی **Next free** را بزنید.
- **جابه‌جایی** (↑ و ↓): پارامتر را بین هم‌سطح‌هایش جابه‌جا می‌کند. یادتان باشد ترتیب فهرست، ایندکس‌ها را تعیین می‌کند (بخش ۱۲.۴).
- **حذف** (🗑 یا کلید `Delete`): پس از پرسش، پارامتر را حذف می‌کند و با واگرد برمی‌گردد. اگر استراکچری را حذف کنید، تایپ آن حذف نمی‌شود و در **Structure types…** می‌ماند تا خودتان حذفش کنید.

فیلدهای یک استراکچر را نمی‌توان جداگانه تکثیر، جابه‌جا یا حذف کرد؛ این کارها از راه تایپ استراکچر انجام می‌شوند (بخش ۸.۳).

۷.۷ غیرفعال کردن پارامتر

تیک **Enabled** را بردارید تا پارامتر در کد تولیدی ساخته نشود ولی همه‌ی اطلاعاتش در فایل بماند؛ هر وقت خواستید دوباره روشنش کنید. ردیف غیرفعال در جدول کم‌رنگ است و ایندکس ندارد. اگر استراکچری را غیرفعال کنید، هیچ‌کدام از مقدارهای داخل آن ساخته نمی‌شوند.

> **هشدار:** غیرفعال کردن پارامتری در وسط فهرست، ایندکس و جای ذخیره‌ی همه‌ی پارامترهای بعدی را تغییر می‌دهد (بخش ۱۲.۴).

۷.۸ واگرد، ذخیره و بارگذاری دوباره

- **واگرد و ازنو** (↶ و ↷، یا `Ctrl+Z` و `Ctrl+Y`): تا ۶۰ گام به عقب برمی‌گردد و همه‌ی ویرایش‌های ساب‌لیست باز را پوشش می‌دهد: افزودن، حذف، تکثیر، جابه‌جایی، ویرایش فیلدها، و ساختن یا تغییر تایپ‌های استراکچر. تغییرهایی که در پنجره‌ی ساب‌لیست‌ها (☰) می‌دهید مستقیم در فایل پروژه نوشته می‌شوند و با واگرد برنمی‌گردند. با رفتن به ساب‌لیست دیگر، تاریخچه‌ی واگرد پاک می‌شود.
- **ذخیره** (**Save** یا `Ctrl+S`): ساب‌لیست باز را در فایلش می‌نویسد. ذخیره‌ی خودکار وجود ندارد؛ تا وقتی ذخیره نکنید، فایل روی دیسک دست‌نخورده می‌ماند. علامت * کنار نام فایل وقتی برداشته می‌شود که ساب‌لیست دوباره با فایلش یکی شود؛ با ذخیره، یا با واگرد تا حالتی که روی دیسک است.

- **بارگذاری دوباره** (⟳): ساب‌لیست و فایل پروژه را دوباره از دیسک می‌خواند و تغییرهای ذخیره‌نشده را کنار می‌گذارد. اگر `pl_project.json` را با یک ویرایشگر متن تغییر داده‌اید، با همین دکمه اثرش را می‌بینید.

هر کاری که ساب‌لیست باز را عوض کند (رفتن به ساب‌لیست دیگر، باز کردن پروژه‌ی دیگر، بارگذاری دوباره، ساختن یا افزودن ساب‌لیست)، اگر تغییر ذخیره‌نشده‌ای داشته باشید، اول از شما می‌پرسد.

۸. استراکچرها

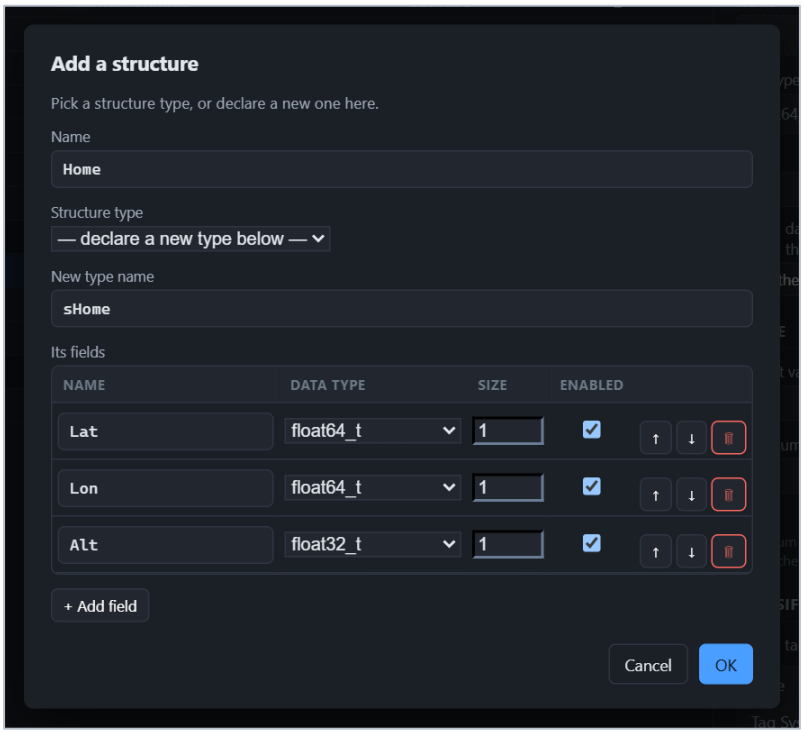
۸.۱ تایپ استراکچر چیست؟

تایپ استراکچر (Structure type) مانند `struct` در C فقط شکل را تعریف می‌کند: یک نام و تعدادی فیلد، هرکدام با نام، نوع داده و اندازه. هر استراکچر در ساب‌لیست نمونه‌ای از یک تایپ است. اگر تایپ را تغییر دهید، همه‌ی استراکچرهایی که از آن ساخته شده‌اند همان لحظه تغییر می‌کنند.

هر تایپ مال ساب‌لیستی است که در آن تعریف شده و در فایل `_types.xml` کنار همان ساب‌لیست ذخیره می‌شود. استراکچرهای یک ساب‌لیست فقط می‌توانند از تایپ‌های خود همان ساب‌لیست استفاده کنند.

۸.۲ ساختن استراکچر

▾ + Add variable ← **User defined…** را بزنید. در پنجره‌ای که باز می‌شود نام استراکچر و تایپ آن را تعیین می‌کنید و اگر تایپ تازه است، فیلدهایش را هم همین‌جا تعریف می‌کنید.



شکل ۱۲ — پنجره‌ی ساختن استراکچر با یک تایپ تازه

بخش	کار
Name	نام استراکچر در ساب‌لیست.
Structure type	یکی از تایپ‌های موجود، یا **declare a new type below** — برای تعریف تایپ تازه.
New type name	نام تایپ تازه، که در کد C به یک `typedef` تبدیل می‌شود.
Its fields	فیلدهای تایپ: برای هر فیلد نام، **Data type** (یک نوع داده، یک اینام، یا تایپ استراکچر دیگری برای استراکچر تودرتو)، **Size** و **Enabled**. با ↑ ↓ 🗑 ترتیب و حذف، و با **+ Add field** فیلد تازه.

پیش از ساخته شدن هر چیز، همه‌ی ورودی‌ها بررسی می‌شوند. اگر چیزی درست نباشد، مثلاً فیلد بی‌نام یا نام تکراری، پنجره باز می‌ماند، دلیل را می‌گوید و چیزی ساخته نمی‌شود. یک واگرد، استراکچر و تایپ تازه‌اش را با هم برمی‌دارد.

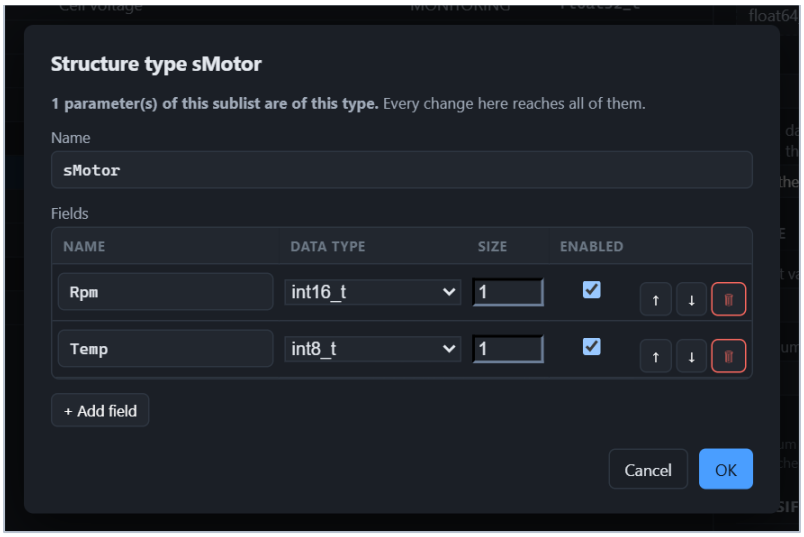
فیلدهای استراکچر تازه با همان مقدارهای اولیه‌ی پارامتر تازه شروع می‌شوند: مقدار پیش‌فرض `0`، بدون محدودیت بازه، و رزولوشن پیش‌فرض نوع داده؛ برای `bool_t` مقدار `FALSE`. بنابراین استراکچر تازه، حتی اگر آرایه‌ای صدتایی باشد، از همان ابتدا هشداری نمی‌گیرد.

۸.۳ ویرایش تایپ‌ها

Structure types… همه‌ی تایپ‌های ساب‌لیست باز را با فیلدها و تعداد استفاده‌هایشان نشان می‌دهد. **+ New structure type** تایپ تازه می‌سازد، **Edit…** نام و فیلدهای یک تایپ را تغییر می‌دهد و **Delete** تایپ را حذف می‌کند. تایپی که استراکچری از آن استفاده می‌کند، حذف نمی‌شود.



شکل ۱۳ — فهرست تایپ‌های استراکچر ساب‌لیست



شکل ۱۴ — ویرایش فیلدهای یک تایپ

فیلدی که به تایپ اضافه کنید همان لحظه، در جای درست، به همه‌ی استراکچرهای آن تایپ اضافه می‌شود و شناسه‌های لاگ بعد از آن هم جابه‌جا می‌شوند. اگر نوع داده‌ی یک فیلد را تغییر دهید، مقدارهای آن فیلد در همه‌ی استراکچرها با نوع تازه هماهنگ می‌شوند. اگر نام یک فیلد را تغییر دهید، مقدارهای قبلی آن (بازه، واحد و توضیح) از دست می‌روند؛ درست مانند تغییر نام یک عضو struct در C.

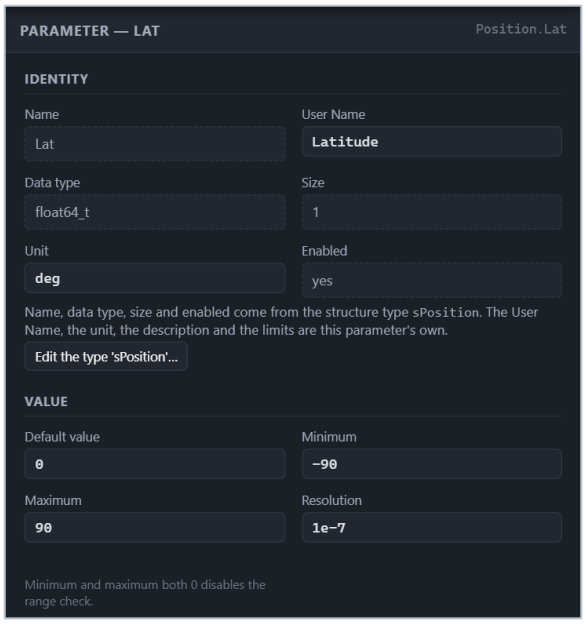
۸.۴ سهم تایپ و سهم هر استراکچر

مال هر استراکچر و فیلدهای آن	مال تایپ (برای همه‌ی استراکچرهای آن تایپ یکسان)
برای هر فیلد: واحد، مقدار پیش‌فرض، بازه، رزولوشن، نام نمایشی و توضیح. برای خود استراکچر: نام، Type، اندازه‌ی آرایه، تگ‌ها، شناسه‌ی لاگ، نام نمایشی و توضیح.	نام فیلد، نوع داده‌ی آن، اندازه‌ی آرایه‌ی فیلد و روشن یا خاموش بودنش؛ چون همه‌ی این استراکچرها یک `typedef` مشترک در C دارند.

تگ‌ها مال خود استراکچرند، نه فیلدهایش: یک استراکچر یا به یک زیرسیستم تعلق دارد یا نه، و یا لاگ می‌شود یا نه. فیلدهای داخلش همان تگ‌ها را به ارث می‌برند. برای همین فرم یک فیلد به‌جای کادرهای تگ، جدولی فقط‌خواندنی با عنوان **Classification — from …** دارد و دکمه‌ای که شما را به خود استراکچر می‌برد. شناسه‌ی لاگ هم همین‌طور است: عددی که روی استراکچر می‌نویسید اولین شماره‌ی بلوک است و مقدارهای داخلش به ترتیب از آن ادامه می‌دهند.



پایین فرم: تگ‌ها از استراکچر Position



بالای فرم: نام، نوع و اندازه از تایپ می‌آیند

شکل ۱۵ — فرم فیلد Position.Lat

در فرم فیلد، نام، نوع داده، اندازه و روشن بودن فیلد فقط‌خواندنی‌اند؛ این‌ها را در تایپ تغییر دهید (دکمه‌ی **Edit the type …**).

۸.۵ آرایه‌ای از استراکچر

استراکچر هم مانند پارامتر ساده می‌تواند **Size** داشته باشد. `Motor` با اندازه‌ی `4` و فیلدهای `Rpm` و `Temp` چهار نمونه‌ی کامل است. در جدول ابتدا `Motor[0]` با همه‌ی فیلدهایش می‌آید، بعد `Motor[1]` و همین‌طور تا آخر؛ همان ترتیبی که C حافظه را می‌چیند. تعداد مقدارها برابر است با تعداد فیلدها ضرب در اندازه؛ یعنی هشت مقدار:

عنصر	ایندکس Rpm	ایندکس Temp	شناسه‌ی لاگ Rpm	شناسه‌ی لاگ Temp
`Motor[0]`	10	11	400	401
`Motor[1]`	12	13	402	403
`Motor[2]`	14	15	404	405
`Motor[3]`	16	17	406	407

هر عنصر در جدول ردیف خودش را دارد (`Motor[0]`، `Motor[1]` و…) و فیلدهای هر عنصر زیر همان ردیف می‌آیند.

۸.۶ قواعد

قاعده	اگر رعایت نشود
هر استراکچر دست‌کم یک فیلد روشن دارد و اندازه‌اش 1 یا بیشتر است.	خطا (STR001)؛ کد ساخته نمی‌شود.
نام فیلدهای یک تایپ یکتاست. تکرار یک نام در دو تایپ مختلف مجاز است.	خطا (TYP008)
نام هر تایپ در کل پروژه یکتاست؛ دو ساب‌لیست نمی‌توانند تایپ هم‌نام داشته باشند.	خطا (TYP016)
هر استراکچر فقط از تایپ‌های ساب‌لیست خودش استفاده می‌کند.	خطا (TYP017)
یک تایپ نمی‌تواند خودش را، مستقیم یا غیرمستقیم، در بر بگیرد.	خطا (TYP015)
ماکروهایی که از نام‌ها ساخته می‌شوند نباید یکی شوند؛ مثلاً MaxAltitude و MAX_ALTITUDE هر دو ماکروی MAX_ALTITUDE را می‌سازند.	خطا (NAME004)
در پروژه‌ی جدول پارامتر، استراکچر و آرایه مجاز نیست.	خطا؛ ویرایشگر هم اجازه‌ی ساختنشان را نمی‌دهد.

عمق تودرتویی محدودیتی ندارد: یک استراکچر می‌تواند استراکچر دیگری را به‌عنوان فیلد داشته باشد.

۹. اینام‌ها

بعضی پارامترها عدد دلخواه نمی‌گیرند و فقط یکی از چند حالت نام‌دار را دارند؛ مثل حالت پرواز، که دستی، پایدارساز، نگه‌داشتن، مأموریت، بازگشت یا فرود است. برای چنین پارامتری یک **اینام** تعریف کنید و آن را نوع داده‌ی پارامتر قرار دهید. از آن به بعد در ویرایشگر مقدار پیش‌فرض را از یک فهرست کشویی انتخاب می‌کنید، و در فرم‌ور هر نوشتنی که عددش جزو این حالت‌ها نباشد، رد می‌شود.

۹.۱ اینام چیست؟

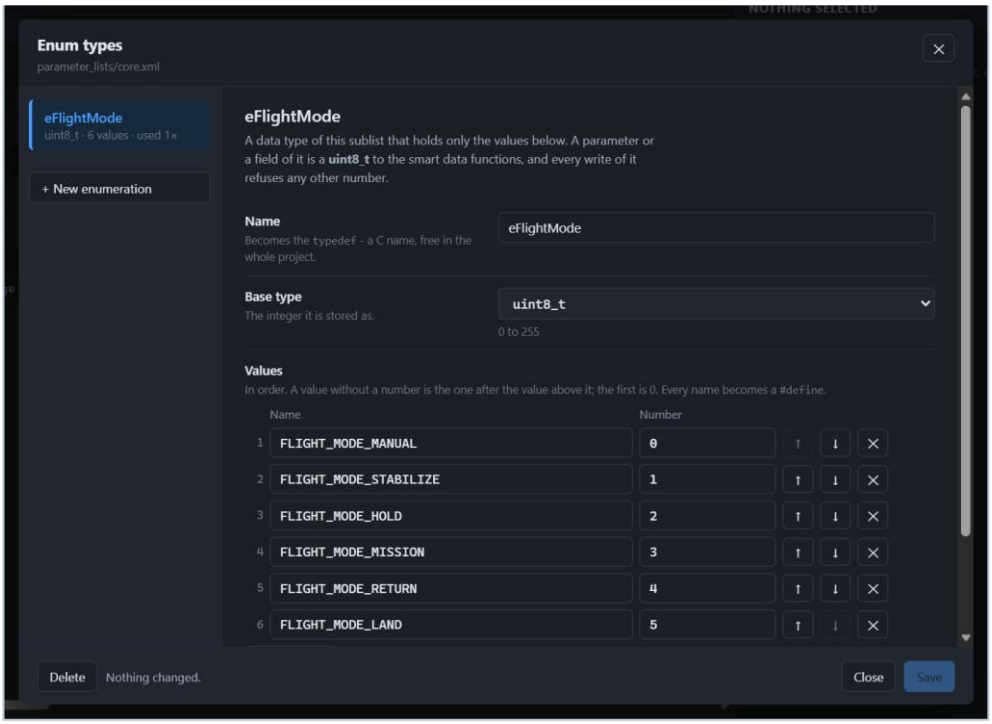
یک اینام یک نام، یک **نوع پایه** و فهرستی از مقدارها دارد؛ هر مقدار یک نام و یک عدد. نوع پایه یکی از نوع‌های صحیح است (`uint8_t`، `int8_t`، `uint16_t`، `int16_t`، `uint32_t`، `int32_t`، `uint64_t` یا `int64_t`) و تعیین می‌کند مقدار در حافظه چند بایت جا می‌گیرد. پارامتر اینام در واقع یک مقدار از همین نوع پایه است؛ پس جای حافظه، ایندکس، شناسه‌ی لاگ و آفست تگ‌هایش درست مانند یک پارامتر معمولی از همان نوع است.

هر اینام، مانند تایپ استراکچر، مال ساب‌لیستی است که در آن تعریف شده و در فایل `_types.xml` کنار همان ساب‌لیست ذخیره می‌شود. پارامترهای یک ساب‌لیست فقط از اینام‌های خود همان ساب‌لیست استفاده می‌کنند، و نام هر اینام در کل پروژه یکتاست. فیلد یک استراکچر هم می‌تواند از نوع اینام باشد.

> **نکته:** اینام فقط در قالب Smart data هست. در پروژه‌ی جدول پارامتر، دکمه‌ی **Enum types…** غیرفعال است (بخش ۱۴.۳).

۹.۲ تعریف و ویرایش اینام

دکمه‌ی **Enum types…** را در نوار ابزار بزنید. سمت چپ پنجره اینام‌های ساب‌لیست باز فهرست شده‌اند و زیر هرکدام نوع پایه، تعداد مقدارها و تعداد استفاده‌هایش نوشته شده است. با کلیک روی هرکدام، صفحه‌ی آن سمت راست باز می‌شود. **New enumeration +** صفحه‌ی یک اینام تازه را باز می‌کند.



شکل ۱۶ — پنجره‌ی Enum types با اینام eFlightMode

بخش	کار
Name	نام اینام، که در کد C نام `typedef` آن می‌شود؛ مثل `eFlightMode`. باید نام معتبر C باشد و در کل پروژه با هیچ نوع داده، تایپ استراکچر یا اینام دیگری یکی نباشد.
Base type	نوع صحیحی که مقدار در آن نگه داشته می‌شود. بازه‌ی آن زیر کادر نوشته می‌شود؛ مثلاً `0 to 255` برای `uint8_t`.
Values	مقدارها، به ترتیب. هر سطر یک **Name** و یک **Number** دارد. با ↑ و ↓ ترتیب را عوض کنید، با ✕ مقداری را بردارید و با **+ Add value** (یا Enter در یک سطر) سطر تازه بسازید.
Used by	پارامترها و فیلدهایی که از این اینام استفاده می‌کنند.

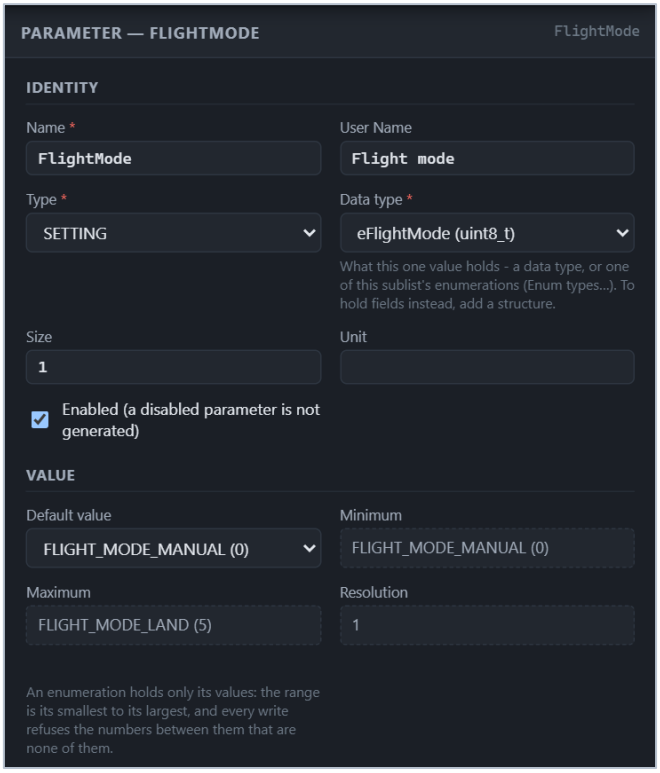
- نام هر مقدار در کد C یک `#define` می‌شود؛ پس باید نام معتبر C باشد و در اینام تکرار نشود. معمولاً با حروف بزرگ و پیشوندی مشترک نوشته می‌شود: `FLIGHT_MODE_HOLD`.
- اگر **Number** سطری را خالی بگذارید، عدد سطر بالایی به‌اضافه‌ی یک را می‌گیرد و سطر اول `0` را؛ کادر خالی همین عدد را کم‌رنگ نشان می‌دهد. عدد را می‌توانید دهدهی یا هگز (`0x10`) بنویسید و منفی هم مجاز است اگر نوع پایه علامت‌دار باشد.
- عددها لازم نیست پشت سر هم باشند (مثلاً `0, 1, 5, 6` هم درست است)، ولی دو مقدار نمی‌توانند یک عدد داشته باشند و هر عدد باید در نوع پایه جا شود.

Save (برای اینام تازه **Declare**) تغییرها را اعمال می‌کند. اگر چیزی درست نباشد، پیامی بالای پنجره دلیلش را می‌گوید و چیزی تغییر نمی‌کند. تا وقتی ذخیره نکرده‌اید، کنار نام اینام در فهرست سمت چپ نقطه‌ای آبی هست؛ اگر با تغییر

ذخیره‌نشده **Close** را بزنید، پنجره یک بار هشدار می‌دهد و با زدن دوباره‌ی **Close** تغییرها کنار گذاشته می‌شوند. همه‌ی این کارها با واگرد برمی‌گردند و مانند بقیه‌ی ویرایش‌ها با **Save** نوار بالا در فایل نوشته می‌شوند.

۹.۳ پارامتری از نوع اینام

در فرم پارامتر، فهرست **Data type** پس از نوع‌های داده گروهی به نام **Enumerations** دارد که اینام‌های ساب‌لیست را با نوع پایه‌شان نشان می‌دهد؛ مثل `eFlightMode (uint8_t)`. یکی را انتخاب کنید:



شکل ۱۷ — فرم پارامتر FlightMode از نوع eFlightMode

- نوع داده‌ی پارامتر همان نوع پایه‌ی اینام می‌شود.
- **Default value** فهرستی کشویی از مقدارهای اینام است، هرکدام با عددش؛ مثل `FLIGHT_MODE_MANUAL (0)`. اگر مقدار پیش‌فرض قبلی جزو مقدارهای اینام نباشد، اولین مقدار اینام جای آن را می‌گیرد.
- **Minimum** و **Maximum** کوچک‌ترین و بزرگ‌ترین مقدار اینام‌اند و **Resolution** برابر `1`؛ هر سه فقط‌خواندنی‌اند و با تغییر اینام خودبه‌خود به‌روز می‌شوند.
- برای برگرداندن پارامتر به یک عدد معمولی، یک نوع داده‌ی عادی را از همان فهرست انتخاب کنید.

در جدول پارامترها، ستون **Value type** نام اینام را نشان می‌دهد و ستون **Default** نام مقدار پیش‌فرض را. برای فیلد استراکچر، اینام در پنجره‌ی تایپ استراکچر انتخاب می‌شود: فهرست **Data type** هر فیلد همان گروه **Enumerations** را دارد (بخش ۸.۳).

۹.۴ تغییر اینامی که استفاده شده است

چه اتفاقی می‌افتد	تغییر
مقدار پیش‌فرض‌هایی که همان مقدار بودند، همراهش به عدد تازه می‌روند؛ چون معنی پیش‌فرض نام مقدار است، نه عددش. بازه‌ی پارامترها هم به‌روز می‌شود.	عدد یک مقدار
همه‌ی پارامترها و فیلدها نام تازه را می‌گیرند.	نام اینام
نوع داده‌ی همه‌ی پارامترها و فیلدهای آن عوض می‌شود. نوع پایه‌ای که مقداری در آن جا نشود، پذیرفته نمی‌شود.	نوع پایه
اگر آن مقدار پیش‌فرض پارامتری باشد، پذیرفته نمی‌شود و پیام می‌گوید کدام پارامتر؛ اول پیش‌فرض آن را عوض کنید.	برداشتن یک مقدار
تا وقتی پارامتر یا فیلدی از آن استفاده می‌کند، پذیرفته نمی‌شود و پیام همه را نام می‌برد.	حذف اینام (**Delete**)

> **هشدار:** تغییر نوع پایه، اندازه‌ی مقدار را عوض می‌کند و مانند تغییر نوع داده‌ی هر پارامتر دیگری، آفست مقدارهای بعدی را جابه‌جا می‌کند (بخش ۱۲.۴).

۹.۵ اینام در کد C

فایل `cg_parameter_list_types.h` برای هر اینام ساب‌لیست‌های ساخت، حتی اگر هنوز پارامتری از آن استفاده نکند، یک `typedef`، یک `#define` برای هر مقدار و ماکروی `IS_VALID_` می‌سازد:

```
typedef uint8_t eFlightMode;

#define FLIGHT_MODE_MANUAL     ((eFlightMode)0U)
#define FLIGHT_MODE_STABILIZE  ((eFlightMode)1U)
#define FLIGHT_MODE_HOLD       ((eFlightMode)2U)
...
#define FLIGHT_MODE_LAND       ((eFlightMode)5U)

#define IS_VALID_eFlightMode(value_)  ( \
    ((value_) == FLIGHT_MODE_MANUAL) || \
    ...
    ((value_) == FLIGHT_MODE_LAND) )
```

اینام یک `typedef` از نوع پایه است، نه `enum` زبان C؛ پس `eFlightMode` همان `uint8_t` است و با تابع‌های smart data همان نوع کار می‌کند. هر بار که مقداری نوشته شود، چه با `fSmartData_U8_Write`، چه با نسخه‌ی ایندکسی آن و چه به‌عنوان مقدار پیش‌فرض، کتابخانه بررسی می‌کند عدد جزو مقدارهای اینام باشد. عددی که نباشد، حتی اگر بین کمینه و بیشینه باشد، رد می‌شود: مقدار قبلی می‌ماند و تابع `eSMART_DATA_ERROR_VALUE_OUT_OF_RANGE` برمی‌گرداند.

```
eFlightMode mode = FLIGHT_MODE_HOLD;
fSmartData_U8_Write(&Cg_SmartData_FlightMode, &mode);   /* eSMART_DATA_OK */

mode = 7U;                                              /* none of its values */
fSmartData_U8_Write(&Cg_SmartData_FlightMode, &mode);   /* ..._VALUE_OUT_OF_RANGE */

if (IS_VALID_eFlightMode(mode)) { ... }                 /* the same check, yourself */
```

ماکروی `IS_VALID_` را هر جا خودتان بخواهید عددی را پیش از نوشتن بسنجید به کار ببرید. این ماکرو آرگومانش را چند بار می‌خواند؛ پس به آن متغیر ساده بدهید، نه عبارتی مثل `x++`.

۹.۶ قواعد

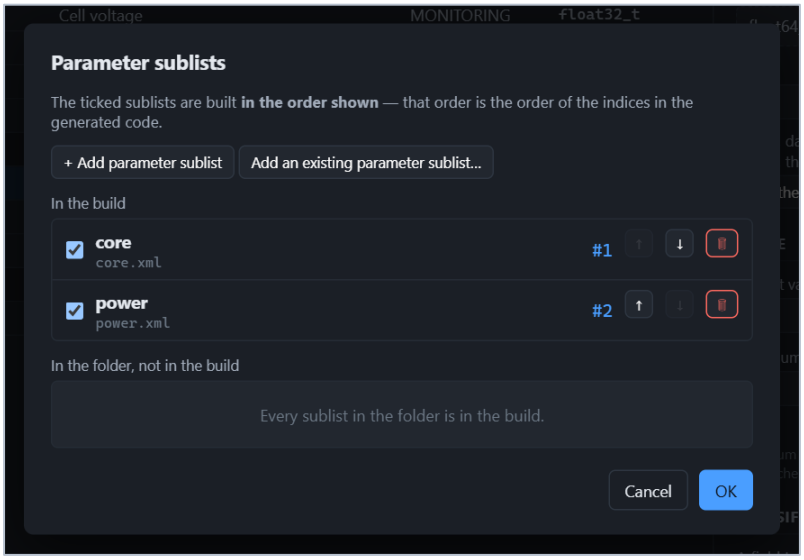
اگر رعایت نشود	قاعده
خطا (`ENUM001`، `ENUM004`)	نام اینام و نام هر مقدار، نام معتبر C است.
خطا (`ENUM003`، `ENUM005`، `ENUM006`)	اینام دست‌کم یک مقدار دارد، و نام‌ها و عددهای مقدارهایش تکراری نیستند.
خطا (`ENUM007`)	هر عدد در نوع پایه جا می‌شود.
خطا (`ENUM008`)	نام هر مقدار در کل پروژه یکتاست؛ دو اینام نمی‌توانند مقدار هم‌نام داشته باشند.
خطا (`ENUM011`)	پارامتر فقط از اینام‌های ساب‌لیست خودش استفاده می‌کند.
خطا (`ENUM015`)	مقدار پیش‌فرض پارامتر یکی از مقدارهای اینام است.
خطا (`ENUM013`)	در پروژه‌ی جدول پارامتر اینامی به کار نمی‌رود.

ویرایشگر بیشتر این‌ها را همان هنگام ذخیره‌ی اینام رد می‌کند؛ کدها برای فایلی‌اند که دستی ویرایش شده باشد.

۱۰. ساب‌لیست‌ها

۱۰.۱ چند ساب‌لیست در یک پروژه

یک پروژه‌ی واقعی معمولاً چند ساب‌لیست دارد؛ مثلاً یکی برای هر زیرسیستم یا هر تیم. ساب‌لیست‌ها در پوشه‌ی `parameter_lists` پروژه نگه داشته می‌شوند و پنجره‌ی **Parameter sublists** (دکمه‌ی ☰) تعیین می‌کند کدام‌ها در ساخت کد شرکت کنند و به چه ترتیبی. همه‌ی ساب‌لیست‌های انتخاب‌شده به همان ترتیب ادغام می‌شوند و مانند یک فهرست واحد پردازش می‌شوند.



شکل ۱۸ — پنجره‌ی ساب‌لیست‌های پروژه

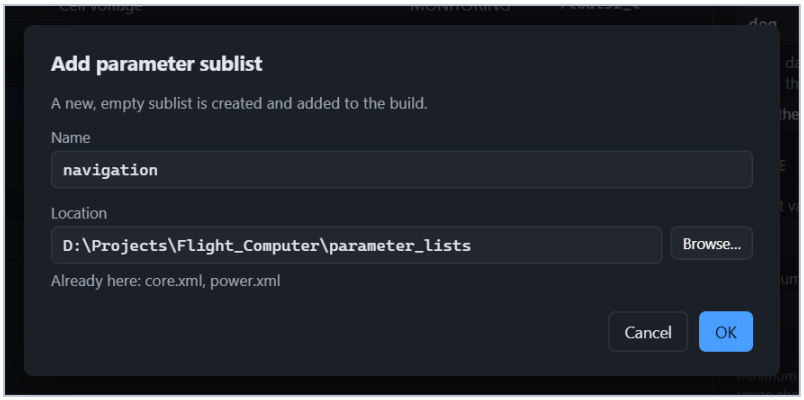
کار	در پنجره
آیا این ساب‌لیست در ساخت کد شرکت می‌کند.	تیک هر ردیف
جایگاه ساب‌لیست در ساخت.	#1، #2 و…
تغییر ترتیب. این ترتیب، ترتیب ایندکس‌ها در کد تولیدی است؛ پس تصمیمی واقعی است، نه جزئیاتی ظاهری.	↑ و ↓
برداشتن ساب‌لیست از ساخت، یا حذف فایل آن (بخش ۱۰.۴).	🗑
ساختن ساب‌لیست تازه (بخش ۱۰.۲).	**+ Add parameter sublist**
افزودن ساب‌لیستی که از قبل وجود دارد (بخش ۱۰.۳).	**Add an existing parameter sublist…**
ساب‌لیست‌هایی که در ساخت شرکت می‌کنند، به ترتیب.	**In the build**
ساب‌لیست‌هایی که در پوشه هستند ولی در ساخت نیستند؛ مثلاً ساب‌لیستی که همکارتان تازه اضافه کرده است.	**In the folder, not in the build**

با زدن **OK**، انتخاب شما مستقیم در فایل `pl_project.json` نوشته می‌شود؛ بنابراین تولید کد از خط فرمان هم دقیقاً همین ساب‌لیست‌ها را به همین ترتیب به کار می‌برد.

همیشه فقط یک ساب‌لیست باز است و ویرایش می‌شود، ولی بررسی خطاها روی مجموع همه‌ی ساب‌لیست‌های ساخت انجام می‌شود؛ پس نام یا شناسه‌ی لاگ تکراری بین دو ساب‌لیست هم دیده می‌شود.

۱۰.۲ ساختن ساب‌لیست

در پنجره‌ی ساب‌لیست‌ها **+ Add parameter sublist** را بزنید.



شکل ۱۹ — پنجره‌ی ساختن ساب‌لیست تازه

کار	بخش
نام ساب‌لیست، که نام فایل آن هم می‌شود؛ پس فقط حرف انگلیسی، رقم، `_` و `-` بنویسید. فایلی با همین نام و پسوند `.xml` ساخته می‌شود.	**Name**
پوشه‌ای که فایل در آن ساخته می‌شود؛ پیش‌فرض پوشه‌ی `parameter_lists` پروژه است. **Browse…** پنجره‌ی انتخاب پوشه‌ی ویندوز را باز می‌کند. زیر کادر، فایل‌هایی که از قبل در آن پوشه هستند نمایش داده می‌شوند. اگر پوشه هنوز وجود نداشته باشد، همراه ساب‌لیست ساخته می‌شود.	**Location**

ساب‌لیست تازه بلافاصله ساخته، به انتهای ساخت اضافه و باز می‌شود. اگر فایلی با همین نام در آن پوشه وجود داشته باشد، ساب‌لیست ساخته نمی‌شود.

> **نکته:** در پروژه‌ی جدول پارامتر، به‌جای کادر نام، فهرست **Sub-system** نمایش داده می‌شود و نام فایل از روی زیرسیستم ساخته می‌شود (بخش ۱۴.۲).

۱۰.۳ افزودن ساب‌لیست موجود

Add an existing parameter sublist… پنجره‌ی Open ویندوز را باز می‌کند که فایل‌های `.xml` را نشان می‌دهد و از پوشه‌ی ساب‌لیست‌های پروژه شروع می‌کند. فایل را انتخاب کنید:

- اگر فایل خارج از پروژه باشد، همراه فایل تایپ‌های استراکچرش به پروژه **کپی** می‌شود. به این ترتیب پروژه همچنان یک پوشه‌ی مستقل می‌ماند و ویرایش این نسخه روی پروژه‌ی دیگر اثری ندارد.
- اگر فایل از قبل داخل پروژه باشد، فقط به ساخت اضافه می‌شود.
- اگر از قبل در ساخت باشد، فقط باز می‌شود.
- اگر فایل ساب‌لیست نباشد یا با پروژه سازگار نباشد (مثلاً تایپ استراکچری با نام تکراری داشته باشد)، پذیرفته نمی‌شود و دلیلش گفته می‌شود.

۱۰.۴ برداشتن یا حذف ساب‌لیست

دکمه‌ی 🗑 کنار هر ساب‌لیست دو پرسش جداگانه می‌پرسد:

۱. **از ساخت برداشته شود؟** ساب‌لیست دیگر در کد تولیدی نیست، ولی فایلش می‌ماند و هر وقت بخواهید می‌توانید دوباره آن را به ساخت اضافه کنید.

۲. **فایل هم حذف شود؟** فایل ساب‌لیست و فایل تایپ‌های استراکچرش از دیسک پاک می‌شوند. این کار برگشت‌پذیر نیست.

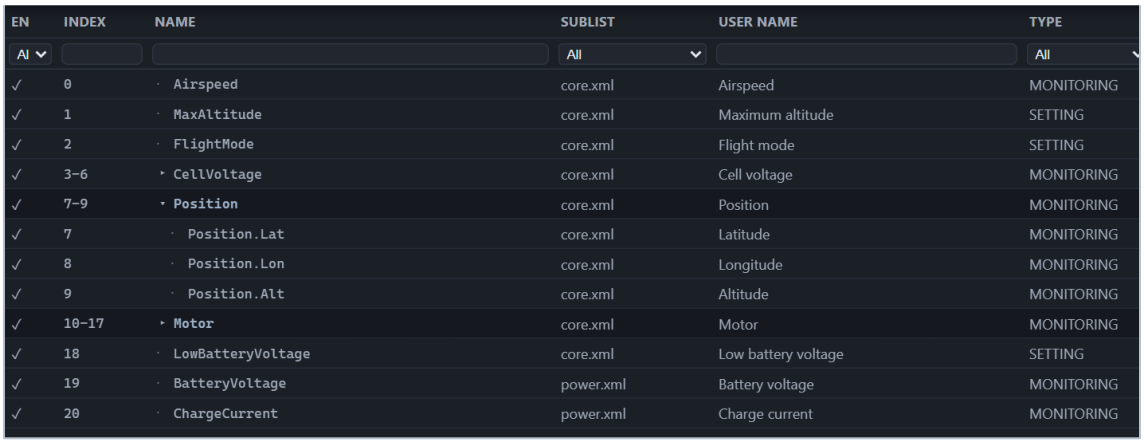
این کار همان لحظه انجام می‌شود و منتظر دکمه‌ی **OK** نمی‌ماند. آخرین ساب‌لیست ساخت را نمی‌توان برداشت؛ هر پروژه دست‌کم یک ساب‌لیست لازم دارد.

> **هشدار:** برداشتن یک ساب‌لیست یا تغییر ترتیب ساب‌لیست‌ها، ایندکس همه‌ی پارامترهای ساب‌لیست‌های بعدی را تغییر می‌دهد (بخش ۱۲.۴).

۱۰.۵ رفتن بین ساب‌لیست‌ها و نمای ادغام‌شده

کادر انتخاب ساب‌لیست در نوار بالا (شماره‌ی ۲ در شکل ۵) ساب‌لیست باز را نشان می‌دهد و با آن به ساب‌لیست دیگری می‌روید. این کادر فقط وقتی دیده می‌شود که پروژه دست‌کم دو ساب‌لیست داشته باشد.

گزینه‌ی **every sublist, merged** — کل ساخت را یک‌جا نشان می‌دهد: همه‌ی ساب‌لیست‌ها به ترتیب ساخت، با ایندکس‌های نهایی و ستون **Sublist** که می‌گوید هر ردیف از کدام فایل آمده است. این تنها نمایی است که به پرسش «در جدول تولیدی دقیقاً چه چیزی هست؟» پاسخ می‌دهد.



EN	INDEX	NAME	SUBLIST	USER NAME	TYPE
All			All		All
✓	0	Airspeed	core.xml	Airspeed	MONITORING
✓	1	MaxAltitude	core.xml	Maximum altitude	SETTING
✓	2	FlightMode	core.xml	Flight mode	SETTING
✓	3–6	▸ CellVoltage	core.xml	Cell voltage	MONITORING
✓	7–9	▾ Position	core.xml	Position	MONITORING
✓	7	Position.Lat	core.xml	Latitude	MONITORING
✓	8	Position.Lon	core.xml	Longitude	MONITORING
✓	9	Position.Alt	core.xml	Altitude	MONITORING
✓	10–17	▸ Motor	core.xml	Motor	MONITORING
✓	18	LowBatteryVoltage	core.xml	Low battery voltage	SETTING
✓	19	BatteryVoltage	power.xml	Battery voltage	MONITORING
✓	20	ChargeCurrent	power.xml	Charge current	MONITORING

شکل ۲۰ — نمای ادغام‌شده: ستون Sublist و ایندکس‌های نهایی

این نما فقط‌خواندنی است و دکمه‌های ویرایش در آن خاموش‌اند. با کلیک روی هر ردیف، ساب‌لیستی که آن ردیف مال آن است باز و همان ردیف انتخاب می‌شود.

۱۱. تگ‌ها، شناسه‌ی لاگ و لیست‌های کشویی

۱۱.۱ تگ‌ها

بخش **Tags** فرم، برچسب‌های دسته‌بندی هر پارامتر را می‌گیرد. نام همه‌ی تگ‌ها در فرم با `Tag` شروع می‌شود.

تگ	مقدار	معنی
Tag System Name	یکی از مقدارهای فهرست **System names**، مثل `SYS_NAV`	زیرسیستمی که پارامتر به آن تعلق دارد.
Tag Reset Proof	`TRUE` یا `FALSE`	مقدار پارامتر پس از ریست حفظ می‌شود (ذخیره می‌شود).
Tag Loggable	`TRUE` یا `FALSE`	مقدار پارامتر در لاگ ثبت می‌شود.
Tag Log Encryption	`TRUE` یا `FALSE`	مقدار پارامتر هنگام ذخیره رمزنگاری می‌شود.
Tag ParamMode	یکی از مقدارهای فهرست **Parameter modes**، مثل `PARAM_MODE_READ_ONLY`؛ در پروژه‌ی Smart data همیشه `PARAM_MODE_NONE`	سطح دسترسی به پارامتر.
Tag1 تا **Tag4**	یک عدد یا یک نام C	تگ‌های آزاد برای استفاده‌ی خود برنامه‌ی فرم‌ور؛ معنی هرکدام را پروژه تعیین می‌کند (بخش ۱۱.۶).

پنج تگ اول الزامی‌اند و خالی ماندن هرکدام برای پارامتر فعال خطاست (`TAG001`). تگ‌های بله/خیر سه حالت دارند: خالی، `TRUE` و `FALSE`.

در پروژه‌ی Smart data، **Tag ParamMode** فقط‌خواندنی و همیشه `PARAM_MODE_NONE` است؛ جدول و فرم همین را نشان می‌دهند و کد تولیدی هم برای همه‌ی پارامترها همین را می‌نویسد. مقداری که در خود ساب‌لیست ذخیره شده دست نمی‌خورد، چون همان ساب‌لیست ممکن است در یک پروژه‌ی جدول پارامتر هم به کار برود که آنجا حالت دسترسی یک ستون واقعی است (فصل ۱۴). پارامتر تازه با `PARAM_MODE_NONE` ذخیره می‌شود.

> **نکته:** ابزار فقط مقدار تگ‌ها را می‌گیرد، درستی‌شان را بررسی می‌کند و در کد تولیدی می‌گذارد. اینکه Reset Proof دقیقاً کجا و کی ذخیره شود یا Loggable چطور لاگ شود، تصمیم برنامه‌ی فرم‌ور است.

۱۱.۲ مقدار تگ‌ها

- تگ‌های فهرستی (**Tag System Name** و **Tag ParamMode**) فقط مقدارهای فهرست کشویی خودشان را می‌پذیرند (بخش ۱۱.۵). در کد C، مقدار عددی هر گزینه همان عددی است که در پنجره‌ی **Drop-down lists…** جلوی آن نوشته شده.
- تگ‌های آزاد یک عدد (از `0` تا `4294967295`) یا یک نام C مثل `APP_FLAG_X` می‌پذیرند. اگر نام C بنویسید، فایل هدری که آن نام را تعریف می‌کند باید در `codegen.extra_includes` فایل پروژه آمده باشد؛ وگرنه کد تولیدی کامپایل نمی‌شود (بخش ۱۵.۵). اگر مقدار یک تگ آزاد را پاک کنید، آن تگ دیگر مقداری ندارد و در فایل ساب‌لیست هم نوشته نمی‌شود.
- در استراکچر، تگ‌ها روی خود استراکچر تنظیم می‌شوند و فیلدهای آن، همان‌ها را به ارث می‌برند (بخش ۸.۴).

۱۱.۳ شناسه‌ی لاگ

توضیح	قاعده
اگر **Tag Loggable** برابر `TRUE` نباشد، شناسه نادیده گرفته می‌شود و در جدول `0` نوشته می‌شود. فرم در این حالت کادر شناسه را کم‌رنگ نشان می‌دهد و زیرش همین را یادآوری می‌کند.	فقط برای مقدار لاگ‌شونده
عددی صحیح از `0` تا `65535`. نبودن آن خطاست (`TAG010`).	الزامی برای مقدار لاگ‌شونده
دو مقدار نمی‌توانند شناسه‌ی یکسانی داشته باشند، حتی اگر در دو ساب‌لیست مختلف باشند (`TAG012`).	یکتا
آرایه‌ی `n` تایی، `n` شناسه‌ی پیاپی می‌گیرد. در استراکچر، عددی که روی آن می‌نویسید اولین شناسه‌ی بلوک است و مقدارهای داخلش به ترتیب، هرکدام یک شناسه، ادامه می‌دهند.	پشت سر هم
شناسه یک مقدار را شماره‌گذاری می‌کند، نه یک بایت؛ `float64_t` هم فقط یک شناسه می‌گیرد.	مستقل از نوع داده

مثال: `Position` با شناسه‌ی `300` سه مقدار دارد و بلوک `300` تا `302` را می‌گیرد: `Position.Lat` شماره‌ی `300`، `Position.Lon` شماره‌ی `301` و `Position.Alt` شماره‌ی `302`.

دکمه‌ی **Next free** کنار کادر LogID، اولین شماره‌ی آزاد پس از همه‌ی شماره‌های به‌کاررفته در کل پروژه را در کادر می‌گذارد. با این دکمه عملاً هیچ‌وقت تداخلی پیش نمی‌آید.

۱۱.۴ آفست حافظه‌ی تگ‌ها

برای هر تگ، ابزار جای هر پارامتر را در ناحیه‌ی حافظه‌ی آن تگ حساب می‌کند. هر تگ یک شمارنده‌ی جداگانه دارد که از صفر شروع می‌شود و برای هر پارامتری که آن تگ را دارد، به اندازه‌ی «اندازه‌ی نوع داده × تعداد عنصرها» جلو می‌رود؛ در

تگ‌های بله/خیر یعنی پارامترهایی که مقدار آن تگ‌شان `TRUE` است. پارامتری که آن تگ را ندارد، جایی مصرف نمی‌کند. مثال از پروژه‌ی Flight Computer برای **Tag Reset Proof**:

آفست	اندازه	نوع داده	پارامتر
`0`	۲ بایت	`uint16_t`	`MaxAltitude`
`2`	۱ بایت	`uint8_t`	`FlightMode`
`3`	۴ بایت	`float32_t`	`LowBatteryVoltage`
`PARAMETER_LIST_TAG_RESET_PROOF_SIZE`	۷ بایت		جمع

فرم‌ور می‌تواند برای همه‌ی مقدارهای ماندگار یک ناحیه‌ی هفت‌بایتی کنار بگذارد. اندازه‌ی کل هر تگ به شکل ماکرو در `cg_parameter_list_defines.h` می‌آید، ولی آفست تک‌تک پارامترها فقط در جدول‌های توضیحی فایل `cg_parameter_list_table.h` نوشته می‌شود؛ آن فایل فقط توضیح است و کدی ندارد.

به همین دلیل ترتیب پارامترها مهم است: افزودن، حذف، غیرفعال کردن یا جابه‌جا کردن یک پارامتر، آفست همه‌ی پارامترهای بعدی را تغییر می‌دهد (بخش ۱۲.۴).

۱۱.۵ لیست‌های کشویی

چهار فهرستی که ویرایشگر در فیلدهای کشویی پیشنهاد می‌دهد، از ⚙ ← **Drop-down lists…** ویرایش می‌شوند.



شکل ۲۱ — پنجره‌ی لیست‌های کشویی

کاربرد	فهرست
مقدارهای فیلد **Type**: `MONITORING`، `COMMAND` و `SETTING`.	**Parameter types**
مقدارهای **Tag System Name**، هر کدام با عددش. در کد، هر نام یک `#define` با همان عدد است.	**System names**
مقدارهای **Tag ParamMode**، هر کدام با عددش. اولین مقدار `PARAM_MODE_NONE` است.	**Parameter modes**
نوع‌های داده‌ای که در فیلد **Data type** پیشنهاد می‌شوند.	**Value types**

هر سطر یک مقدار است. در **System names** و **Parameter modes** هر سطر نام و عددی است که در کد به آن داده می‌شود: `SYS_NAV = 16`. عدد را می‌توانید دهدهی یا هگز (`0x10`) بنویسید، از `-2147483648` تا `4294967295`؛ پس عدد منفی هم مجاز است، مثل `PARAM_MODE_NONE = -1`. سطری که عدد ندارد، عدد سطر بالایی به‌اضافه‌ی یک را می‌گیرد و سطر اول `0` را؛ پنجره هر بار همه‌ی سطرها را با عددشان نشان می‌دهد. کنار هر فهرست تعداد مقدارهایش نوشته می‌شود تا فهرست ناقص فوراً دیده شود.

```
SYS_UNKNOWN = 0
SYS_GS_MANAGER = 2
SYS_NAV = 0x10
SYS_NEW
```

در این نمونه `SYS_NAV` عدد `16` و `SYS_NEW` عدد `17` را می‌گیرد. اگر سطری خوانده نشود (مثلاً `SYS_NAV = abc`)، **OK** چیزی ذخیره نمی‌کند، پیامی می‌گوید کدام سطر غلط است و پنجره با همان متن باز می‌ماند. دو نام یک فهرست نباید یک عدد داشته باشند؛ وگرنه خطای `DEF005` می‌گیرید و کدی تولید نمی‌شود.

> **مهم:** هر مقدار **Parameter types** باید در فایل `parameter_descriptor.h` فرم‌ور هم تعریف شده باشد (`ePARAMETER_TYPE_…`)؛ وگرنه خطا می‌گیرید. این فهرست را فقط هم‌زمان با آن فایل تغییر دهید.

این فهرست‌ها مال رایانه‌اند، نه یک پروژه: همه‌ی پروژه‌های این رایانه از یک نسخه استفاده می‌کنند، چون مثلاً `SYS_AC_POWER` در همه‌ی پروژه‌ها یک معنی دارد. این نسخه در فایل `paramgen/var/choices.json` نگه داشته می‌شود.

دکمه‌ی **Restore the company defaults** فهرست‌ها را به مقدارهای پیش‌فرض شرکت برمی‌گرداند. این مقدارها، هر نام با عددش، در فایل `paramgen/templates/choices.json` آمده‌اند.

لازم نیست مقدارهای `SYS_…` و `PARAM_MODE_…` را در کد C دستی تعریف کنید. ابزار آن‌ها را در فایل تولیدی `cg_parameter_list_defines.h` می‌سازد، هر کدام با عددش و داخل `#ifndef`:

```
#ifndef SYS_UNKNOWN
#define SYS_UNKNOWN          (0U)
#endif
#ifndef SYS_GS_COMPUTER
#define SYS_GS_COMPUTER      (1U)
#endif
...
```

عدد منفی بدون پسوند `U` و داخل پرانتز نوشته می‌شود: `#define PARAM_MODE_NONE (-1)`. در فیلدهای تگ جدول پارامترها، که از نوع `uint32_t`اند، همان عدد به شکل بی‌علامتش ذخیره می‌شود (`-1` یعنی `0xFFFFFFFF`)؛ برای همین دو نامی که عددشان در ۳۲ بیت یکی شود، مثل `-1` و `4294967295`، هم خطای `DEF005` دارند.

`#ifndef` یعنی اگر فرم‌ور همین نام را جای دیگری تعریف کرده باشد، تعریف خود فرم‌ور به کار می‌رود و کامپایلر هشدار تعریف دوباره نمی‌دهد. در پروژه‌ی جدول پارامتر، نام سیستم‌ها در فایل جداگانه‌ی `nsrSystemNames.h` می‌آیند (بخش ۱۴.۸).

۱۱.۶ تگ‌های آزاد و توضیح آن‌ها

چهار تگ آزاد `Tag1` تا `Tag4` معنی ثابتی ندارند؛ هر پروژه خودش تعیین می‌کند هرکدام برای چیست. این معنی را در ⚙ **Project settings…** ← صفحه‌ی **Tags** بنویسید (بخش ۱۵.۸)؛ مثلاً برای `Tag1` متن `Ground station page`. از آن به بعد:

- فرم پارامتر کنار نام تگ، توضیحش را هم نشان می‌دهد: **Tag1 : Ground station page**.
- عنوان ستون همان تگ در جدول پارامترها توضیح را دارد (شکل‌های بخش ۶.۲).

- فایل `cg_parameter_list.h` بالای جدول پارامترها فهرستی از معنی تگ‌ها می‌نویسد، تا خواننده‌ی کد هم بداند هر تگ برای چیست.

```
/*
  What the free tags mean in this project:
    Tag1 : Ground station page
*/
extern const sParameterDescriptor ParameterList[PARAMETER_LIST_QTY];
```

تگی که توضیحی ندارد، همان نام خودش را دارد و در این فهرست نمی‌آید. توضیح فقط برای خواندن است و روی مقدار تگ‌ها، آفست‌ها یا کد اجرایی اثری ندارد. اگر معنی یک تگ در همه‌ی پروژه‌های شرکت یکی است، آن را در فایل `paramgen/templates/pl_project.json` بنویسید تا هر پروژه‌ی تازه از ابتدا آن را داشته باشد (بخش ۱۵.۷).

۱۲. تولید کد

۱۲.۱ دکمه‌ی Generate code

Generate code (یا `Ctrl+G`) کد پروژه را می‌سازد. این کار به این ترتیب انجام می‌شود:

۱. ساب‌لیست باز ذخیره می‌شود.

۲. فایل پروژه و همه‌ی ساب‌لیست‌های ساخت از دیسک خوانده، ادغام و بررسی می‌شوند.

۳. اگر خطایی نباشد، فایل‌ها در پوشه‌ی خروجی نوشته می‌شوند و پنجره‌ای فهرست آن‌ها را نشان می‌دهد (شکل ۱).

اگر پروژه خطا داشته باشد، هیچ فایلی نوشته نمی‌شود و کد قبلی روی دیسک دست‌نخورده می‌ماند؛ کد قدیمیِ درست بهتر از کد جدیدِ نیمه‌کاره است. در این حالت پیامی قرمز نمایش داده می‌شود و پنل **Problems** باز می‌شود.

ذخیره پیش از تولید باعث می‌شود ویرایشگر و خط فرمان همیشه دقیقاً یک کد بسازند، چون هر دو از همان فایل‌های روی دیسک می‌خوانند.

۱۲.۲ فایل‌های تولیدی

پروژه‌ی Smart data هفت فایل می‌نویسد. پیشوند `cg_` و بخش `parameter_list` نام‌ها را تنظیم‌های `file_prefix` و `base_name` فایل پروژه تعیین می‌کنند:

محتوا	فایل
تعداد مقدارها (`PARAMETER_LIST_QTY`)، ایندکس هر مقدار به شکل ماکرو، اندازه‌ی آرایه‌ها، اندازه‌ی کل هر تگ، و `#define` هر نام **System names** و **Parameter modes** با عددش.	`cg_parameter_list_defines.h`
یک `typedef struct` برای هر تایپ استراکچر، و برای هر اینام یک `typedef`، مقدارهایش و ماکروی `IS_VALID_` (بخش ۹.۵).	`cg_parameter_list_types.h`
شیء smart data هر پارامتر، و ساختن همه‌ی آن‌ها با بازه، مقدار پیش‌فرض و واحدشان. شیء پارامتر اینام همان‌جا مقدارهای مجازش را هم می‌گیرد.	`cg_smart_data.h` و `cg_smart_data.c`
جدول `ParameterList[]` که برای هر مقدار یک ردیف دارد، و تابع‌های `fParameterList_…` (فصل ۱۳). اگر پروژه برای تگ‌های آزاد توضیح نوشته باشد، فهرست آن‌ها هم بالای جدول می‌آید (بخش ۱۱.۶).	`cg_parameter_list.h` و `cg_parameter_list.c`
فقط توضیح: جدول‌های متنی مقدارها، تگ‌ها و آفست حافظه‌ی هر مقدار. هیچ کدی ندارد.	`cg_parameter_list_table.h`

> **هشدار:** همه‌ی این فایل‌ها در سرآیندشان عبارت `DO NOT EDIT IT BY HAND` را دارند. هرگز دستی ویرایششان نکنید؛ با تولید بعدی بازنویسی می‌شوند. هر تغییری را در ساب‌لیست بدهید.

۱۲.۳ نام‌ها در کد

نام ماکرو و نام شیء هر مقدار از روی نام آن در ویرایشگر ساخته می‌شود. نام ماکرو با حروف بزرگ نوشته می‌شود و کلمه‌ها با _ از هم جدا می‌شوند:

شیء smart data	مقدار ماکرو	ماکروی ایندکس	در ویرایشگر
`Cg_SmartData_Airspeed`	`0`	`AIRSPEED`	`Airspeed`
`Cg_SmartData_MaxAltitude`	`1`	`MAX_ALTITUDE`	`MaxAltitude`
`Cg_SmartData_CellVoltage[0]`	`3`	`CELL_VOLTAGE_0`	`CellVoltage[0]`
`Cg_SmartData_Position.Lat`	`7`	`POSITION_LAT`	`Position.Lat`
`Cg_SmartData_Motor[1].Rpm`	`12`	`MOTOR_1_RPM`	`Motor[1].Rpm`

اندازه‌ی هر آرایه هم یک ماکرو دارد؛ مثلاً `CELL_VOLTAGE_ARRAY_SIZE` برابر `4`. پیشوندها با تنظیم‌های بخش `codegen` فایل پروژه تغییر می‌کنند (بخش ۱۵.۵). بخشی از `cg_parameter_list_defines.h` پروژه‌ی نمونه:

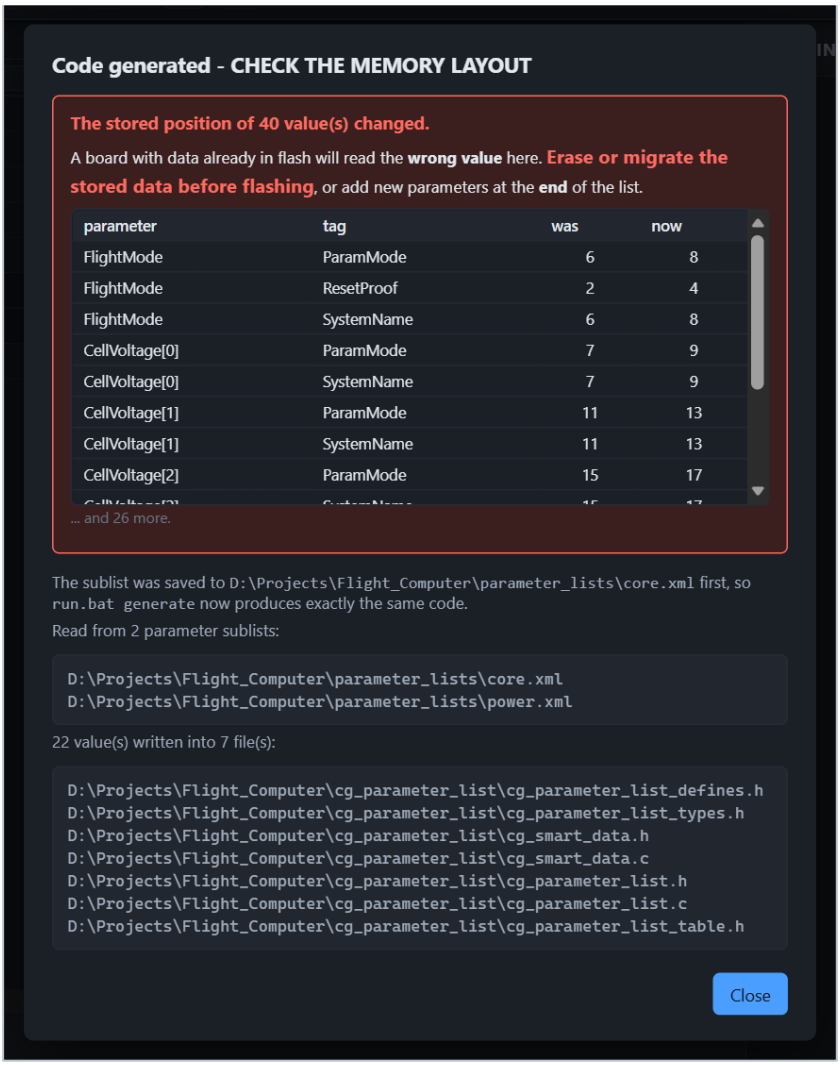
```
#define PARAMETER_LIST_QTY  (21U)  /*!< Number of enabled parameters */

#define AIRSPEED             (0U)   /*!< float32_t [m/s] */
#define MAX_ALTITUDE         (1U)   /*!< uint16_t [m] */
#define CELL_VOLTAGE_0       (3U)   /*!< float32_t (element 0 of 4) [V] */
#define POSITION_LAT         (7U)   /*!< float64_t [deg] */
#define MOTOR_0_RPM          (10U)  /*!< int16_t [rpm] */

#define CELL_VOLTAGE_ARRAY_SIZE  (4U)  /*!< Number of elements of 'CellVoltage' */
#define PARAMETER_LIST_TAG_RESET_PROOF_SIZE  (7U)
```

۱۲.۴ هشدار «چیدمان حافظه عوض شد»

هر بار که کد می‌سازید، ابزار نتیجه را با تولید قبلی (فایل `parameter_list.xml` کنار کد تولیدی) مقایسه می‌کند. اگر مقداری که قبلاً وجود داشت حالا جای دیگری از حافظه قرار گرفته باشد، بالای پنجره‌ی نتیجه هشداری قرمز نمایش داده می‌شود و عنوان پنجره **Code generated – CHECK THE MEMORY LAYOUT** می‌شود.



شکل ۲۲ — هشدار: یک پارامتر در وسط فهرست اضافه شده و جای ۴۰ مقدار در حافظه تغییر کرده است

چرا مهم است؟

آفست هر مقدار ماندگار به جایگاهش بستگی دارد: جمع اندازه‌ی همه‌ی مقدارهای قبل از آن که همان تگ را دارند. پس افزودن پارامتر در وسط فهرست، غیرفعال یا جابه‌جا کردن یک پارامتر، یا تغییر نوع داده یا اندازه‌ی آرایه‌ی آن، جای همه‌ی مقدارهای بعدی را تغییر می‌دهد. هیچ‌کدام از این‌ها در کد خطایی ایجاد نمی‌کند: کد تازه کامپایل می‌شود و فرمور اجرا می‌شود. اما بردی که از قبل داده‌ی ذخیره‌شده دارد، از آفست قدیمی می‌خواند و مقداری را می‌گیرد که حالا مال پارامتر دیگری است.

هشدار دقیقاً می‌گوید کدام پارامتر، در کدام تگ، از چه آفستی به چه آفستی رفته است. وقتی آن را دیدید، یکی از این کارها را انجام دهید:

- داده‌ی ذخیره‌شده را روی بردها پاک کنید.
- داده‌ی قدیمی را منتقل کنید: پیش از به‌روزرسانی فرمور آن را بخوانید و پس از آن دوباره بنویسید.
- پارامترهای تازه را به‌جای وسط فهرست، به انتهای آخرین ساب‌لیست ببرید.

> **پیشنهاد:** راه امن ساده است: پارامتر تازه را همیشه به انتهای آخرین ساب‌لیست ساخت اضافه کنید. در این صورت جای هیچ مقدار موجودی تغییر نمی‌کند و ابزار فقط یک یادداشت سبز نشان می‌دهد:
>
> **The list changed, but nothing moved.**

اولین تولید کد چیزی برای مقایسه ندارد و هشداری هم نمی‌دهد. همین هشدار در خط فرمان هم چاپ می‌شود.

۱۲.۵ تولید کد از خط فرمان

ویرایشگر راه معمول کار است؛ خط فرمان برای اسکریپت‌های ساخت و CI است. کد و بررسی‌ها دقیقاً همان‌اند:

کار	دستور
کد پروژه را می‌سازد. پروژه به این ترتیب پیدا می‌شود: پروژه‌ای که با `-c` نام برده شود؛ وگرنه پروژه‌ی پوشه‌ی جاری؛ وگرنه آخرین پروژه‌ای که در ویرایشگر باز بوده است.	`run.bat generate`
کد یک پروژه‌ی مشخص را می‌سازد؛ پوشه‌ی پروژه یا خود فایل پروژه را بدهید.	`run.bat generate -c D:\Projects\Flight_Computer`
فقط بررسی می‌کند و هیچ فایلی نمی‌نویسد.	`run.bat generate --check`
هشدارها را هم مانند خطا حساب می‌کند.	`run.bat generate -W`
پیام‌های اطلاعاتی را هم نشان می‌دهد.	`run.bat generate -v`
فقط مشکل‌ها را نشان می‌دهد.	`run.bat generate -q`
به‌جای ساب‌لیست‌های پروژه، از یک فایل ساب‌لیست دیگر کد می‌سازد.	`run.bat generate -m other.xml`
فهرست همه‌ی قاعده‌های بررسی را چاپ می‌کند.	`run.bat generate --list-rules`
نام و نسخه‌ی ابزار را چاپ می‌کند.	`run.bat generate --version`

کد خروج	معنی
`0`	موفق.
`1`	پروژه خطا دارد (یا با `-W` هشدار دارد) و کدی ساخته نشد.
`2`	خود ابزار نتوانست کار کند؛ مثلاً فایل پروژه خراب است یا پیدا نشد.

برای CI دستور `run.bat generate --check -W` مناسب است؛ کد خروج آن را مستقیم به خط لوله‌ی CI بدهید.

۱۳. استفاده از کد تولیدی در فرم‌ور

این فصل کوتاه، فقط کارهایی را نشان می‌دهد که برای به کار بردن کد تولیدی لازم است. مثال‌ها از پروژه‌ی نمونه‌ی Flight Computer هستند و همه برای قالب Smart data نوشته شده‌اند. قالب جدول پارامتر در فصل ۱۴ آمده است.

۱۳.۱ افزودن فایل‌ها به پروژه‌ی فرم‌ور

۱. پوشه‌ی خروجی پروژه (پیش‌فرض `cg_parameter_list`) و دو پوشه‌ی کتابخانه، یعنی `smart_data` و `parameter_descriptor`، را به مسیرهای include کامپایلر اضافه کنید.

۲. این چهار فایل را در ساخت فرم‌ور کامپایل کنید: `cg_smart_data.c`، `cg_parameter_list.c`، `smart_data.c` و `parameter_descriptor.c`. `smart_data.c` برای خواندن و نوشتن با ایندکس (بخش ۱۳.۳) تابع `fParameterList_GetByIndex` را از `cg_parameter_list.c` صدا می‌زند؛ پس این دو همیشه با هم کامپایل می‌شوند.

۳. در کد خودتان فقط `cg_parameter_list.h` را include کنید؛ این فایل بقیه‌ی فایل‌های لازم را خودش می‌آورد.

> **پیشنهاد:** اگر می‌خواهید کد مستقیم در پوشه‌ی پروژه‌ی فرم‌ور نوشته شود، مقدار `output.code_directory` را در فایل پروژه به همان پوشه تغییر دهید (بخش ۱۵.۳). به این ترتیب هر بار که کد می‌سازید، فرم‌ور نسخه‌ی تازه را دارد.

۱۳.۲ راه‌اندازی

تابع `fParameterList_Init` را یک بار در شروع برنامه و پیش از هر خواندن یا نوشتنی صدا بزنید. این تابع همه‌ی شیءهای smart data را با بازه، مقدار پیش‌فرض و واحدشان می‌سازد و در پایان، تابع `fParameterList_InitCompletedCallback` را صدا می‌زند. اگر همه‌چیز درست باشد، `0` برمی‌گرداند.

```
#include "cg_parameter_list.h"

int main(void) {

    if (fParameterList_Init() != 0U) {
        /* a smart data object could not be built */
    }
    ...
}
```

نسخه‌ی تولیدی `fParameterList_InitCompletedCallback` خالی است و به شکل weak تعریف شده است. اگر خودتان تابعی با همین نام بنویسید، به‌جای آن اجرا می‌شود. اینجا جای مناسبی برای بازگرداندن مقدارهای ماندگار (Reset Proof) است، چون همه‌ی پارامترها ساخته شده‌اند. اگر این تابع عددی غیر از صفر برگرداند، `fParameterList_Init` هم شکست می‌خورد.

```
uint8_t fParameterList_InitCompletedCallback(void) {

    /* every parameter exists now: restore the Reset Proof values here */
    return 0U;
}
```

۱۳.۳ خواندن و نوشتن یک مقدار

هر پارامتر یک شیء smart data با نام `Cg_SmartData_` و نام خودش دارد. برای هر نوع داده تابع جداگانه‌ای وجود دارد؛ مثلاً `fSmartData_F32_Read` برای `float32_t` و `fSmartData_U16_Read` برای `uint16_t`. به این ترتیب کامپایلر بررسی می‌کند که نوع شیء و نوع متغیر با هم بخوانند. فهرست کامل در پیوست الف آمده است.

```
float32_t airspeed = 0.0f;

/* read */
fSmartData_F32_Read(&Cg_SmartData_Airspeed, &airspeed);

/* write: refused when the value is outside Minimum .. Maximum */
airspeed = 42.5f;
if (fSmartData_F32_Write(&Cg_SmartData_Airspeed, &airspeed) != eSMART_DATA_OK) {
    /* the old value stays */
}
```

نوشتن مقداری خارج از بازه‌ی **Minimum** تا **Maximum** پذیرفته نمی‌شود: مقدار قبلی سر جایش می‌ماند و تابع `eSMART_DATA_ERROR_VALUE_OUT_OF_RANGE` برمی‌گرداند. مقداری که هنوز کسی ننوشته است، همان مقدار پیش‌فرض خوانده می‌شود. برای پارامتری از نوع اینام، عددی هم که جزو مقدارهای اینام نباشد به همین شکل رد می‌شود (بخش ۹.۵).

هر تابع خواندن و نوشتن یک نسخه‌ی ایندکسی هم دارد که به‌جای شیء، ایندکس مقدار را می‌گیرد؛ همان ماکروی ایندکس فایل `cg_parameter_list_defines.h`. این برای کدی مناسب است که پارامتر را با شماره‌اش می‌شناسد؛ مثلاً فرمانی که از ایستگاه زمینی می‌رسد:

```
fSmartData_F32_ReadByIndex(AIRSPEED, &airspeed);
fSmartData_F32_WriteByIndex(AIRSPEED, &airspeed);
```

نسخه‌ی ایندکسی دقیقاً همان کار نسخه‌ی معمولی را می‌کند، با همان بررسی بازه. اگر ایندکس از تعداد مقدارها بیشتر باشد، `eSMART_DATA_ERROR_NULL_POINTER` و اگر نوع مقدار آن ایندکس با نوع تابع نخواند (مثلاً `fSmartData_U16_ReadByIndex` روی یک `float32_t`)، `eSMART_DATA_ERROR_DATA_TYPE_NOT_SUPPORTED` برمی‌گرداند و چیزی نوشته نمی‌شود. این تابع‌ها مال کتابخانه‌ی `smart_data` هستند، نه کد تولیدی.

۱۳.۴ فیلد استراکچر و عنصر آرایه

استراکچر در C یک `struct` واقعی است و آرایه یک آرایه‌ی واقعی؛ پس فیلد با نامش و عنصر با ایندکسش در دسترس است. تعداد عنصرها همیشه از ماکروی `…_ARRAY_SIZE` می‌آید:

```
float64_t latitude = 0.0;
fSmartData_F64_Read(&Cg_SmartData_Position.Lat, &latitude);

int16_t rpm = 0;
for (uint16_t i = 0U; i < MOTOR_ARRAY_SIZE; i++) {
    fSmartData_I16_Read(&Cg_SmartData_Motor[i].Rpm, &rpm);
}
```

۱۳.۵ نوشتن همراه با اعلام تغییر

اگر بخشی از فرم‌ور (مثلاً لاگ یا لینک تله‌متری) باید از هر تغییر باخبر شود، از راه جدول پارامترها بنویسید. تابع‌های `fParameterDescriptor_Write…` همان بررسی بازه را انجام می‌دهند و اگر مقدار پذیرفته شد، تابع `fParameterDescriptor_ParameterWriteCompletedCallback` را با توصیف‌گر همان پارامتر صدا می‌زنند. این تابع را برنامه‌ی شما تعریف می‌کند.

```
const sParameterDescriptor *pAirspeed = fParameterList_GetByIndex(AIRSPEED);

if (fParameterDescriptor_WriteF32(pAirspeed, 42.5f) != ePARAMETER_DESCRIPTOR_OK) {
    /* refused: outside Minimum .. Maximum */
}
```

۱۳.۶ گشتن در همه‌ی پارامترها

آرایه‌ی `ParameterList` برای هر مقدار یک ردیف دارد، به ترتیب ایندکس‌ها. برنامه‌ای مثل لاگر یا منوی تنظیمات می‌تواند بدون دانستن نام پارامترها، همه را یکی‌یکی ببیند:

```
for (uint16_t index = 0U; index < fParameterList_GetQuantity(); index++) {

    const sParameterDescriptor *pParam = fParameterList_GetByIndex(index);

    if (pParam->TagLoggable != 0U) {
        /* pParam->LogId, pParam->pUserName, pParam->pSmartData ... */
    }
}
```

یک پارامتر را با نامش هم می‌توان پیدا کرد؛ همان نامی که ویرایشگر نشان می‌دهد، مثل `"Position.Lat"` یا `"CellVoltage[2]"`:

```
const sParameterDescriptor *pLat = fParameterList_GetByName("Position.Lat");
```

۱۳.۷ تابع‌ها و فیلدها

کار	تابع
همه‌چیز را می‌سازد؛ یک بار در شروع برنامه. `0` یعنی موفق.	`fParameterList_Init`
در پایان راه‌اندازی صدا زده می‌شود؛ نسخه‌ی خودتان را بنویسید.	`fParameterList_InitCompletedCallback`
تعداد مقدارها؛ همان `PARAMETER_LIST_QTY`.	`fParameterList_GetQuantity`
توصیف‌گر یک مقدار با ایندکسش؛ برای ایندکس نامعتبر `NULL`.	`fParameterList_GetByIndex`
توصیف‌گر یک مقدار با نامش؛ اگر پیدا نشود `NULL`.	`fParameterList_GetByName`
خواندن و نوشتن یک مقدار با ایندکسش؛ در کتابخانه‌ی `smart_data` (بخش ۱۳.۳).	`fSmartData_…_ReadByIndex` و `…_WriteByIndex`
اندازه‌ی کل ناحیه‌ی حافظه‌ی هر تگ، به بایت (بخش ۱۱.۴).	`fParameterList_GetResetProofSize` و مانند آن

هر ردیف جدول `ParameterList` یک `sParameterDescriptor` با این فیلدهاست:

محتوا	فیلد
نام مقدار در ویرایشگر؛ مثلاً `Motor[1].Rpm`.	`pName`
نام نمایشی همان مقدار (بخش ۷.۴). اگر نباشد، رشته‌ی خالی است و هرگز `NULL` نیست.	`pUserName`
نقش پارامتر: `ePARAMETER_TYPE_MONITORING`، `ePARAMETER_TYPE_SETTING` یا `ePARAMETER_TYPE_COMMAND`.	`Type`
نشانی شیء smart data همین مقدار.	`pSmartData`
شناسه‌ی لاگ؛ اگر مقدار لاگ‌شونده نباشد `0`.	`LogId`
مقدار پنج تگ اصلی.	`TagSystemName` تا `TagParamMode`
مقدار تگ‌های آزاد.	`Tag1` تا `Tag4`
توضیح؛ اگر نباشد، رشته‌ی خالی.	`pDescription`

۱۳.۸ نمونه‌ی کامل

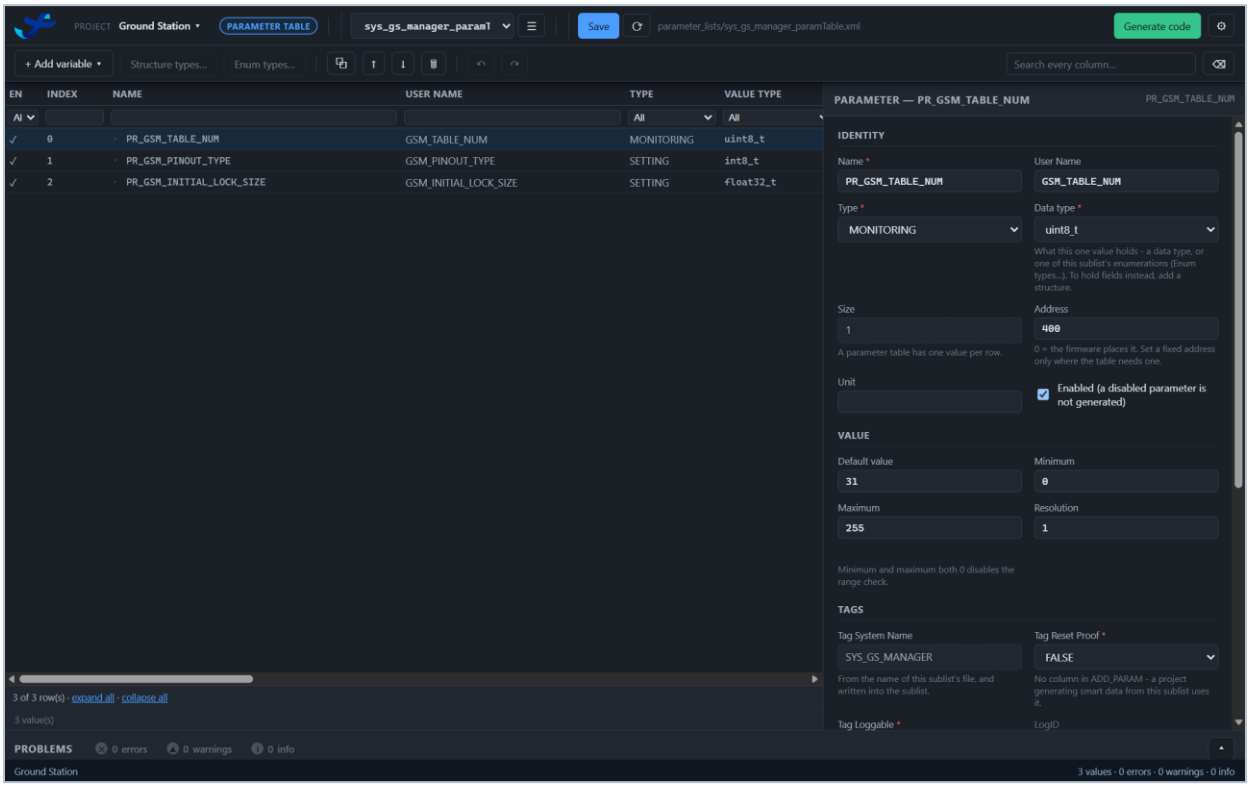
فایل `paramgen/docs/example/parameter_list_usage_example.c` همه‌ی این کارها را در هشت تابع کوتاه و با توضیح نشان می‌دهد؛ از راه‌اندازی تا خواندن مقداری که فقط برای یک خواننده‌ی مشخص تازه است. آزمون‌های ابزار این فایل را هر بار با کد تازه کامپایل و اجرا می‌کنند؛ بنابراین همیشه با کد تولیدی هماهنگ است.

۱۴. قالب جدول پارامتر (param_table)

۱۴.۱ این قالب برای چیست؟

فرمورهای قدیمی‌تر کتابخانه‌ی smart data ندارند و پارامترهایشان را در یک فایل هدر با سطرهای `ADD_PARAM(...)` تعریف می‌کنند. قالب جدول پارامتر دقیقاً همین فایل را می‌سازد. نوع قالب هنگام ساختن پروژه انتخاب می‌شود (بخش ۵.۲) و در فایل پروژه با `"format": "param_table"` ثبت می‌شود.

در پروژه‌ای با این قالب، کنار نام پروژه برچسب **PARAMETER TABLE** دیده می‌شود. کار با ویرایشگر همان است که در فصل‌های قبل آمد؛ فقط چند تفاوت دارد که در این فصل می‌خوانید.



شکل ۲۳ — پروژه‌ی Ground Station با قالب جدول پارامتر

تولید کد در این قالب سه فایل می‌سازد:

فایل	محتوا
`nsrParamTables.h`	سطرهای `ADD_PARAM` همه‌ی زیرسیستم‌ها. فرمور همین فایل را include می‌کند.
`nsrParamOffsets.h`	فقط توضیح: جای هر پارامتر در حافظه. هیچ کدی ندارد.
`nsrSystemNames.h`	نام زیرسیستم‌ها (`SYS_...`) با عددشان (بخش ۱۴.۸).

۱۴.۲ هر ساب‌لیست، یک زیرسیستم

در این قالب هر ساب‌لیست پارامترهای یک زیرسیستم را نگه می‌دارد و نام فایلش زیرسیستم را تعیین می‌کند: فایل `sys_gs_manager_paramTable.xml` مال زیرسیستم `SYS_GS_MANAGER` است.

- در پنجره‌ی ساختن ساب‌لیست، به‌جای کادر نام، فهرست **Sub-system** نمایش داده می‌شود که فقط زیرسیستم‌هایی را دارد که هنوز ساب‌لیستی ندارند. نام فایل زیر آن نوشته می‌شود و خودبه‌خود ساخته می‌شود.
- اگر همه‌ی زیرسیستم‌ها ساب‌لیست داشته باشند، زیرسیستم تازه را به فهرست **System names** اضافه کنید (⚙ ← **Drop-down lists...**)، با عددی که فرم‌ور برایش می‌شناسد، مثل `SYS_RADAR = 21`.
- **Tag System Name** در فرم فقط‌خواندنی است. مقدار آن از نام فایل می‌آید و ویرایشگر آن را در همه‌ی پارامترهای ساب‌لیست می‌نویسد.
- اگر نام فایل ساب‌لیستی با هیچ زیرسیستمی جور نباشد، خطای `PTB005` می‌گیرید. اگر تگ پارامتری با نام فایلش یکی نباشد (مثلاً پس از ویرایش دستی فایل)، هشدار `PTB008` نمایش داده می‌شود.

۱۴.۳ محدودیت‌ها

در ویرایشگر	چیزی که در این قالب نیست
User defined... پذیرفته نمی‌شود و **Structure types...** غیرفعال است. استراکچری که از قبل باشد، خطای `PTB001` می‌گیرد.	استراکچر
Size همیشه `1` و فقط‌خواندنی است (`PTB002`).	آرایه
خطای `PTB006`؛ به‌جای آن `uint8_t` با بازه‌ی `0` تا `1` به کار ببرید.	نوع `bool_t`
Enum types... غیرفعال است. پارامتری که از قبل نوع اینام داشته باشد، خطای `ENUM013` می‌گیرد.	اینام
در `ADD_PARAM` ستونی ندارند و در این قالب اثری ندارند. با این حال قابل ویرایش‌اند و در ساب‌لیست ذخیره می‌شوند، چون ممکن است همین ساب‌لیست در پروژه‌ای با قالب Smart data هم به کار برود. زیر هرکدام همین نکته یادآوری می‌شود.	**Tag Reset Proof**، **Tag Loggable**، **Tag Log Encryption** و **LogID**

۱۴.۴ آدرس

فرم پارامتر در این قالب کادری به نام **Address** دارد که مقدارش در ستون آدرس `ADD_PARAM` نوشته می‌شود. مقدار `0` یعنی فرم‌ور خودش جای پارامتر را تعیین می‌کند؛ این برای تقریباً همه‌ی پارامترها درست است. فقط جایی که جدول به آدرس ثابتی نیاز دارد، عدد بنویسید. آدرس عددی صحیح از `0` تا `4294967295` است (`PTB009`). پارامتری که تکثیر کنید، آدرس `0` می‌گیرد.

۱۴.۵ فایل nsrParamTables.h

برای هر زیرسیستم یک بلوک `#if` ساخته می‌شود و فرمور با تعریف `DESIRED_SUB_SYSTEM` تعیین می‌کند کدام بلوک‌ها را بخواهد. مقدارهای `PARAM_MODE_…` هم بالای فایل از روی فهرست **Parameter modes** تعریف می‌شوند؛ در این فایل عدد هر حالت جایگاه آن در فهرست است، با شروع از صفر (نه عدد پنجره‌ی **Drop-down lists…**). بخشی از فایل پروژه‌ی نمونه (سطرها برای جا شدن در صفحه شکسته شده‌اند):

```
#define PARAM_MODE_NONE             (0)
#define PARAM_MODE_USER             (1)
#define PARAM_MODE_READ_ONLY        (2)
...

#if DESIRED_SUB_SYSTEM == SYS_GS_MANAGER || DESIRED_SUB_SYSTEM == SYS_ALL
ADD_PARAM(uint8_t, TYPE_UINT8, PR_GSM_TABLE_NUM, "GSM_TABLE_NUM",
          SYS_GS_MANAGER, 0, 31, 0, 255, 1, PARAM_MODE_READ_ONLY, 400,
          "Returns the number of tables.")
ADD_PARAM(int8_t, TYPE_INT8, PR_GSM_PINOUT_TYPE, "GSM_PINOUT_TYPE",
          SYS_GS_MANAGER, 1, 0, -1, 3, 1, PARAM_MODE_USER, 0,
          "Which pin-out the board uses.")
...
#endif
```

هر ستون `ADD_PARAM` از این فیلدها می‌آید:

از کجا می‌آید	ستون
Data type. دو نوع اعشاری با نام C نوشته می‌شوند: `float` و `double`.	data type
از نوع داده؛ مثل `TYPE_UINT8`، `TYPE_FLOAT` و `TYPE_DOUBLE`.	TYPE_ENUM
Name	PR_NAME
User Name؛ اگر خالی باشد، همان **Name**.	gs name
زیرسیستم ساب‌لیست، از نام فایل.	SYS_NAME
ایندکس پارامتر در زیرسیستم خودش (بخش ۱۴.۶).	param index
بخش **Value** فرم. بازه‌ی خالی `0` نوشته می‌شود.	default value، min، max و resolution
Tag ParamMode؛ در این قالب قابل انتخاب است و پارامتر تازه `PARAM_MODE_NONE` می‌گیرد.	mode
Address (بخش ۱۴.۴).	address
Description	description

ستون‌های همه‌ی سطرهای فایل هم‌تراز نوشته می‌شوند تا جدول با چشم به‌راحتی خوانده شود.

۱۴.۶ ایندکس در این قالب

در فایل تولیدی، ایندکس هر زیرسیستم از صفر شروع می‌شود. جدول ویرایشگر هم وقتی یک ساب‌لیست باز است، همین شماره را نشان می‌دهد. اما نمای ادغام‌شده (**— every sublist, merged —**) شماره‌ی کلی پروژه را نشان می‌دهد؛ یعنی جایگاه هر پارامتر در کل ساخت:

ایندکس در نمای ادغام‌شده	ایندکس در فایل و در نمای ساب‌لیست	ساب‌لیست	پارامتر
0	0	sys_gs_manager	PR_GSM_TABLE_NUM
1	1	sys_gs_manager	PR_GSM_PINOUT_TYPE
2	2	sys_gs_manager	PR_GSM_INITIAL_LOCK_SIZE
3	0	sys_rhs	PR_RHS_HEATER_ON
4	1	sys_rhs	PR_RHS_SET_POINT

۱۴.۷ فایل nsrParamOffsets.h

این فایل فقط توضیح است و include کردنش هیچ کدی نمی‌سازد. برای هر زیرسیستم نشان می‌دهد هر پارامتر در حافظه کجا قرار می‌گیرد. مقدارها بدون فاصله پشت سر هم چیده می‌شوند و هر زیرسیستم از صفر شروع می‌شود:

```
/* ---- SYS_GS_MANAGER -------------------------------------------------
     index  offset  size     type  parameter
         0       0     1  uint8_t  PR_GSM_TABLE_NUM
         1       1     1   int8_t  PR_GSM_PINOUT_TYPE
         2       2     4    float  PR_GSM_INITIAL_LOCK_SIZE

     3 parameter(s), 6 byte(s) in total
*/
```

۱۴.۸ فایل nsrSystemNames.h

سطرهای `#if DESIRED_SUB_SYSTEM == SYS_…` فایل جدول به نام زیرسیستم‌ها نیاز دارند. این فایل همه‌ی نام‌های فهرست **System names** را با عددی که در پنجره‌ی **Drop-down lists…** دارند تعریف می‌کند، و چیز دیگری در آن نیست:

```
#ifndef SYS_UNKNOWN
#define SYS_UNKNOWN          (0)
#endif
#ifndef SYS_GS_COMPUTER
#define SYS_GS_COMPUTER      (1)
#endif
...
```

عدد منفی هم به همین شکل نوشته می‌شود؛ مثل `#define SYS_TEST (-5)`.

هر نام داخل `#ifndef` است: اگر فرم‌ور خودش همان نام را تعریف کرده باشد، تعریف فرم‌ور به کار می‌رود. فایل جدول آن را include نمی‌کند؛ هر جا فرم‌ور به این نام‌ها نیاز دارد، `#include "nsrSystemNames.h"` را بنویسد.

۱۴.۹ هشدار تغییر شماره

ایستگاه زمینی و فرم‌ور هر پارامتر را با شماره‌اش می‌خواهند. اگر شماره‌ی پارامتری که از قبل وجود داشت تغییر کند، مثلاً چون پارامتری پیش از آن اضافه یا غیرفعال شده است، پنجره‌ی نتیجه‌ی تولید کد هشداری قرمز نشان می‌دهد. این هشدار می‌گوید شماره‌ی چند پارامتر تغییر کرده است و آن‌ها را با شماره‌ی قبلی و تازه‌شان فهرست می‌کند.

> **پیشنهاد:** پارامتر تازه را همیشه به انتهای ساب‌لیست زیرسیستم خودش اضافه کنید. همین را دو سطر هشدار بالای فایل تولیدی هم یادآوری می‌کنند.

۱۵. فایل تنظیمات پروژه (pl_project.json)

۱۵.۱ نگاه کلی

فایل `pl_project.json` هنگام ساختن پروژه ساخته می‌شود (می‌توانید بعداً نامش را عوض کنید؛ بخش ۵.۳). همه‌ی تنظیم‌های آن را می‌توانید از پنجره‌ی ⚙ ← **Project settings…** تغییر دهید (بخش ۱۵.۸)؛ ساب‌لیست‌ها را پنجره‌ی ☰ هم در همین فایل می‌نویسد. اگر فایل را با یک ویرایشگر متن تغییر دادید، در ویرایشگر ⟳ را بزنید تا دوباره خوانده شود.

این فایل JSON معمولی است، با دو آسان‌گیری: می‌توانید با `//` توضیح بنویسید و پس از آخرین عضو یک فهرست ویرگول بگذارید. وقتی ویرایشگر فهرست ساب‌لیست‌ها را بازنویسی می‌کند، توضیح‌های شما می‌مانند. کلیدی که ابزار نمی‌شناسد خطا نیست، ولی در کنسول درباره‌اش هشدار داده می‌شود؛ معمولاً یعنی نام کلید غلط تایپ شده است.

فایل پروژه‌ی نمونه‌ی Flight Computer:

```
{
  "project": { "name": "Flight Computer", "description": "" },
  "inputs": {
    "model_files": [ "core.xml", "power.xml" ],
    "model_directory": "parameter_lists"
  },
  "output": {
    "format": "smart_data",
    "code_directory": "cg_parameter_list",
    "full_parameter_list_file": "cg_parameter_list/parameter_list.xml",
    "file_prefix": "cg_",
    "base_name": "parameter_list"
  },
  "style": { "company": "FAS", "indent": "    " },
  "codegen": {
    "emit_description": true,
    "emit_unit": true,
    "emit_generation_info": true,
    "emit_timestamp": true,
    "index_define_prefix": "",
    "offset_define_prefix": "PARAMETER_LIST_",
    "smart_data_prefix": "Cg_SmartData_",
    "struct_type_prefix": "sCgStruct_",
    "descriptor_symbol": "ParameterList",
    "quantity_define": "PARAMETER_LIST_QTY",
    "extra_includes": []
  },
  "validation": { "treat_warnings_as_errors": false, "require_unit": false },
  "tag_descriptions": { "Tag1": "Ground station page" }
}
```

همه‌ی مسیرها نسبت به پوشه‌ی پروژه (پوشه‌ای که `pl_project.json` در آن است) خوانده می‌شوند، مگر اینکه مسیر کامل بنویسید.

۱۵.۲ بخش‌های project و inputs

کلید	معنی
`project.name`	نام پروژه، که روی دکمه‌ی **Project** ▾ و در فهرست پروژه‌های اخیر دیده می‌شود. اگر خالی باشد، نام پوشه به کار می‌رود.
`project.description`	توضیح آزاد.
`inputs.model_files`	ساب‌لیست‌های ساخت، به ترتیب. این فهرست را پنجره‌ی ☰ می‌نویسد. نام تنها (مثل `core.xml`) یعنی فایلی در `model_directory`.
`inputs.model_directory`	پوشه‌ی ساب‌لیست‌ها؛ پیش‌فرض `parameter_lists`.

۱۵.۳ بخش output

کلید	معنی
`format`	`smart_data` یا `param_table` (فصل ۱۴). این مقدار تعیین می‌کند چه کدی ساخته شود و چه چیزهایی مجاز باشد.
`code_directory`	پوشه‌ی کد تولیدی؛ پیش‌فرض `cg_parameter_list`. برای نوشتن مستقیم کد در پروژه‌ی فرمور، همین را تغییر دهید. مقدار خالی یعنی خود پوشه‌ی پروژه.
`full_parameter_list_file`	نسخه‌ای از کل فهرست ادغام‌شده که پس از هر تولید نوشته می‌شود. تولید بعدی با همین فایل مقایسه می‌کند تا جابه‌جایی حافظه را تشخیص دهد (بخش ۱۲.۴).
`file_prefix`	پیشوند نام فایل‌های تولیدی؛ پیش‌فرض `cg_`.
`base_name`	بخش اصلی نام فایل‌ها؛ پیش‌فرض `parameter_list`. اگر خالی باشد، نام اولین ساب‌لیست به کار می‌رود.

مهم: اگر `full_parameter_list_file` را پاک کنید، تولید بعدی چیزی برای مقایسه ندارد و هشدار جابه‌جایی حافظه نمی‌دهد. این فایل را همراه پروژه نگه دارید.

نکته: اگر `code_directory` را تغییر دهید، فایل‌هایی که قبلاً ساخته شده بودند در جای قبلی می‌مانند و دیگر به‌روز نمی‌شوند. ابزار هر نسخه‌ی جامانده داخل پوشه‌ی پروژه را با هشدار `OUT001` نشان می‌دهد تا اشتباهی از آن استفاده نکنید. آن فایل‌ها را دستی پاک کنید.

۱۵.۴ بخش style

کلید	معنی
`company`	نام شرکت در سطرهای Copyright فایل‌های تولیدی؛ پیش‌فرض `FAS`.
`indent`	تورفتگی کد تولیدی؛ پیش‌فرض چهار فاصله.
`snippet_file`	اختیاری: قالب دیگری برای سرآیند و پایان فایل‌های C. اگر نباشد، قالب شرکت (`paramgen/templates/c.json`) به کار می‌رود.

۱۵.۵ بخش codegen

کلید	پیش‌فرض	معنی
`emit_description`	`true`	توضیح‌ها در کد تولیدی بیایند. با `false`، فیلد `pDescription` خالی می‌شود.
`emit_unit`	`true`	واحدها در کد تولیدی بیایند. با `false` رشته‌ی واحد خالی می‌شود و حافظه‌ی فلش کمتری مصرف می‌شود.
`emit_generation_info`	`true`	نام و نسخه‌ی ابزار در سرآیند فایل‌ها نوشته شود.
`emit_timestamp`	`true`	زمان تولید در سرآیند فایل‌ها نوشته شود. با `false`، دو تولید پیاپی از یک فهرست فایل‌های کاملاً یکسانی می‌سازند.
`index_define_prefix`	خالی	پیشوند ماکروهای ایندکس؛ مثلاً با `PL_` ماکروی `AIRSPEED` به `PL_AIRSPEED` تبدیل می‌شود.
`offset_define_prefix`	`PARAMETER_LIST_`	پیشوند ماکروهای اندازه‌ی تگ‌ها.
`smart_data_prefix`	`Cg_SmartData_`	پیشوند نام شیءهای smart data.
`struct_type_prefix`	`sCgStruct_`	پیشوند نام `typedef` تایپ‌های استراکچر.
`descriptor_symbol`	`ParameterList`	نام آرایه‌ی جدول پارامترها.
`quantity_define`	`PARAMETER_LIST_QTY`	نام ماکروی تعداد مقدارها.
`extra_includes`	خالی	هدرهای اضافه برای کد تولیدی؛ مثلاً هدری که نامی را که در تگ‌های آزاد نوشته‌اید تعریف می‌کند (بخش ۱۱.۲).

۱۵.۶ بخش validation

کلید	معنی
`treat_warnings_as_errors`	با `true`، هشدارها هم مانند خطا جلوی تولید کد را می‌گیرند؛ مانند گزینه‌ی `-W` خط فرمان.
`require_unit`	با `true`، خالی بودن **Unit** برای هر مقدار غیر از `bool_t` خطاست (`META002`).

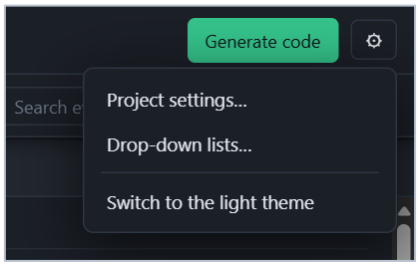
۱۵.۷ بخش tag_descriptions

معنی تگ‌های آزاد در این پروژه (بخش ۱۱.۶): برای هر تگی که توضیح دارد، یک کلید `Tag1` تا `Tag4` با متن توضیح. تگی که توضیح ندارد، در این بخش نیست؛ اگر توضیح تگی را در پنجره‌ی **Project settings…** پاک کنید، کلیدش از فایل برداشته می‌شود، و اگر آخرین توضیح باشد، خود بخش هم. کلیدی که از قبل با متن خالی (`"Tag1": ""`) در فایل مانده باشد، با اولین **Apply** برداشته می‌شود.

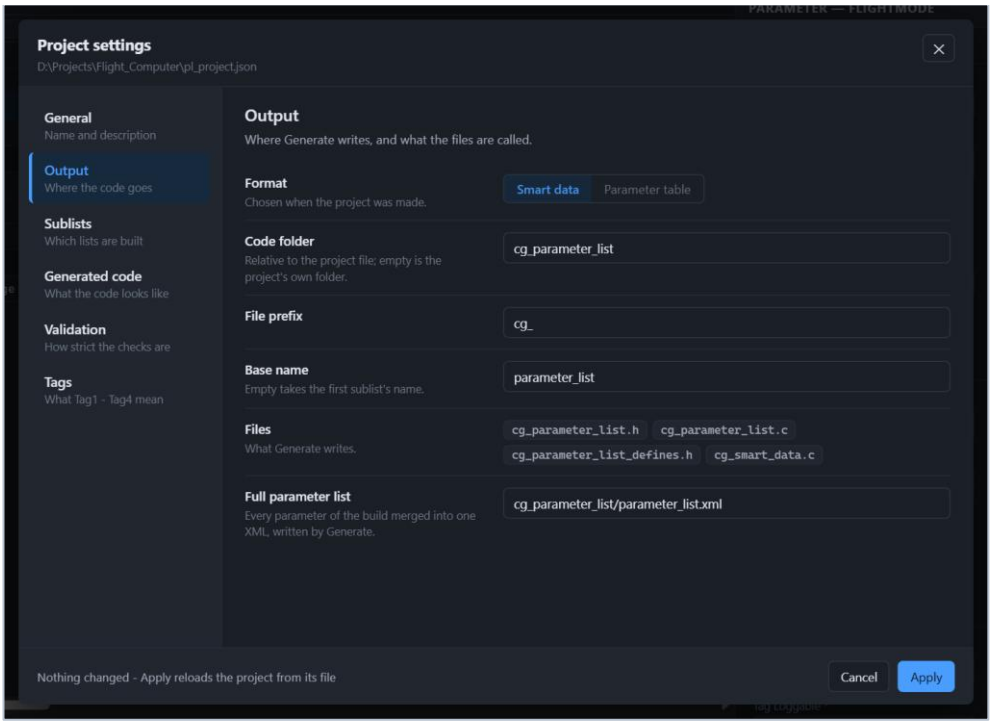
فایل `paramgen/templates/pl_project.json` جای هر چهار توضیح را دارد. توضیحی که آنجا بنویسید، در هر پروژه‌ی تازه‌ای که ساخته شود می‌آید؛ تگ‌هایی که آنجا خالی‌اند، در پروژه‌ی تازه اصلاً نوشته نمی‌شوند.

۱۵.۸ پنجره‌ی Project settings

⚙ ← **Project settings…** همه‌ی تنظیم‌های فایل پروژه را در یک پنجره نشان می‌دهد؛ هر بخش در یک صفحه که از فهرست سمت چپ انتخاب می‌شود.

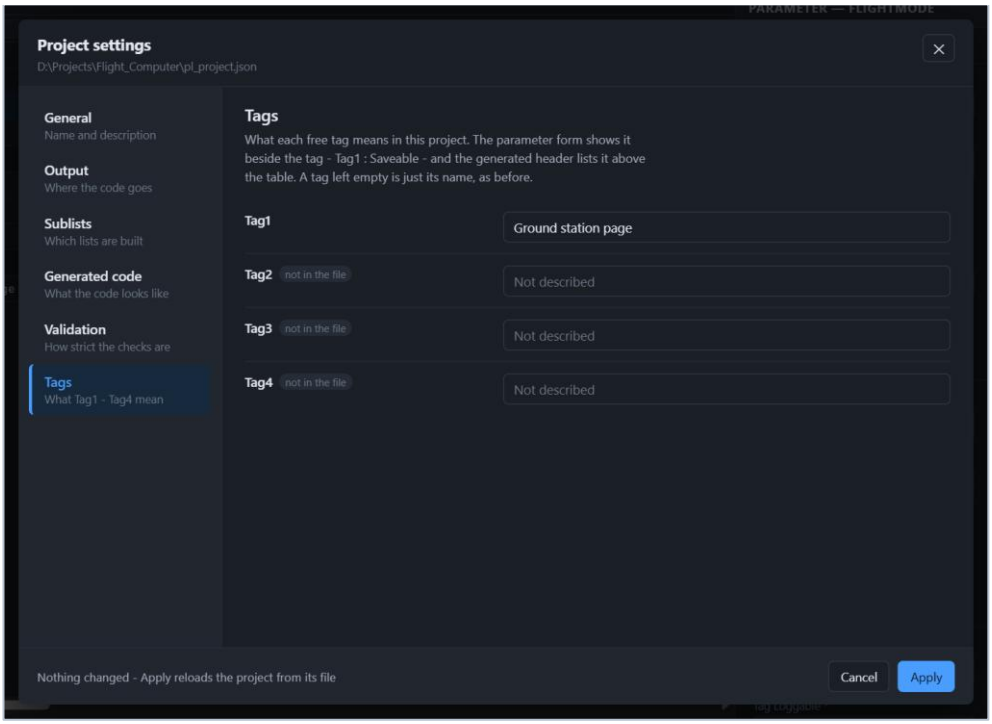


شکل ۲۴ — منوی ⚙



شکل ۲۵ — پنجره‌ی Project settings، صفحه‌ی Output

تنظیم‌ها	صفحه
نام و توضیح پروژه (بخش ۱۵.۲).	**General**
قالب (فقط نمایش)، پوشه‌ی کد، پیشوند و نام فایل‌ها با پیش‌نمایش نام فایل‌های تولیدی، و فایل فهرست کامل (بخش ۱۵.۳).	**Output**
پوشه‌ی ساب‌لیست‌ها و ساب‌لیست‌های ساخت به ترتیب؛ همان فهرستی که پنجره‌ی ☰ نشان می‌دهد (بخش ۱۵.۲).	**Sublists**
آنچه در کد می‌آید، تورفتگی، نام شرکت، هدرهای اضافه و نام‌ها در کد (بخش‌های ۱۵.۴ و ۱۵.۵).	**Generated code**
سخت‌گیری بررسی‌ها (بخش ۱۵.۶).	**Validation**
توضیح تگ‌های آزاد (بخش‌های ۱۱.۶ و ۱۵.۷).	**Tags**



شکل ۲۶ — صفحه‌ی Tags: Tag1 توضیح دارد و بقیه در فایل نیستند

- پنجره هنگام باز شدن خود فایل را می‌خواند، نه آنچه ویرایشگر قبلاً بارگذاری کرده است. کنار تنظیمی که در فایل نیست، برچسب **not in the file** هست و کادرش مقداری را نشان می‌دهد که ساخت کد به کار می‌برد؛ تا وقتی آن را تغییر ندهید، در فایل نوشته نمی‌شود.
- کنار هر تنظیمی که تغییر داده‌اید برچسب **changed** می‌آید و کنار نام صفحه‌اش نقطه‌ای آبی. پایین پنجره تعداد تغییرها نوشته می‌شود.
- **Apply** فقط تنظیم‌هایی را که تغییر داده‌اید، هرکدام در جای خودش، در فایل می‌نویسد؛ توضیح‌ها (//)، ترتیب کلیدها، پایان سطرها و بقیه‌ی فایل دست نمی‌خورند. سپس پروژه از همان فایل دوباره خوانده می‌شود.
- اگر ساب‌لیست باز تغییر ذخیره‌نشده داشته باشد، **Apply** چیزی نمی‌نویسد و می‌گوید اول ذخیره کنید، چون خواندن دوباره‌ی پروژه آن تغییرها را از بین می‌برد. اگر فایل پس از باز شدن پنجره جای دیگری تغییر کرده باشد هم چیزی نوشته نمی‌شود و پنجره فایل را همان‌طور که حالا هست نشان می‌دهد.
- مقدار نادرست (مثلاً پیشوندی با فاصله) نوشته نمی‌شود؛ پیام بالای پنجره دلیلش را می‌گوید و همان تنظیم را نشان می‌دهد.
- قالب خروجی (**Format**) فقط نمایش داده می‌شود؛ هنگام ساختن پروژه انتخاب می‌شود.

Cancel یا ✕ پنجره را بدون نوشتن چیزی می‌بندد. تغییرهای این پنجره با واگرد برنمی‌گردند، چون مستقیم در فایل پروژه نوشته می‌شوند.

۱۶. خطاها و هشدارها

۱۶.۱ پنل Problems

ویرایشگر پس از هر تغییر، کل پروژه را دوباره بررسی می‌کند و نتیجه را در پنل **Problems** پایین صفحه نشان می‌دهد. هر خط یک مشکل است: سطح آن، کد آن، شرح مشکل، جایی که مشکل در آن است، و در خط دوم، راه رفع آن.



شکل ۲۷ — پنل Problems با سه خطا

- با کلیک روی یک خط، پارامتر مربوط انتخاب و فرمش باز می‌شود؛ اگر در ساب‌لیست دیگری باشد، اول همان ساب‌لیست باز می‌شود. مشکلی که مال یک تایپ استراکچر است، پنجره‌ی همان تایپ را باز می‌کند.
- با کلیک روی شمارنده‌ی هر سطح (**errors**، **warnings** یا **info**)، خط‌های آن سطح پنهان یا دوباره نمایان می‌شوند.
- دکمه‌ی گوشه‌ی پنل آن را جمع یا باز می‌کند. وقتی مشکلی نباشد، پنل خودبه‌خود جمع می‌شود.

سطح	معنی	تولید کد
error	چیزی که کد درستی از آن ساخته نمی‌شود.	انجام نمی‌شود.
warning	کد ساخته می‌شود، ولی احتمالاً چیزی آن‌طور که می‌خواستید نیست.	انجام می‌شود؛ مگر با `-W` یا `treat_warnings_as_errors`.
info	فقط برای اطلاع.	انجام می‌شود.

۱۶.۲ پیام‌های رایج

هر پیام یک کد دارد. بخش اول کد نشان می‌دهد مشکل از چه نوعی است: `NAME` نام‌ها، `TYPE` و `ARR` نوع داده و آرایه، `RNG` مقدار و بازه، `TAG` تگ‌ها و شناسه‌ی لاگ، `TYP` و `STR` استراکچرها، `ENUM` اینام‌ها، `PTB` قالب جدول پارامتر، `DEF` لیست‌های کشویی، و `OUT` و `VAL` خود پروژه.

کد	سطح	معنی	راه رفع
`NAME001`	error	نام، نام معتبر C نیست.	فقط حرف انگلیسی، رقم و _؛ و بدون رقم در ابتدا.
`NAME002`	error	نام، کلیدواژه‌ی C یا C++ است.	نام دیگری انتخاب کنید.

راه رفع	معنی	سطح	کد
یکی را تغییر دهید.	یک نام دو بار به کار رفته است؛ شاید در دو ساب‌لیست.	error	NAME003
یکی را تغییر دهید (بخش ۸.۶).	دو نام به یک ماکرو یا نماد C تبدیل می‌شوند.	error	NAME004
Data type را انتخاب کنید.	نوع داده انتخاب نشده است.	error	TYPE001
Size را اصلاح کنید.	اندازه‌ی آرایه کمتر از 1 یا بیشتر از 65535 است.	error	ARR001 و ARR002
مقدار یا نوع داده را تغییر دهید.	مقداری در نوع داده جا نمی‌شود.	error	RNG001
جای دو مقدار را عوض کنید.	کمینه از بیشینه بزرگ‌تر است.	error	RNG002
مقدار پیش‌فرض یا بازه را اصلاح کنید.	مقدار پیش‌فرض خارج از بازه است.	error	RNG003
عدد صحیح بنویسید یا نوع اعشاری انتخاب کنید.	نوع صحیح، مقدار اعشاری گرفته است.	error	RNG017
مقدار پیش‌فرض بنویسید.	مقدار پیش‌فرض خالی است و 0 به کار می‌رود.	warning	RNG011
هر دو را پر کنید، یا هر دو را خالی بگذارید.	فقط یکی از کمینه و بیشینه پر شده است.	warning	RNG012 و RNG013
رزولوشن را بنویسید.	رزولوشن خالی است و پیش‌فرض نوع داده به کار می‌رود.	warning	RNG014
رزولوشن را اصلاح کنید.	رزولوشن صفر یا منفی است، یا از کل بازه بزرگ‌تر است.	warning	RNG015 و RNG016
خالی بگذارید.	برای bool_t بازه یا رزولوشن نوشته شده است.	warning	RNG010
تگ‌ها را پر کنید (بخش ۱۱.۱).	یکی از پنج تگ اصلی خالی است.	error	TAG001
مقداری از فهرست انتخاب کنید.	مقدار تگ در فهرست کشویی‌اش نیست.	error	TAG003
مقدار را اصلاح کنید (بخش ۱۱.۲).	مقدار تگ آزاد عدد معتبر یا نام C نیست.	error	TAG004 و TAG005
دکمه‌ی **Next free** را بزنید.	مقدار لاگ‌شونده شناسه‌ی لاگ ندارد.	error	TAG010
User Name را پر کنید.	مقدار لاگ‌شونده نام نمایشی ندارد.	error	TAG011
دکمه‌ی **Next free** را بزنید.	شناسه‌ی لاگ تکراری است.	error	TAG012
شناسه‌ی کوچک‌تری بنویسید.	شناسه‌ی لاگ از 65535 بزرگ‌تر شده است.	error	TAG015
کاری لازم نیست.	شناسه‌ی لاگ برای مقداری که لاگ نمی‌شود نادیده گرفته می‌شود.	info	TAG014
در تایپ آن فیلدی روشن کنید.	استراکچر هیچ فیلد روشنی ندارد.	error	STR001
نام یکی را تغییر دهید.	دو ساب‌لیست تایپ استراکچر هم‌نام دارند.	error	TYP016

راه رفع	معنی	سطح	کد
تایپ را در همین ساب‌لیست تعریف کنید.	استراکچر از تایپ ساب‌لیست دیگری استفاده کرده است.	error	`TYP017`
اینام را در همین ساب‌لیست تعریف کنید.	پارامتر از اینام ساب‌لیست دیگری استفاده کرده است.	error	`ENUM011`
نوع داده‌ی عادی انتخاب کنید.	پروژه‌ی جدول پارامتر پارامتری از نوع اینام دارد.	error	`ENUM013`
یکی از مقدارها را انتخاب کنید (بخش ۹.۳).	مقدار پیش‌فرض جزو مقدارهای اینام نیست.	error	`ENUM015`
آن را به فهرست اضافه کنید یا نوع دیگری انتخاب کنید.	نوع داده در فهرست **Value types** نیست.	warning	`TYPE003`
Unit را پر کنید.	واحد خالی است (فقط با `require_unit`).	error	`META002`
نام را در **Drop-down lists…** اصلاح کنید.	مقداری در لیست‌های کشویی نام معتبر C نیست.	error	`DEF002`
به یکی عدد دیگری بدهید (بخش ۱۱.۵).	دو نام **System names** یا **Parameter modes** یک عدد دارند.	error	`DEF005`
عدد دیگری بنویسید.	عدد یک نام خارج از `-2147483648` تا `4294967295` است.	error	`DEF006`
بخش ۱۴.۲	نام فایل ساب‌لیست با هیچ زیرسیستمی جور نیست.	error	`PTB005`
بخش ۱۴.۲	تگ زیرسیستم پارامتر با نام فایل یکی نیست.	warning	`PTB008`
بخش ۱۴.۴	آدرس عدد صحیح معتبری نیست.	error	`PTB009`
بخش ۱۵.۳	نسخه‌ی قدیمی فایل‌های تولیدی جا مانده است.	warning	`OUT001`
پارامتر یا ساب‌لیست اضافه کنید.	پروژه هیچ پارامتر یا هیچ ساب‌لیستی ندارد.	error	`VAL001` و `VAL002`

فهرست کامل قاعده‌ها را دستور `run.bat generate --list-rules` چاپ می‌کند.

۱۷. تغییر شماره‌ی نسخه‌ی ابزار

ابزار یک شماره‌ی نسخه دارد که در منوی اصلی و در سرآیند همه‌ی فایل‌های تولیدی دیده می‌شود. اگر خود ابزار را تغییر می‌دهید یا نسخه‌ی تازه‌ای از آن را به همکاران می‌دهید، شماره‌ی نسخه را هم بالا ببرید تا روشن باشد هر کد با کدام نسخه ساخته شده است.

۱۷.۱ قالب شماره‌ی نسخه

شماره‌ی نسخه سه جفت رقم است که با نقطه از هم جدا شده‌اند؛ مثل `01.01.85`. هر بخش دقیقاً دو رقم دارد، پس `1.1.85` یا `01.01` پذیرفته نمی‌شود.

۱۷.۲ تغییر نسخه با set_version.bat

برای تغییر نسخه، فایل `set_version.bat` در پوشه‌ی `paramgen/bin` را به کار ببرید:

۱. ویرایشگر را ببندید و اگر منوی `run.bat` باز است، از آن خارج شوید.

۲. در پوشه‌ی `paramgen/bin` روی `set_version.bat` دوبار کلیک کنید. پنجره‌ای باز می‌شود که نسخه‌ی فعلی را نشان می‌دهد.

۳. نسخه‌ی تازه را بنویسید و Enter بزنید.

۴. پیام **Done** یعنی نسخه عوض شد. برای بستن پنجره یک کلید بزنید.

۵. `run.bat` را اجرا کنید. سطر اول منو باید نسخه‌ی تازه را نشان دهد.

۶. گزینه‌ی `3` (Tests) را اجرا کنید تا مطمئن شوید ابزار سالم است.

پنجره‌ی `set_version.bat` چنین است؛ در این مثال نسخه از `01.01.01` به `01.01.02` تغییر می‌کند:

```
  ParamGen is version 01.01.01 now.

  Type the new version as three pairs of digits, like xx.xx.xx
  Press Enter on its own to keep 01.01.01.

  New version: 01.01.02

  Done: ParamGen is now version 01.01.02 (it was 01.01.01).

  The run.bat menu and 'run.bat generate --version' show it right away.
  Generated files and saved sublists carry it from the next time they
  are generated or saved.

Press any key to continue . . .
```

- اگر بدون نوشتن چیزی Enter بزنید، نسخه تغییر نمی‌کند.
- اگر نسخه را در قالب درست ننویسید، پیامی دلیلش را می‌گوید و چیزی تغییر نمی‌کند.

- در خط فرمان، در پوشه‌ی `paramgen`، می‌توانید نسخه را مستقیم بدهید: `bin\set_version.bat 01.01.02`
- در لینوکس، در پوشه‌ی `paramgen` این دستور را اجرا کنید: `python3 bin/set_version.py 01.01.02`

۱۷.۳ شماره‌ی نسخه کجاها دیده می‌شود؟

نمونه	جا
`Parameter list code generator 01.01.02`	سطر اول منوی `run.bat`
`parameter_list_code_generator 01.01.02`	خروجی `run.bat generate --version`
`Generator : parameter_list_code_generator v01.01.02`	سرآیند هر فایل C تولیدی
`generatorVersion="01.01.02"`	ویژگی `generatorVersion` در فایل‌های XML ساب‌لیست‌ها

منو و دستور `--version` نسخه‌ی تازه را همان لحظه نشان می‌دهند. فایل‌های C تولیدی و فایل‌های XML ساب‌لیست‌ها آن را از تولید یا ذخیره‌ی بعدی می‌گیرند. تغییر نسخه هیچ اثری روی کد تولیدی ندارد، جز همین سطر سرآیند، و ساب‌لیست‌ها و پروژه‌های قبلی هم بدون هیچ تغییری باز می‌شوند.

> **نکته:** `set_version.bat` فقط سطر `__version__` در فایل `paramgen/src/pl_codegen/__init__.py` را تغییر می‌دهد؛ همه‌ی جاهای دیگر شماره را از همین سطر می‌خوانند. شماره‌ی نسخه‌ی روی جلد راهنماها را جداگانه به‌روز کنید.

> **هشدار:** ویژگی `schemaVersion` در فایل‌های XML را تغییر ندهید. آن شماره، نسخه‌ی ساختار فایل است، نه نسخه‌ی ابزار، و فقط وقتی تغییر می‌کند که خود ساختار فایل تغییر کند.

۱۸. میان‌برها و پرسش‌های رایج

۱۸.۱ میان‌برهای صفحه‌کلید

کار	کلید
ذخیره‌ی ساب‌لیست باز	`Ctrl+S`
تولید کد	`Ctrl+G`
افزودن پارامتر ساده	`Ctrl+N`
تکثیر پارامتر انتخاب‌شده	`Ctrl+D`
واگرد	`Ctrl+Z`
ازنو	`Ctrl+Y` یا `Ctrl+Shift+Z`
حذف پارامتر انتخاب‌شده (پس از پرسش)	`Delete`
بستن پنجره یا منوی باز	`Esc`

- وقتی در یک کادر متن می‌نویسید، `Ctrl+Z` و `Ctrl+Y` فقط نوشته‌ی همان کادر را برمی‌گردانند و کلید `Delete` فقط حرف حذف می‌کند.
- `Ctrl+S` و `Ctrl+G` اول مقدار کادری را که در آن هستید ثبت می‌کنند و بعد ذخیره یا تولید می‌کنند.
- تا وقتی پنجره‌ای جلوی صفحه باز است، میان‌برها کار نمی‌کنند.

۱۸.۲ پرسش‌های رایج

پرسش	پاسخ
پارامتر تازه را کجا اضافه کنم تا دردسری پیش نیاید؟	انتهای آخرین ساب‌لیست ساخت. در این صورت ایندکس و جای حافظه‌ی هیچ پارامتر موجودی تغییر نمی‌کند (بخش ۱۲.۴).
چرا دکمه‌های افزودن و ذخیره خاموش‌اند؟	یا پروژه هنوز ساب‌لیستی ندارد (بخش ۷.۱)، یا در نمای ادغام‌شده هستید که فقط‌خواندنی است (بخش ۱۰.۵).
چرا تکثیر، جابه‌جایی و حذف برای این ردیف خاموش‌اند؟	ردیف انتخاب‌شده یک عنصر آرایه یا فیلد استراکچر است. ردیف اصلی پارامتر را انتخاب کنید، یا فیلد را در تایپ استراکچر تغییر دهید (بخش‌های ۷.۳ و ۸.۳).
فایل `pl_project.json` را دستی تغییر دادم ولی ویرایشگر چیزی نشان نمی‌دهد.	دکمه‌ی ⟳ را بزنید تا پروژه دوباره خوانده شود.
چند نفر می‌توانند روی یک پروژه کار کنند؟	بله. بهتر است هر نفر ساب‌لیست خودش را داشته باشد. نام‌ها و شناسه‌های لاگ در کل پروژه بررسی می‌شوند؛ پس تداخل دو ساب‌لیست فوراً در پنل **Problems** دیده می‌شود.

پرسش	پاسخ
پنجره‌ی ویرایشگر را بستم؛ تغییرهایم ماند؟	فقط آنچه ذخیره کرده بودید. بستن پنجره درباره‌ی تغییرهای ذخیره‌نشده چیزی نمی‌پرسد (بخش ۳.۵).
کد قدیمی‌ام دست‌نخورده ماند و کد تازه ساخته نشد. چرا؟	پروژه خطا داشت. وقتی خطایی باشد، هیچ فایلی نوشته نمی‌شود. پنل **Problems** را ببینید.
ابزار را به رایانه‌ی دیگری ببرم؟	پوشه‌ی `paramgen` را کپی کنید. پروژه‌ها پوشه‌های جداگانه‌اند و جداگانه کپی می‌شوند. برای داشتن همان لیست‌های کشویی، فایل `paramgen/var/choices.json` را هم کپی کنید.
پس از رفتن به ساب‌لیست دیگر، واگرد کار نمی‌کند.	با عوض شدن ساب‌لیست باز، تاریخچه‌ی واگرد پاک می‌شود (بخش ۷.۸).
پارامتری می‌خواهم که فقط چند مقدار مشخص بگیرد.	یک اینام تعریف کنید (**Enum types…**) و آن را نوع داده‌ی پارامتر قرار دهید (فصل ۹).
چرا نمی‌توانم استراکچر یا آرایه بسازم؟	پروژه با قالب جدول پارامتر ساخته شده است که استراکچر، آرایه و اینام ندارد (بخش ۱۴.۳).

پیوست الف. نوع‌های داده

ابزار یازده نوع داده دارد. ستون «پسوند» بخشی از نام نوع smart data و تابع‌های آن است؛ مثلاً برای `uint16_t`، نوع `sSmartDataU16` و تابع‌های `fSmartData_U16_Read` و `fSmartData_U16_Write`.

پسوند	رزولوشن پیش‌فرض	بازه	بایت	نوع داده
`B8`	—	`false`، `true`	۱	`bool_t`
`U8`	`1`	`0 .. 255`	۱	`uint8_t`
`I8`	`1`	`-128 .. 127`	۱	`int8_t`
`U16`	`1`	`0 .. 65535`	۲	`uint16_t`
`I16`	`1`	`-32768 .. 32767`	۲	`int16_t`
`U32`	`1`	`0 .. 4294967295`	۴	`uint32_t`
`I32`	`1`	`-2147483648 .. 2147483647`	۴	`int32_t`
`U64`	`1`	`0 .. 18446744073709551615`	۸	`uint64_t`
`I64`	`1`	`-9223372036854775808 .. 9223372036854775807`	۸	`int64_t`
`F32`	`0.001`	`±3.4e38`	۴	`float32_t`
`F64`	`0.000001`	`±1.8e308`	۸	`float64_t`

وقتی **Minimum** و **Maximum** هر دو `0` یا هر دو خالی باشند، کل بازه‌ی نوع داده مجاز است. در قالب جدول پارامتر، `float32_t` و `float64_t` با نام‌های `float` و `double` نوشته می‌شوند و `bool_t` وجود ندارد.

پیوست ب. واژه‌نامه

واژه	معنی
پروژه	پوشه‌ای که `pl_project.json` در آن است، با ساب‌لیست‌ها و کد تولیدی‌اش.
ساب‌لیست (Sublist)	یک فایل XML با بخشی از پارامترهای پروژه.
ساخت (Build)	ساب‌لیست‌هایی که به ترتیب، با هم ادغام می‌شوند و کد از آن‌ها ساخته می‌شود.
نمای ادغام‌شده	نمایش همه‌ی ساب‌لیست‌های ساخت با هم، با ایندکس‌های نهایی؛ فقط‌خواندنی.
پارامتر	هر چیزی که در ساب‌لیست تعریف می‌کنید: یک مقدار ساده، یک آرایه یا یک استراکچر.
مقدار	یک خانه‌ی واقعی در حافظه و یک ردیف در جدول تولیدی. آرایه‌ی چهارتایی، چهار مقدار است.
عنصر	یکی از خانه‌های یک آرایه؛ مثل `CellVoltage[2]`.
استراکچر	پارامتری که چند فیلد را کنار هم نگه می‌دارد؛ در C یک `struct`.
تایپ استراکچر	شکل یک استراکچر: نام و فیلدهایش؛ در C یک `typedef struct`.
نوع داده	نوع C یک مقدار؛ مثل `uint8_t` یا `float32_t` (پیوست الف).
اینام (Enumeration)	نوع داده‌ای با چند مقدار نام‌دار؛ در C یک `typedef` از یک نوع صحیح و یک `#define` برای هر مقدار (فصل ۹).
ایندکس	جایگاه یک مقدار در جدول تولیدی، با شروع از صفر.
شناسه‌ی لاگ (LogID)	عددی که سامانه‌ی لاگ هر مقدار را با آن می‌شناسد.
تگ	برچسب دسته‌بندی هر پارامتر؛ مثل زیرسیستم یا ماندگار بودن (فصل ۱۱).
تگ آزاد	یکی از `Tag1` تا `Tag4`، که معنی‌اش را هر پروژه خودش تعیین می‌کند (بخش ۱۱.۶).
آفست	جای یک مقدار در ناحیه‌ی حافظه‌ی یک تگ، به بایت.
زیرسیستم	بخشی از سامانه که پارامتر به آن تعلق دارد؛ مقدار **Tag System Name**.
Smart data	قالب اصلی کد تولیدی: یک شیء smart data برای هر مقدار و یک جدول توصیف‌گر.
جدول پارامتر (param_table)	قالب قدیمی کد تولیدی: سطرهای `ADD_PARAM` (فصل ۱۴).
واگرد (Undo)	برگرداندن آخرین تغییر.